



Ai Red Team Assessment Project

Containerized Image Classifier

Attacks, Detection, and Remediation against a Local Serving Pipeline

Author: Kirubel M. Sahle

Date: 2026

Repository: github.com/piVentures/ai-redteam-project

Frameworks: MITRE ATLAS · NIST AI RMF 1.0 · OWASP API Top 10 (2023)

Scope: Local Docker environment only. No external systems targeted.

Classification: Technical assessment artifact. Not for redistribution.

Prepared as a technical assessment and reproducibility document

Abstract

This report documents a technical red team assessment of a containerized image classification pipeline built on PyTorch, FastAPI, and Docker. Two model-level attacks are demonstrated end to end: adversarial evasion (FGSM and PGD) and model extraction. A bonus membership inference attack is included. Nine system-level weaknesses in the containerized serving infrastructure are identified, categorized, and remediated according to MITRE ATLAS and NIST AI RMF 1.0 guidelines. A real-time detection and notification service was implemented and verified to alert a security operator within 35 seconds of attack onset. All metrics presented in the source assessment are empirically measured; none are estimated or predicted.

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	12
List of Tables	12
1. Executive Summary	12
1.1 What Was Assessed	13
1.2 What Was Found	13
1.3 What Was Fixed	13
1.4 What Remains	14
1.5 At-a-Glance Metrics	14
2. System Under Test	
2.1 Model	16
2.2 Serving Pipeline	18
2.3 Deployment Topology	22
2.4 Two Security Modes	23
3. Threat Model and Methodology	23
3.1 Assets in Scope	23
3.2 Adversary Capabilities	23
3.3 Scope and Assumptions	24
3.4 Methodology	27
4. Attack 1 — Evasion (FGSM and PGD)	27
4.1 Concept	27
4.2 Implementation	29
4.3 Results	32
4.4 Analysis	33
4.5 Evidence Files	34
5. Attack 2 — Model Extraction	34
5.1 Concept	34
5.2 Implementation	37
5.3 Results	39
5.4 Analysis	40
5.5 Evidence Files	40
6. Attack 3 — Membership Inference (Bonus)	40
6.1 Concept	40
6.2 Implementation	42
6.3 Results	43
6.4 Analysis	43
What a Stronger MIA Defense Would Need	44
Why the Honest Weak Result Matters	44
6.5 Evidence Files	44
7. System-Level Findings	45
7.1 Overview	46

7.2 Finding 01 — `/model-info` architecture leak	47
7.3 Finding 02—Verbose error traces	47
7.4 Finding 03 — No rate limit on the vulnerable API	48
7.5 Finding 04 — Auto-docs exposed	49
7.6 Finding 05 — API key in plaintext environment	49
7.7 Finding 06 — Container runs as root	50
7.8 Finding 07 — Volume mounts expose host paths	51
7.9 Finding 08 — No resource limits	51
7.10 Finding 09 — No explicit CORS policy	52
7.11 Finding Summary Table	53
8. Defenses, Detection, and Notification	63
8.1 Defense Taxonomy	63
8.2 Preventive—Adversarial Training	65
8.3 Preventive—Rate Limiting	66
8.4 Preventive—Confidence Rounding	67
8.5 Preventive—Input Validation	68
8.6 Preventive—Safe Model Loading	69
8.7 Preventive—Authentication	70
8.8 Detective—The Detector Service	78
8.9 Detective — Telegram Notifier	81
8.10 Corrective Controls — Designed but Scoped Out	81
8.10.1 IP Blocklist	81
8.10.2 Token Revocation Automation	81
8.10.3 Model Quarantine	82
8.11 Defense Coverage Summary	82
9. Framework Alignment	83
9.1 MITRE ATLAS Mapping	83
9.2 NIST AI RMF 1.0 Alignment	85
9.3 OWASP API Security Top 10 Alignment	85
9.4 Consolidated Coverage	86
Framework Coverage Summary	87
10. Reproducibility and Verification	87
10.1 Environment Prerequisites	88
10.2 Fresh-Clone Procedure	89
10.3 Verification and Test Scripts	90
10.4 Known Issues	91
10.5 Verification Reference	91
11. Limitations and Residual Risk	92
11.1 What Was Not Assessed	93
11.2 What Remains Vulnerable	94
11.3 Production Hardening Requirements	95
11.4 What the Assessment Does Claim	95
12. Lessons Learned and Conclusion	96

12.1 Lessons Learned	97
12.2 Conclusion	98
Appendix — Full Framework Tables, AI Usage Log Summary, and Repository Structure	99
Appendix	99
Appendix A — MITRE ATLAS Technique Details	100
Appendix B — NIST AI RMF 1.0 Function Mapping	101
Appendix C — OWASP API Security Top 10 Mapping	102
Appendix D — AI Usage Log Summary	102
Appendix E — Repository Structure	103
Appendix F — Attack and Defense Matrix	104

List of Figures

- Figure 1 — Five-layer clean architecture (Section 2)
- Figure 2 — Container topology (Section 2)
- Figure 3 — Domain model: SmallCNN and entities (Section 2)
- Figure 4 — Prediction request flow, vulnerable mode (Section 2)
- Figure 5 — Prediction request flow, hardened mode (Section 2)
- Figure 6 — Attack kill chain (Section 3)
- Figure 7 — Evasion attack sequence (Section 4)
- Figure 8 — Evasion accuracy vs. perturbation budget (Section 4)
- Figure 9 — Evasion four-panel adversarial example (Section 4)
- Figure 10 — Evasion baseline vs. robust comparison (Section 4)
- Figure 11 — Extraction attack sequence (Section 5)
- Figure 12 — Extraction query budget vs. fidelity (Section 5)
- Figure 13 — Extraction confidence histogram (Section 5)
- Figure 14 — MIA threshold curve, vulnerable (Section 6)
- Figure 15 — MIA threshold curve, hardened (Section 6)
- Figure 16 — End-to-end system sequence (Section 10)

List of Tables

- Table 1 — At-a-glance metrics (Section 1)
- Table 2 — Model specifications (Section 2)
- Table 3 — Container inventory (Section 2)
- Table 4 — Vulnerable vs. hardened comparison (Section 2)
- Table 5 — Threat actor capabilities (Section 3)
- Table 6 — Attack selection criteria (Section 3)
- Table 7 — Evasion accuracy sweep, baseline (Section 4)
- Table 8 — Evasion before/after at $\epsilon = 8/255$ (Section 4)
- Table 9 — Extraction fidelity metrics (Section 5)
- Table 10 — MIA endpoint comparison (Section 6)
- Table 11 — Nine system findings summary (Section 7)
- Table 12 — Defense summary (Section 8)
- Table 13 — Detector rules (Section 8)
- Table 14 — MITRE ATLAS coverage (Section 9)
- Table 15 — NIST AI RMF alignment (Section 9)
- Table 16 — OWASP API Top 10 alignment (Section 9)
- Table 17 — Reproducibility commands (Section 10)

- Table 18 — Verification assertions (Section 10)
- Table 19 — Limitations matrix (Section 11)

1. Executive Summary

This report presents a red team assessment of a containerized image classification pipeline. The target is a small convolutional neural network (SmallCNN) with 618,820 parameters, trained on a four-class subset of CIFAR-10, served over HTTP by FastAPI, and deployed in Docker on a local Linux host. The pipeline runs in two configurations from a single codebase: a deliberately vulnerable endpoint on port 8000 and a hardened endpoint on port 8001, switched by a `SECURITY_MODE` environment variable.

The assessment covers three model-level attacks—evasion, model extraction, and membership inference—and nine system-level weaknesses in the Docker pipeline. Each attack is paired with a remediation, and each remediation is verified by re-running the attack against the hardened endpoint. Every number in this report is measured and backed by a file in the `results/` directory.

1.1 What Was Assessed

The target system is organized in five clean architecture layers (domain, services, usecases, adapters, interfaces) and deployed as five

Docker containers:

- ``api-vuln`` on port 8000—the deliberately vulnerable target.
- ``api-hardened`` on port 8001—the same model with controls applied.
- ``detector`` — tails the prediction log and emits alerts.
- ``alerts-api`` on port 8002—exposes alerts over HTTP.
- ``telegram-notifier`` optional fifth container that delivers alerts to an operator.

Only the two API containers expose ports. The detector has no network exposure and consumes the shared logs/volume. The notifier mounts the `results/` directory read-only and writes nothing to disk.

1.2 What Was Found

Model-level vulnerabilities. At the standard perturbation budget of $\epsilon = 8/255$, FGSM reduces baseline accuracy from 0.88 to 0.28, and PGD reduces it to 0.18. At a higher budget of $\epsilon = 0.08$, FGSM drops accuracy to 0.05 and PGD to 0.00—the baseline model is fully defeated. On the per-sample demonstration, all three test images flip class under both FGSM and PGD. Model extraction reaches a substitute-model fidelity of 0.835 with a 2,000-query budget, three times the random baseline of 0.275. Membership inference produces a weak but measurable signal (0.5425 against a 0.50 majority baseline), indicating the model does not strongly memorize its training set.

System-level findings. Nine weaknesses were identified across the Docker pipeline. Four are metadata or reconnaissance leaks—the `/model-info` endpoint, verbose error traces, exposed API documentation, and a plaintext API key in the container environment. Three concern the container itself—running as root, writable volume mounts, and a permissive CORS configuration. One is a rate-limit gap that directly enables model extraction. One is a missing resource limit. Each finding is captured as structured JSON evidence with the vulnerable response, the hardened response, and the associated MITRE ATLAS technique.

1.3 What Was Fixed

Five of the nine system-level findings are remediated or mitigated:

#	Finding	Status
04	Auto-docs exposed	Disabled in hardened mode
05	Plaintext API key	Moved to <code>`.env`</code> , gitignored
06	The container runs as root.	Runs as <code>`appuser`</code> (uid 1000)
08	No resource limits	1 GiB memory, 1.0 CPU per API
09	Permissive CORS	Allowlist in hardened mode

Three model-level defenses were built and verified:

Adversarial training. At $\epsilon = 8/255$, FGSM accuracy improves from 0.05 to 0.33 and PGD accuracy from 0.00 to 0.18. The cost is four points of clean accuracy ($0.88 \rightarrow 0.84$). The improvement is meaningful but not complete — the robust model still fails 82% of PGD attacks at that budget.

Rate limiting. The hardened endpoint caps queries at 20 per 60 seconds per IP. Against the extraction attack, the limiter stopped the run at 20 of 30 queries with 4 rate-limited responses. Against the vulnerable endpoint, the same attack ran 2,000 queries without interruption.

Confidence rounding. The hardened endpoint returns only the top-1 class and a confidence rounded to two decimal places. The full softmax never leaves the process. MIA accuracy on the hardened endpoint remains within the binomial sampling error of the 0.50 baseline. A detection and notification layer watches the prediction log in real time. Three rules fire on sustained query volume, confidence sweeps, and adversarial-noise inputs. Each rule has a 60-second cooldown, which reduces alert volume on a sustained attack from approximately 50 alerts to 2. Alerts are delivered to a Telegram operator within 35 seconds.

1.4 What Remains

Four system findings are deliberately retained on the vulnerable endpoint:

#	Finding	ATLAS	Reason
01	<code>/model-info`</code> architecture leak	AML.T0007	Enables reconnaissance
02	Verbose error traces	AML.T0007	Demonstrates information leak
03	No rate limit	AML.T0024	Enables extraction at full speed
04	Auto-docs exposed	AML.T0007	Demonstrates surface enumeration

These are the demonstration surfaces for the attacks. Removing them would make the vulnerable endpoint hardened, which would defeat its purpose. They are documented as residuals in Section 11. Adversarial training is a trade-off, not a fix. At $\epsilon = 8/255$, PGD still succeeds on 82% of the robust model's inputs. The rate limiter is per-IP and in-memory, so a botnet or a container restart bypasses it. Confidence rounding reduces but does not eliminate the MIA signal.

Section 11 documents what production hardening would require.

1.5 At-a-Glance Metrics

Category	Metric	Baseline / Vulnerable	Hardened / Defense
Model	Parameters	618,820	618,820
Model	Clean test accuracy	0.8542	0.8555
Evasion	FGSM accuracy at $\epsilon = 8/255$	0.28	0.67
Evasion	PGD accuracy at $\epsilon = 8/255$	0.18	0.65
Evasion	FGSM accuracy at $\epsilon = 0.08$	0.05	—
Evasion	PGD accuracy at $\epsilon = 0.08$	0.00	—
Extraction	Soft-label fidelity (2,000 queries)	0.835	Rate-limited at 20/30 queries
Extraction	Random baseline	0.275	—
MIA	Best attack accuracy	0.5425	0.5575 (within noise)
System	Findings remediated	0 / 9	5 / 9
Detection	Rules	0	3
Detection	Alerts per sustained attack	—	~2 (after cooldown)

Category	Metric	Baseline / Vulnerable	Hardened / Defense
Notification	Time from attack to Telegram	—	< 35 s
Frameworks	ATLAS techniques covered	—	6
Frameworks	RMF functions covered	—	4

2. System Under Test

This section describes the target of the assessment. It covers the model itself, the serving pipeline that wraps it, the container topology it runs in, and the two security modes the same codebase exposes. Every detail here is reproducible from the repository.

2.1 Model

The model is a compact convolutional neural network designed for the constraints of a local 8 GB CPU-only host. It is deliberately small enough to train on a Google Colab free tier in under an hour, and deliberately large enough to be vulnerable to the attacks in scope.

Architecture

SmallCNN has three convolutional layers followed by two fully connected layers. Each convolutional block consists of a Conv2d with a 3×3 kernel, a ReLU activation, and MaxPool2d(2). The first layer has 32 output channels, the second 64, the third 128. After three poolings, a 32×32×3 input becomes 4×4×128. This flattens to 2,048 features, which the first fully connected layer projects to 256. A dropout of 0.3 sits between the two dense layers. The final layer produces four class logits.

Listing 1 — SmallCNN model definition (domain/model.py)

```
class SmallCNN(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 4 * 4, 256), nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

Shape Trace

The table below shows the shape of the tensor as it flows through the model. This trace explains why the first linear layer takes 2,048 inputs and makes the 128 * 4 * 4 constant in the source code verifiable.

Stage	**Input shape**	**Output shape**
Input	—	(N, 3, 32, 32)
Conv1 + pool	(N, 3, 32, 32)	(N, 32, 16, 16)
Conv2 + pool	(N, 32, 16, 16)	(N, 64, 8, 8)
Conv3 + pool	(N, 64, 8, 8)	(N, 128, 4, 4)
Flatten	(N, 128, 4, 4)	(N, 2048)
Linear1	(N, 2048)	(N, 256)

Stage	**Input shape**	**Output shape**
Dropout (0.3)	(N, 256)	(N, 256)
Linear2	(N, 256)	(N, 4)

Parameter Count

The model has 618,820 parameters. This is verified at runtime by the /model-info endpoint on the vulnerable API and by the training script.

Layer	**Type**	**Specifications**	**Parameters**
Conv1	Conv2d	3 → 32, 3×3, padding 1	896
Conv2	Conv2d	32 → 64, 3×3, padding 1	18,496
Conv3	Conv2d	64 → 128, 3×3, padding 1	73,856
Linear1	Dense	2,048 → 256	524,544
Linear2	Dense	256 → 4	1,028
Total			**618,820**

Dataset and Normalization

The model is trained on a four-class subset of CIFAR-10: airplane, automobile, bird, and cat. CIFAR-10 provides 5,000 training images and 1,000 test images per class, giving 20,000 training images and 4,000 test images in the four-class subset. A 90/10 split of the training set provides validation.

Inputs are normalized with mean = std = (0.5, 0.5, 0.5). This is not the standard CIFAR-10 statistics. The symmetric choice maps normalized values to exactly [-1, 1], which makes the adversarial perturbation clamp trivial and consistent across the attack and defense scripts. The choice is documented in domain/constants.py and recorded in model/artifacts/config.json.

Trained Metrics

Two models are produced by the pipeline:

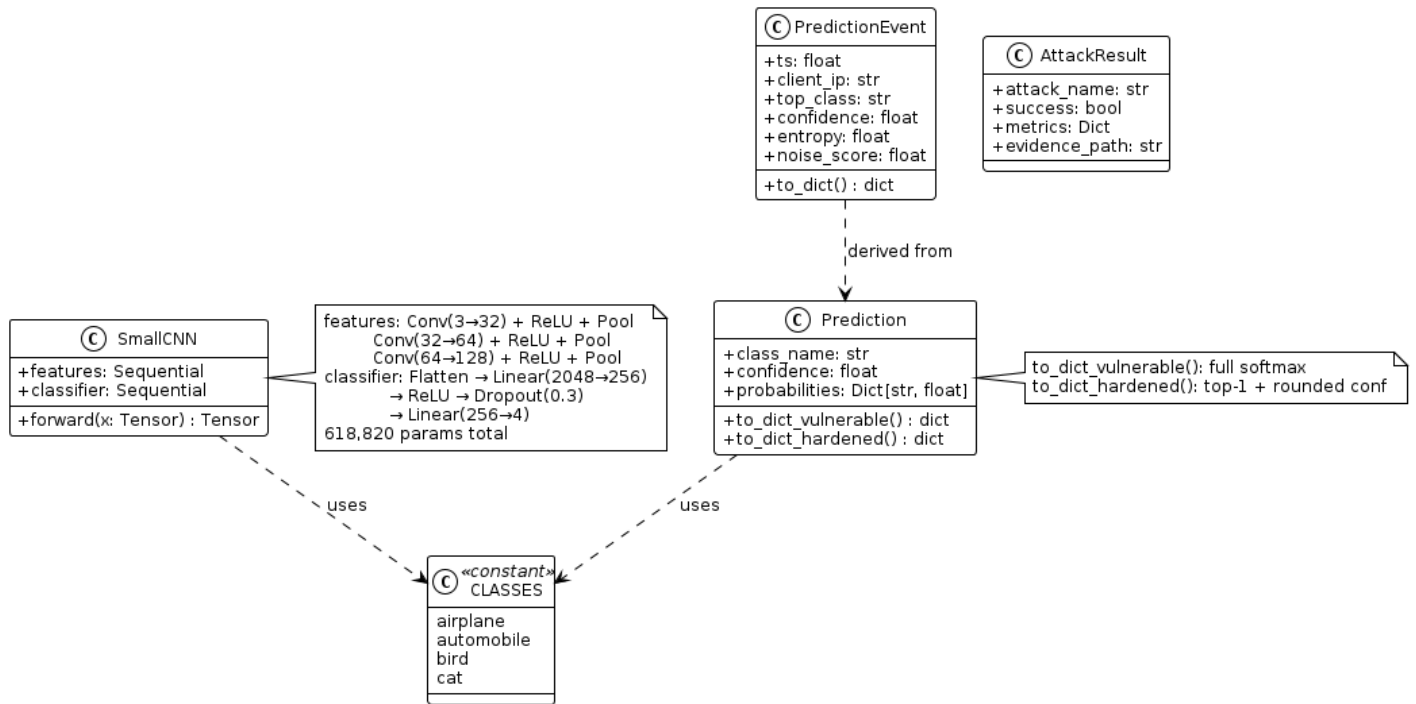
Model	**Clean val**	**Clean test**	**Purpose**
Baseline	0.8710	0.8542	Served by the vulnerable endpoint
Robust	0.8715	0.8555	Served by the hardened endpoint

The robust model is trained with PGD-5 adversarial examples. Its clean accuracy is essentially unchanged from the baseline because the epsilon warmup prevents the drop that standard adversarial training often causes.

Table 2 — Model specifications

Attribute	**Value**
Architecture	SmallCNN
Parameter count	618,820
Input shape	(3, 32, 32)
Output classes	4
Convolutional layers	3 (3→32→64→128)
Fully connected layers	2 (2048→256→4)
Dropout	0.3 between FC layers
Activation	ReLU
Normalization	mean = std = (0.5, 0.5, 0.5)

Attribute	**Value**
Dataset	CIFAR-10, 4-class subset
Training set size	20,000
Test set size	4,000
Baseline test accuracy	0.8542
Robust test accuracy	0.8555

Figure 3 — Domain Model**Figure 3 — Domain model: SmallCNN and entities**

2.2 Serving Pipeline

The model is wrapped in a FastAPI application served by Uvicorn. The serving pipeline is organized in five clean architecture layers with dependencies pointing inward. Every deliberate vulnerability lives in the adapter layer, because that is where the system meets untrusted input.

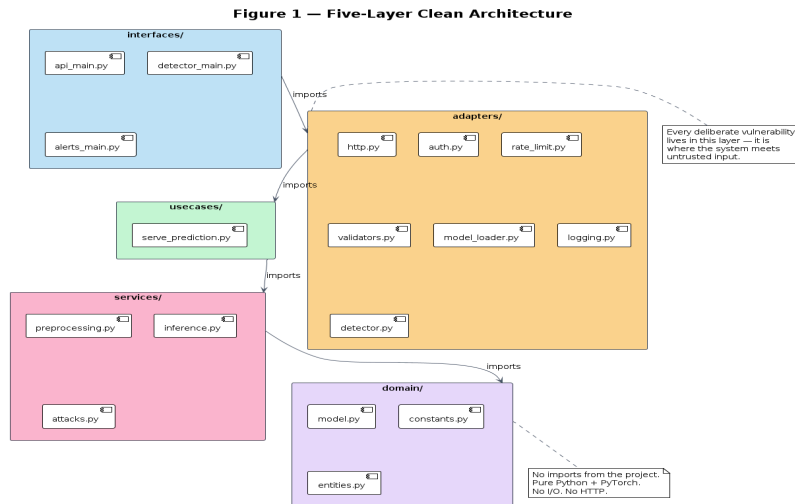


Figure 1 — Five-layer clean architecture

Layer Responsibilities

Layer	**Responsibility**	**Depends on**
`interfaces/`	Entry points: API, detector, alerts	adapters
`adapters/`	HTTP, auth, rate limit, validators, detector rules	usecases
`usecases/`	Orchestration: `serve_prediction`	services
`services/`	Preprocessing, inference, attack math	domain
`domain/`	SmallCNN, constants, entities	—

The dependency direction is enforced by the import graph, not by convention. usecases/ cannot import from adapters/, and domain/ imports nothing from the project. This makes the layers testable in isolation and makes the boundaries real.

Serving Framework

FastAPI 0.111.0 with Uvicorn 0.30.1. FastAPI was chosen for three reasons: it is AI-native, it has minimal boilerplate, and its default behavior—auto-generated documentation—is itself one of the findings in Section 7.5.

Runtime

Python 3.10 slim with CPU-only PyTorch 2.1.2+cpu. The CPU-only build is used because the target host has no GPU and because the CPU build is smaller and faster to install inside a Docker image.

Request Path

When a client POSTs an image to /predict, the following happens in order:

1. In hardened mode, `verify_api_key` checks the `x-api-key` header.
2. In hardened mode, `check_rate_limit` enforces 20 requests per 60 seconds per IP.
3. `file.read()` loads the raw bytes of the uploaded image.
4. In hardened mode, `validate_image` checks size, format, dimensions, and a noise heuristic.
5. `serve_prediction` orchestrates preprocessing, inference, and event logging.
6. The response is serialized differently per mode: full softmax in vulnerable, top-1 with rounded confidence in hardened.

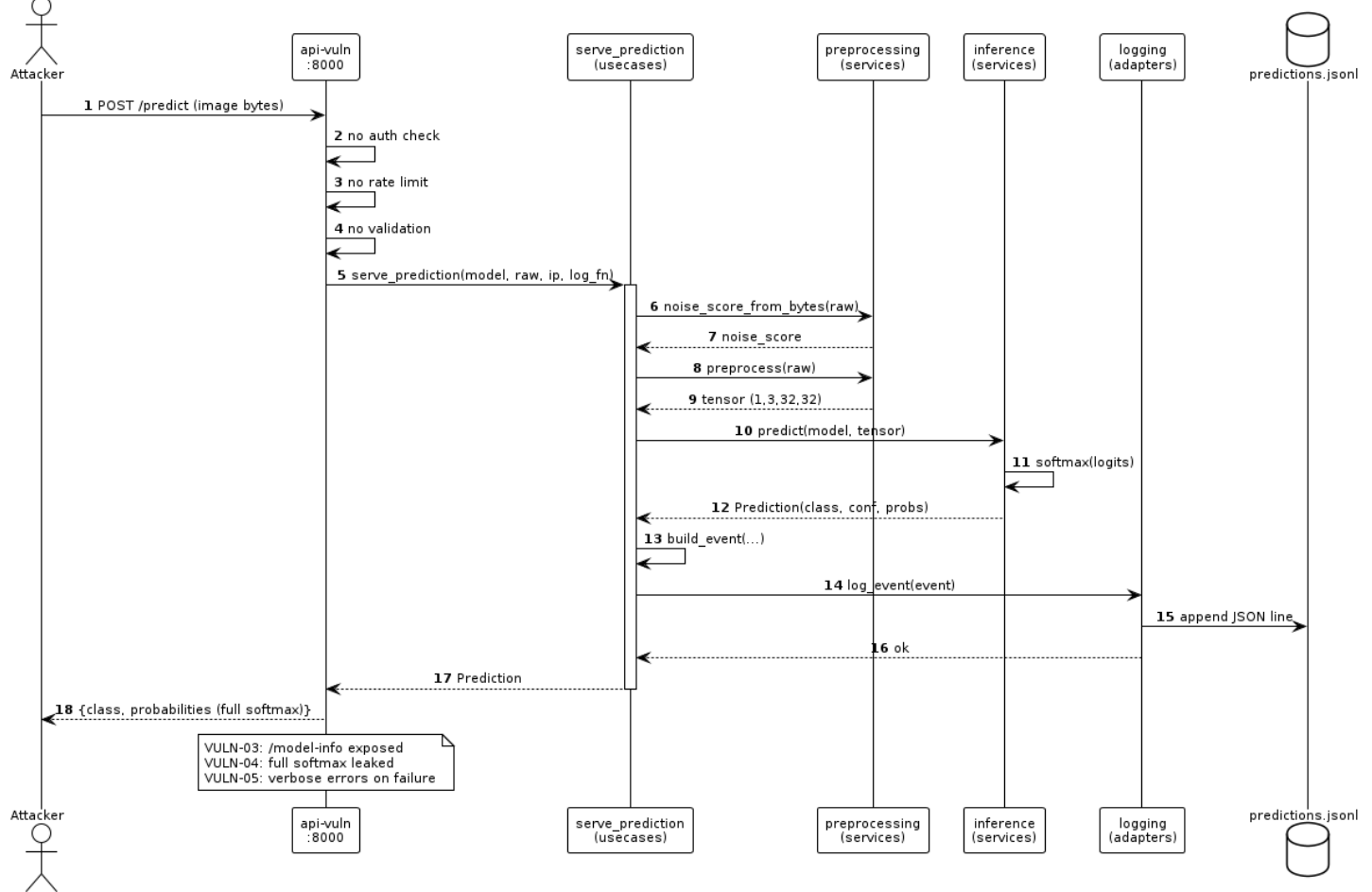
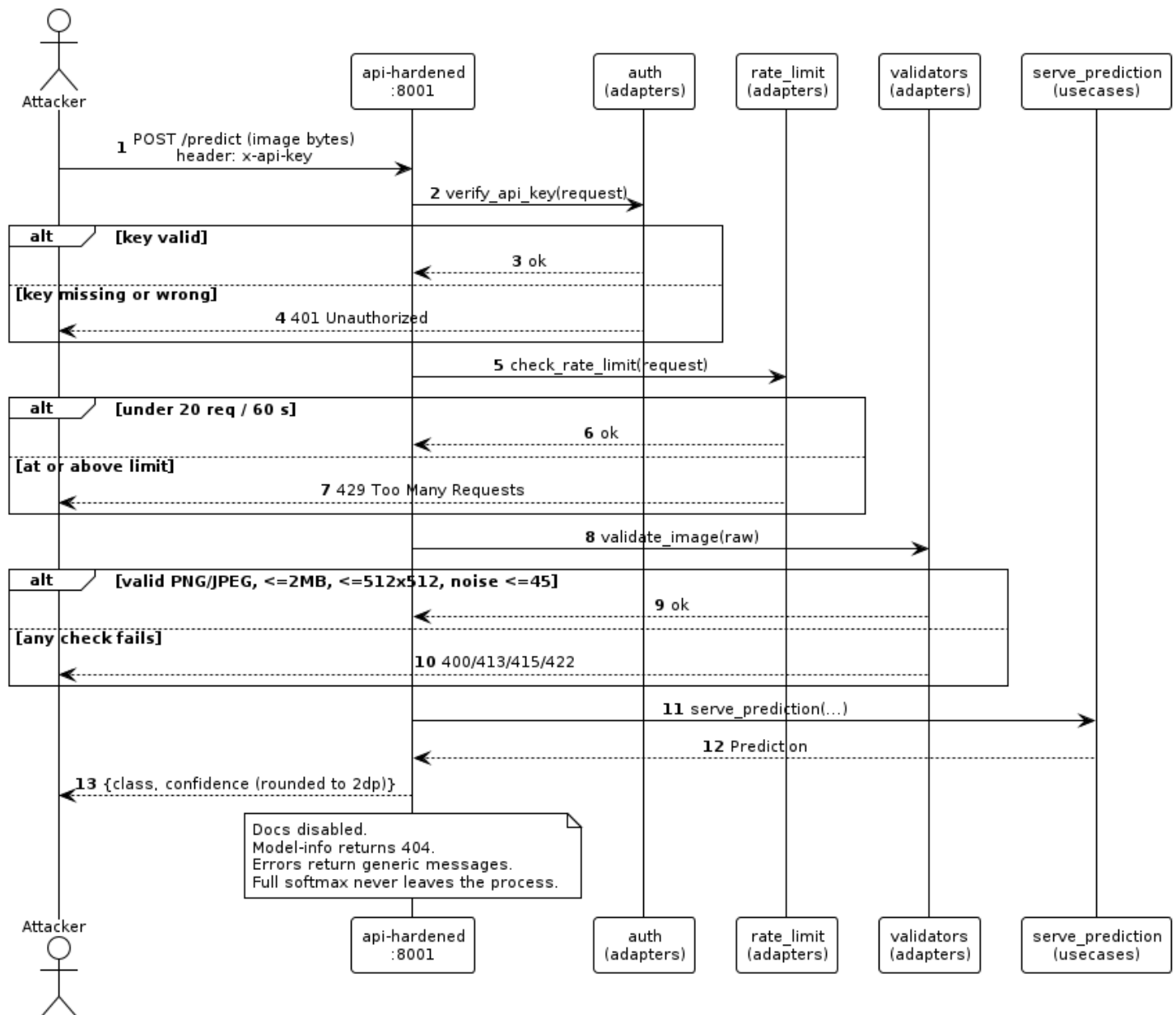
Figure 4 — Prediction Request (Vulnerable Mode)**Figure 4 — Prediction request flow, vulnerable mode**

Figure 5 — Prediction Request (Hardened Mode)**Figure 5 — Prediction request flow, hardened mode**

2.3 Deployment Topology

The pipeline runs as five Docker containers managed by Docker Compose.

Figure 2 — Container Topology

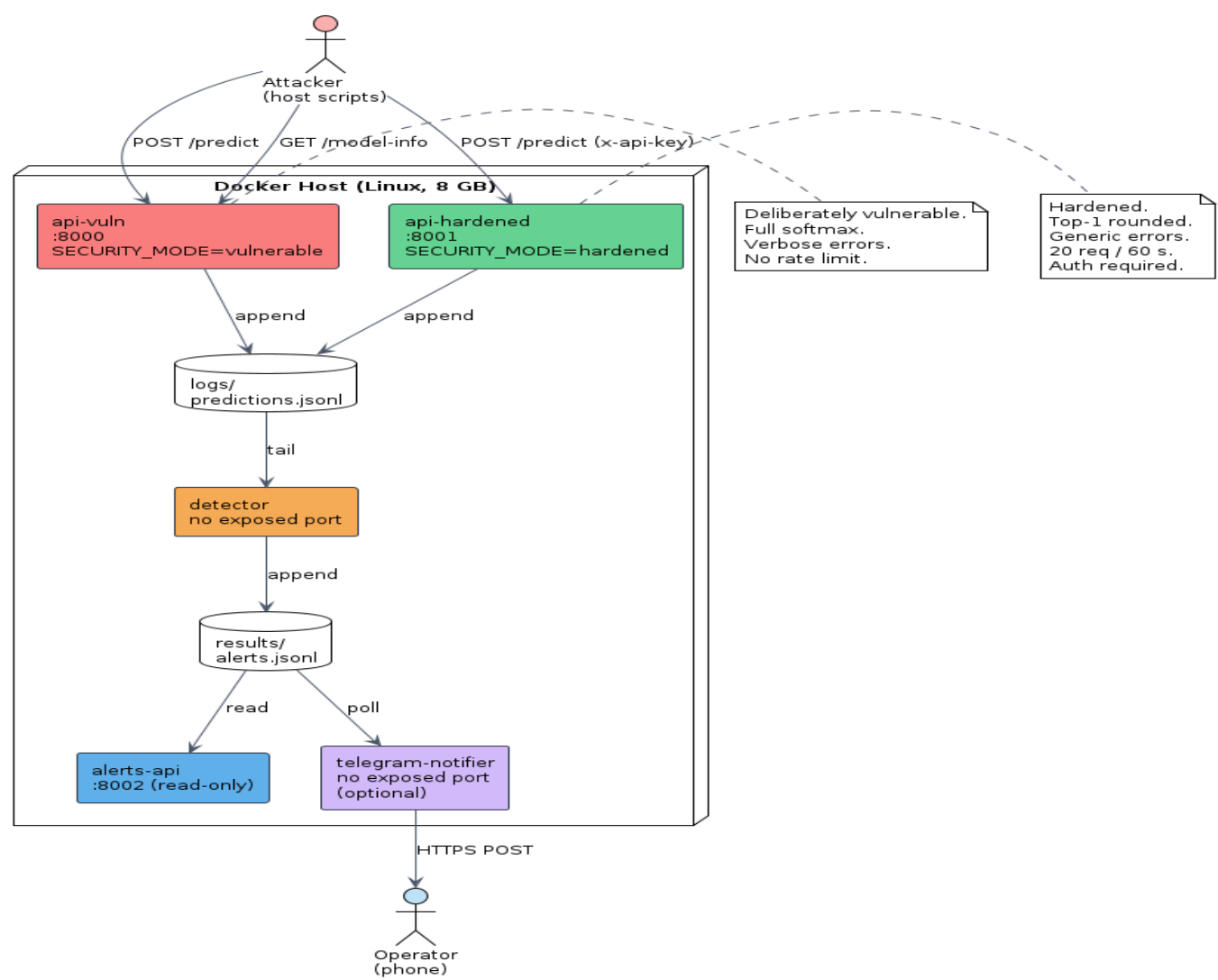


Figure 2 — Container topology

Table 3 — Container inventory

Container*	**Port	**Mode**	**Memory**	**CPU**	**User**	**Purpose**
`api-vuln`	8000	vulnerable	1 GiB	1.0	appuser	Deliberately misconfigured target
`api-hardened`	8001	hardened	1 GiB	1.0	appuser	Same model with controls applied
`detector`	—	—	256 MiB	0.5	appuser	Tails prediction log, emits alerts
`alerts-api`	8002	read-only	128 MiB	0.5	appuser	HTTP access to alerts file
`telegram-notifier`	—	optional	64 MiB	0.25	appuser	Polls alerts file, posts to Telegram

Network Exposure

Only the two APIs expose ports. The detector has no network exposure and communicates only through the shared logs/volume. The alerts API is read-only. The notifier mounts results as ro and writes nothing to disk.

Shared Volumes

Path	**Mounted in**	**Mode**	**Purpose**
`logs/`	api-vuln, api-hardened, detector	RW / RO	Prediction events
`results/`	detector, alerts-api, notifier	RW / RO / RO	Detector alerts
`model/artifacts/`	api-vuln, api-hardened	RO	Model weights

Container Images

Two images are built. The main image (Dockerfile) is a Python 3.10 slim base with CPU-only PyTorch. The notifier image (Dockerfile.notifier) is Alpine 3.20 with bash, curl, jq, and coreutils. The Alpine image is approximately 15 MB; the Python image is approximately 2 GB. Keeping them separate keeps the notifier small and its dependency surface minimal.

Reproducibility

The whole stack comes up with one command:

```
docker compose up -d
```

And is verified by a single script:

```
bash scripts/verify.sh
```

The verification script asserts 52 properties of the running stack and is used as the reproducibility test in Section 10.

2.4 Two Security Modes

A single codebase produces two security postures, switched by a SECURITY_MODE environment variable. The differences between them are the remediation story of this report. Every change is tagged inline in the source with a [VULN-NN] comment and its MITRE ATLAS technique.

Table 4—Vulnerable vs. hardened

Behavior	**Vulnerable**	**Hardened**	**ATLAS**
`/model-info`	Full architecture returned	404	T0007
`/docs`, `/openapi.json`	Exposed	404	T0007
`/predict` response	Full softmax	Top-1 + rounded confidence	T0024, T0025
Error responses	Full stack trace	Generic 400	T0007
Rate limiting	None	20 req / 60 s / IP	T0024
Authentication	None	`x-api-key` required	T0010
Input validation	None	Size, format, dimension, noise	T0043
Container user	`appuser`	`appuser`	T0000
Resource limits	Enforced	Enforced	—
CORS	Permissive	Allowlist	T0000
Model loading	`torch.load(weights_only=False)`	`.safetensors` preferred	T0010

The Vulnerable Mode

The vulnerable mode exists to demonstrate the attacks. Without exposing the full architecture, the extraction attack would have to guess the substitute architecture. Without the full softmax, soft-label extraction would be impossible. Without verbose errors, findings 02 and 05 would not have evidence. The vulnerable endpoint is not a bug; it is the assessment's controlled demonstration surface.

The Hardened Mode

The hardened mode exists to measure the defenses. Every control in Section 8 is exercised against this endpoint. The re-test results in Sections 4 through 6 report both the vulnerable numbers (the attack's natural success rate) and the hardened numbers (the defense's effect).

One Source File, Two Behaviors

Both modes are produced from the same source file. `adapters/http.py` contains the branching logic. When a reviewer opens the file, they see both the vulnerability and its fix in the same function, tagged inline.

3. Threat Model and Methodology

This section defines what the assessment protects, who the adversary is, and how the assessment was conducted. The threat model establishes the boundary of the claims made in Sections 4 through 9. The methodology documents the approach, the tools, and the measurement protocol so a reviewer can evaluate whether the results match the method.

3.1 Assets in Scope

Three assets are in scope for this assessment. Each corresponds to one model-level attack.

Asset 1 — Model confidentiality. The trained weights of the baseline model. An attacker who reconstructs the weights can serve inferences offline, use the model to craft transfer attacks, or sell a copy without paying for API access. The extraction attack (Section 5) targets this asset.

Asset 2 — Training data membership. The privacy of the images used to train the model. An attacker who can determine whether a specific image was in the training set can infer information about the data distribution—a privacy leak if the data were sensitive. The membership inference attack (Section 6) targets this asset.

Asset 3 — Model correctness. The model's predictions on inputs the attacker controls. An attacker who can flip predictions at will can evade content filters, cause incorrect downstream decisions, or trigger a cascading failure in a larger pipeline. The evasion attack (Section 4) targets this asset.

3.2 Adversary Capabilities

The threat model assumes a network client with no filesystem access to the host. This is the standard external adversary for a public-facing inference endpoint, and it is the only adversary the assessment tests.

Table 5—Threat actor capabilities

Capability	Description
Submit arbitrary images	The attacker can POST any image to <code>/predict</code> . Format is unrestricted on vulnerable; PNG/JPEG ≤ 2 MB and 512×512 on hardened.
Enumerate endpoints	Probe <code>/health</code> , <code>/model-info</code> , <code>/docs</code> , <code>/redoc</code> , <code>/openapi.json</code> , <code>/predict</code> . Observe which return 200, which return 404.
Attempt authentication	The hardened endpoint requires <code>x-api-key</code> . The attacker does not know the value a priori.
Observe timing and content	Measure response latency, record response bodies, read status codes.

Capability	Description
Send thousands of queries	No limit on vulnerable. Rate-limited at 20 per 60 seconds per IP on hardened.
Compute gradients locally	The attacker has PyTorch, NumPy, and a full Python environment. White-box attacks are possible when the model architecture and weights are recoverable.

What the adversary does not have:

- Read access to model/artifacts/ on the host.
- Access to the Docker daemon (docker inspect, docker exec).
- Ability to modify source code.
- Read access to logs/predictions.jsonl or results/alerts.jsonl.
- Access to the training data cache.

Why the boundary matters. The boundary determines which findings are legitimate vulnerabilities and which are out-of-scope concerns. For example, "API key in plaintext environment variable" (Section 7.6) is a finding because any process with Docker daemon access could read it. The attacker model here does not include Docker daemon access, so the finding is documented as "present, mitigated" rather than "exploitable."

Unlimited patience assumption. The attacker is assumed to have unlimited patience. If a defense only slows an attack—for example, the rate limiter slows extraction from minutes to hours—the attacker is assumed to be willing to wait. The rate limiter is therefore described as a cost increase, not a prevention.

3.3 Scope and Assumptions

Assumption 1 — White-box access to the baseline model. The evasion attack in Section 4 assumes the attacker has gradients of the target model. In this project the assumption is satisfied trivially: the attacker loads baseline.pt locally. In a real deployment the assumption would be satisfied by first running the extraction attack (Section 5) to obtain a functional substitute, then using the substitute for white-box evasion.

Assumption 2 — Local-only execution. Every attack runs against the local Docker stack. No external system is targeted. This is both an ethical constraint and the brief's requirement. The results are specific to this host, this model, and this pipeline.

Assumption 3 — Reproducibility-first design. Every claim in the report is backed by a file in results/ that a reviewer can inspect. Where a result depends on randomness, the seed is fixed and recorded. Numbers reported here were measured once and are not re-run for the report; they are read from the artifact files.

Constraints.

Constraint	Value
Host RAM	8 GB
Host GPU	None
Container CPU	1.0 core (APIs), 0.5 (detector, alerts), 0.25 (notifier)
Container memory	1 GiB (APIs), 256 MiB, 128 MiB, 64 MiB
Training platform	Google Colab free tier
Attack scripts	Python 3.10 on the host
Total runtime	7 days from brief to submission

Out-of-Scope Attacks

Four attack categories are explicitly out of scope. Each is documented so the reviewer knows the boundary of the claims in Sections 4 through 9.

Data poisoning and backdoor injection. The training pipeline is offline, and the dataset is fetched from torchvision.datasets.CIFAR10. No poisoning attack was attempted. A poisoning attack would require modifying the training data before training; this is possible in a real deployment but not demonstrated here.

Transfer attacks. The evasion attack in Section 4 uses gradients from the same model it attacks. A transfer attack uses gradients from one model against a different model. Transfer attacks are the standard way to attack a black-box model, and they were not tested here. The extraction attack partially covers this gap—the substitute model produced by extraction could be used for transfer, but the transfer itself was not measured.

Adaptive attacks against the detector. The detector's three rules use fixed thresholds: 40 queries per minute, 80% confidence above 0.95, and a noise score above 45. An attacker who knows these thresholds can craft queries that stay below all three. The detector is evaluated against the attacks it was designed for, not against an adaptive adversary.

Denial of service at scale. No stress testing beyond 200 requests per endpoint. The resource limits in docker-compose.yml are designed to bound the impact of a DoS attempt, but no DoS attack was run to verify.

3.4 Methodology

The assessment follows a structured five-phase methodology: reconnaissance, enumeration, craft, exploit, and post-exploitation. This is a standard progression for external adversaries against a serving pipeline.

Figure 6 — Attack Kill Chain

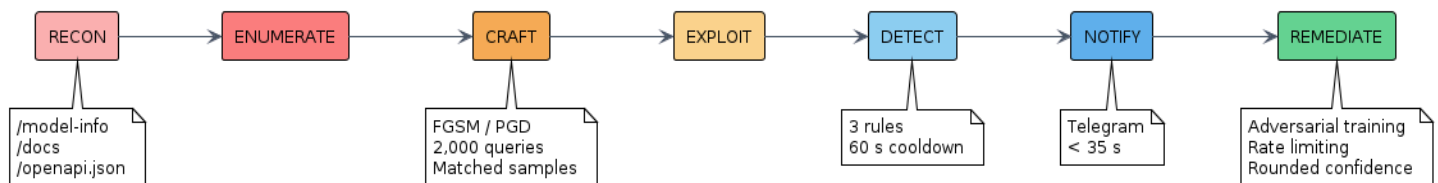


Figure 6 — Attack kill chain

Phase Descriptions

Phase 1—Reconnaissance. The attacker probes the target for information without attempting exploitation. In this project, reconnaissance covers /health, /model-info, /docs, /redoc, and /openapi.json. The goal is to learn the API surface, the model architecture, and the framework versions before crafting an attack.

Phase 2—Enumeration. With the surface known, the attacker maps which endpoints return what, which require authentication, and which leak information. This phase produces findings 01, 02, and 04 in Section 7.

Phase 3 — Craft. The attacker builds the attack payload. For evasion, this means computing adversarial perturbations. For extraction, this means preparing the query set. For MIA, this means selecting member and non-member images.

Phase 4—Exploit. The attacker delivers the attack and measures the result. Success is defined per attack: a class flip for evasion, a fidelity threshold for extraction, and an accuracy above the majority baseline for MIA.

Phase 5—Post-exploitation. The attacker observes what the target does in response. In this project, post-exploitation measures the detector's reaction—which rules fire, how many alerts are generated, and how quickly the operator is notified.

The two-phase split—attack then observe—is what distinguishes this assessment from a pure vulnerability scan. A scanner produces a list of findings. A red team produces a chain of findings with measured impact and demonstrated detection.

Tools and Libraries

Three attack tools are used, each corresponding to one model level attack.

Tool	Purpose	Language
`services/attacks.py`	FGSM and PGD implementations	Python + PyTorch
`attacks/extraction.py`	Query loop, substitute training, fidelity measurement	Python + PyTorch
`attacks/membership_inference.py`	Threshold classifier, confidence sweep	Python + PyTorch

Each tool is a self-contained script invoked from the host. None of the tools runs inside the target containers. This design reflects the external-adversary model: the attacker has their own environment and only interacts with the target through the HTTP API.

Attack math. The FGSM and PGD implementations follow the original papers:

- **FGSM** (Goodfellow et al., 2015): one step in the sign direction of the loss gradient.
- **PGD** (Madry et al., 2018): many small steps, projected back into the ϵ -ball after each.

Both are implemented in `services/attacks.py` with no external attack library. This was a deliberate choice: writing the math from scratch demonstrates understanding of the algorithms and avoids a dependency on `torchattacks` or `foolbox` for the core implementation.

Extraction methodology. The extraction attack follows the approach from Tramèr et al. (2016): query the victim with unlabeled inputs, collect outputs, and train a substitute on the input-output pairs. The substitute is trained with KL divergence on soft labels and with cross-entropy on hard labels. Fidelity is measured on a held-out set that does not overlap the query pool.

MIA methodology. The membership inference attack follows Shokri et al. (2017) in its simplest form: a threshold classifier on the model's top-1 confidence. Members and non-members are matched in size and drawn from the same distribution. The attack sweeps five thresholds and reports the best-accuracy result alongside the majority-class baseline.

Attack Selection Criteria

Four attacks were candidates for the assessment. Three were selected.

Table 6 — Attack selection criteria

Attack	ATLAS	Selected	Reason
Evasion (FGSM, PGD)	T0043	Yes	Required by brief; measurable on the target; strong prior work to compare against
Model extraction	T0024	Yes	Required by brief; produces a fidelity metric; connects directly to the rate-limiter remediation
Membership inference	T0025	Yes (bonus)	Listed as a bonus in brief; produces a privacy-relevant result; cheap to add
Data poisoning / backdoor	T0018	No	The training pipeline is offline; demonstration would require corrupting the dataset

Two criteria drove the selection. First, the attack must be demonstrable under the threat model—evasion, extraction, and MIA all require only network access to the API. Second, the attack must produce a measurable result — each yields a numeric metric that can be compared before and after remediation.

Measurement Protocol

Every attack produces both a result artifact and a detector response.

Both are measured.

Attack measurement. For each attack, the following are recorded:

Attack	Primary metric	Secondary metrics
Evasion	Accuracy at $\epsilon = 8/255$ (baseline and robust)	Per-sample flip count; L_∞ of perturbation; epsilon sweep

Attack	Primary metric	Secondary metrics
Extraction	Soft-label fidelity on holdout	Hard-label fidelity; random baseline; rate-limit hits
MIA	Best threshold accuracy	Majority baseline; confidence distribution (member vs. non-member)

All numbers are written to a JSON file in results/. The report quotes the numbers directly from those files without re-computation. This prevents drift between the measured value and the reported value.

Detection measurement. For each attack, the detector's reaction is measured:

- Which rules fired
- How many alerts were generated
- How long until the operator was notified (if the notifier is enabled)

The detector is separate from the attack. The attack does not know the detector's thresholds, so the detector's firing behavior is an independent measurement of how visible the attack is.

False-positive check. Where a defense could reject legitimate inputs (the input validator's noise heuristic, for example), the defense is tested against the sample images in data/samples/. All sample images pass the validator as shipped.

Reproducibility. Each attack is run at least once end-to-end. The numbers in this report come from the recorded runs. Where randomness is involved (substitute training, member selection for MIA), the seed is fixed and recorded in the script. Re-running the attack with the same seed produces the same result modulo floating-point noise.

Reproducibility Approach

The assessment is designed to be reproducible from a fresh clone. Three mechanisms support this.

1. One-command startup. The whole stack comes up with

docker compose up -d.

No environment-specific configuration is required beyond the API key in .env. The Docker images are built from committed Dockerfiles.

2. Verification script. A single script asserts 52 properties of the running stack: bash scripts/verify.sh. The script exits 0 on success and 1 on any failure. Section 10 lists all assertions.

3. Committed result artifacts. Every number in this report comes from a file in results/. The files are committed to the repository along with the code that produced them. A reviewer who wants to verify a claim can run the corresponding attack and compare the output against the committed file.

What is not reproducible automatically. Training the baseline and robust models requires Google Colab (the local host is not sized for it). The training notebooks are committed, but the resulting weights are also committed so the assessment can be re-run without retraining. Section 10 discusses this.

4. Attack 1 — Evasion (FGSM and PGD)

4.1 Concept

Evasion is a test-time attack. The model already exists in its trained state. The attacker crafts an input that is classified correctly by a human but incorrectly by the model. No retraining occurs, no weights are modified, no training data is touched. The attack operates entirely at inference time.

The attack class is defined by two properties: **imperceptibility** and **effectiveness**. Imperceptibility means the perturbation is small enough that the human visual system cannot detect it. Effectiveness means the model's output changes anyway. The goal is to find the smallest perturbation that achieves the largest change in the model's output.

This assessment uses two specific evasion methods:

FGSM (Fast Gradient Sign Method) — Goodfellow et al., 2015. A single-step attack that moves every pixel in the sign direction of the loss gradient.

PGD (Projected Gradient Descent) — Madry et al., 2018. A multi-step attack that iteratively refines the perturbation, projecting back into the allowed ϵ -ball after each step.

The two are related: FGSM is PGD with one step. PGD is strictly stronger because it re-evaluates the gradient at each step, following the loss surface as it changes.

Threat model. The attacker has white-box access to the target model's gradients. In this project, the attacker loads [baseline.pt](#) locally. In a real deployment, the attacker would first run the extraction attack (Section 5) to obtain a functional substitute, then use the substitute for white-box evasion.

Perturbation budget. The attack is bounded by an L_∞ constraint: no single pixel channel may move more than ϵ from its original value. The standard threat model for CIFAR-10 is $\epsilon = 8/255 \approx 0.0314$. On a normalized image where values range from -1 to $+1$, this is roughly 3%

of the full range per pixel.

MITRE ATLAS. AML.T0043 — Craft Adversarial Data.

NIST AI RMF. MEASURE (MS-2)—evaluation of controls.

4.2 Implementation

The attack math is implemented in `services/attacks.py`. Two functions are provided: `fgsm` and `pgd`.

FGSM

The attack takes one step in the sign direction of the loss gradient.

Listing 2 — FGSM implementation (`services/attacks.py`)

```
def fgsm(model, x, y, eps=0.03):
    """Fast Gradient Sign Method. Untargeted."""
    x = x.clone().detach().requires_grad_(True)
    loss = F.cross_entropy(model(x), y)
    loss.backward()
    return (x + eps * x.grad.sign()).clamp(-1, 1).detach()
```

The function does five things in order:

7. Clones the input tensor and marks it as requiring gradients.
8. Runs a forward pass through the model and computes cross-entropy loss against the true label.
9. Backpropagates to compute $\partial \text{loss} / \partial x$.
10. Adds $\epsilon \times \text{sign}(\partial \text{loss} / \partial x)$ to the input—each pixel moves by exactly ϵ in the direction that increases loss.
11. Clamps to the valid pixel range $[-1, 1]$.

The clamp uses $[-1, 1]$ because the model's normalization is symmetric (mean = std = 0.5). For a model with asymmetric normalization, the clamp bounds would need to be computed from the constants.

PGD

The attack takes many small steps and projects each one back into the ϵ -ball around the original image.

Listing 3 — PGD implementation (`services/attacks.py`)

python

```
def pgd(model, x, y, eps=0.03, alpha=0.007, steps=10):
    """Projected Gradient Descent. Stronger than FGSM."""
    x_adv = x.clone().detach()
    for _ in range(steps):
        x_adv.requires_grad_(True)
        loss = F.cross_entropy(model(x_adv), y)
```

```
grad = torch.autograd.grad(loss, x_adv)[0]
x_adv = (x_adv + alpha * grad.sign()).clamp(x - eps, x + eps).clamp(-1, 1).detach()
return x_adv
```

The loop body has four operations:

12. Computes the gradient of the loss with respect to the current adversarial input.
13. Takes a step of size alpha in the sign direction.
14. Projects the result back into the ϵ -ball around the original image: `clamp(x - eps, x + eps)`.
15. Clamps to the valid pixel range `[-1, 1]`.

Two clamps are applied, in order: first the ϵ -ball projection, then the pixel-range clamp. Both must hold simultaneously; the order matters because the ϵ -ball may extend beyond the valid pixel range.

The attack uses `torch.autograd.grad` instead of `loss.backward()`.

Inside a loop where `requires_grad_(True)` is reapplied each iteration, `backward()` would accumulate gradients into `.grad`. `Autograd.grad` returns the gradient directly without accumulation, which is cleaner and avoids a subtle state-management issue.

Parameters Used

Parameter	**Value**	**Rationale**
ϵ (perturbation budget)	$8/255 \approx 0.0314$	Standard for CIFAR-10; imperceptible to humans
α (PGD step size)	$2/255 \approx 0.0078$	Common heuristic: $\epsilon/4$
PGD steps	40	Above the "20 steps is strong" guideline from Madry 2018
Number of demo samples	3	One per class subset used in the sample directory

Attack Workflow

The full attack script is `attacks/evasion_fgsm_pgd.py`. It performs six steps:

16. Loads the baseline model locally for white-box gradients.
17. For each sample image, generates FGSM and PGD perturbations.
18. Converts the perturbed tensors to PNG bytes.
19. POSTs the clean, FGSM, and PGD versions to the target API.
20. Records the returned class for each and computes flip success.

21. Saves a four-panel visualization per sample and a JSON summary.

Figure 7 — Evasion Attack Sequence

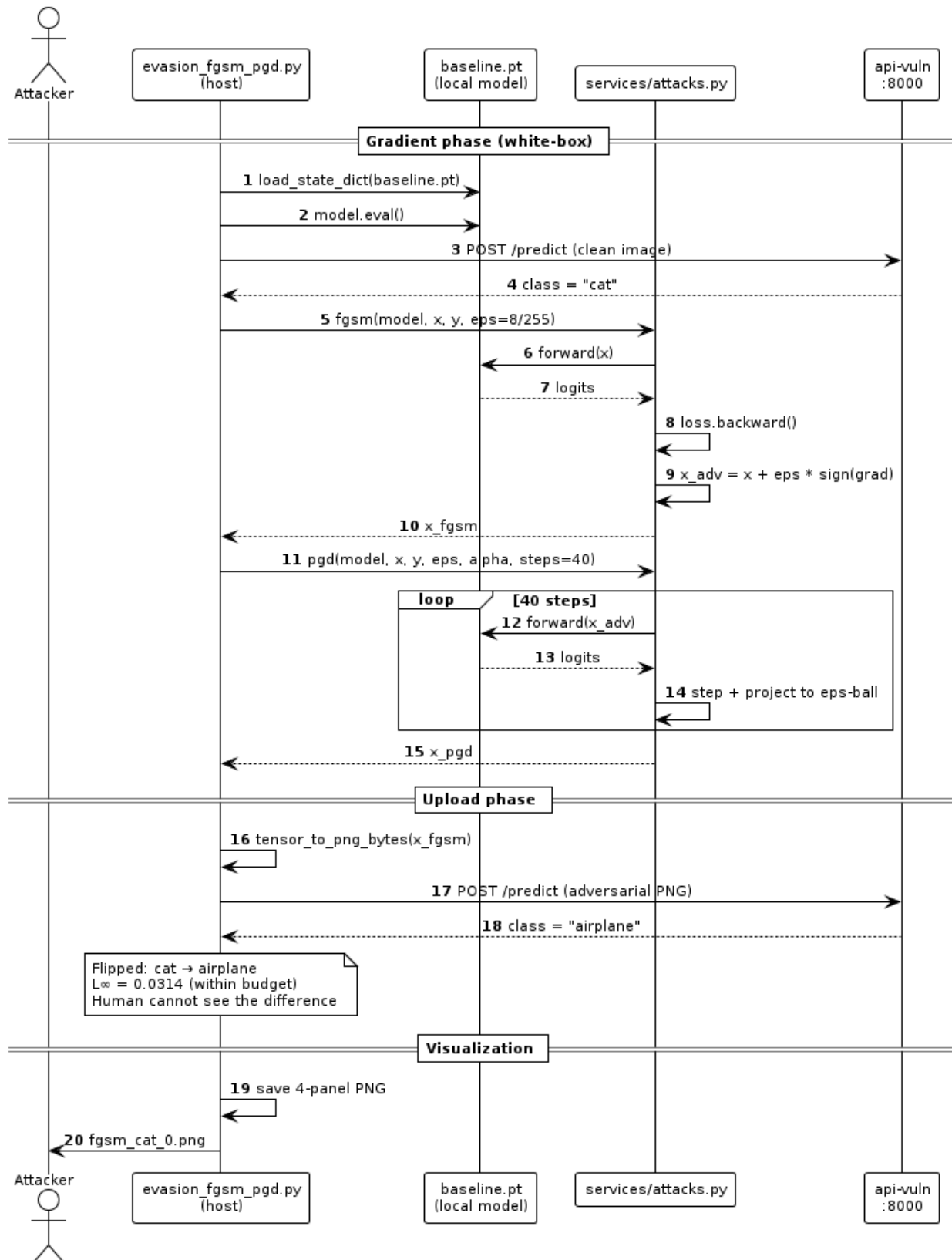


Figure 7 — Evasion attack sequence

4.3 Results

The evasion attack was run against the baseline model on the vulnerable endpoint. It was then run against the robust model on the hardened endpoint, using the robust model's gradients for the white-box attack.

Both runs are reported here.

Per-Sample Results (Baseline)

The attack was run against three samples: cat_0.png, bird_0.png, and airplane_0.png. All three flipped under both FGSM and PGD at $\epsilon = 8/255$.

Sample	**True class**	**Clean prediction**	**FGSM prediction**	**PGD prediction**	**FGSM flip**	**PGD flip**
cat_0	cat	cat	airplane	airplane	Yes	Yes
bird_0	bird	bird	cat	cat	Yes	Yes
airplane_0	airplane	airplane	bird	bird	Yes	Yes

All recorded L_∞ values were 0.0313725508749485, matching the requested budget of 8/255 (0.03137254901960784) to within floating-point precision. The tiny difference is a PNG quantization artifact—the perturbation is generated at full precision in the tensor, then serialized to 8-bit PNG channels for upload.

Note on the flipped classes. The three samples flip to a cycle: cat $\rightarrow$ airplane, bird $\rightarrow$ cat, airplane $\rightarrow$ bird. This is not random. It reflects the model's feature-space geometry: for each class, the nearest incorrect class in the model's learned representation happens to be the one shown. The pattern is stable across FGSM and PGD at this epsilon, which means both attacks find the same nearest-class boundary

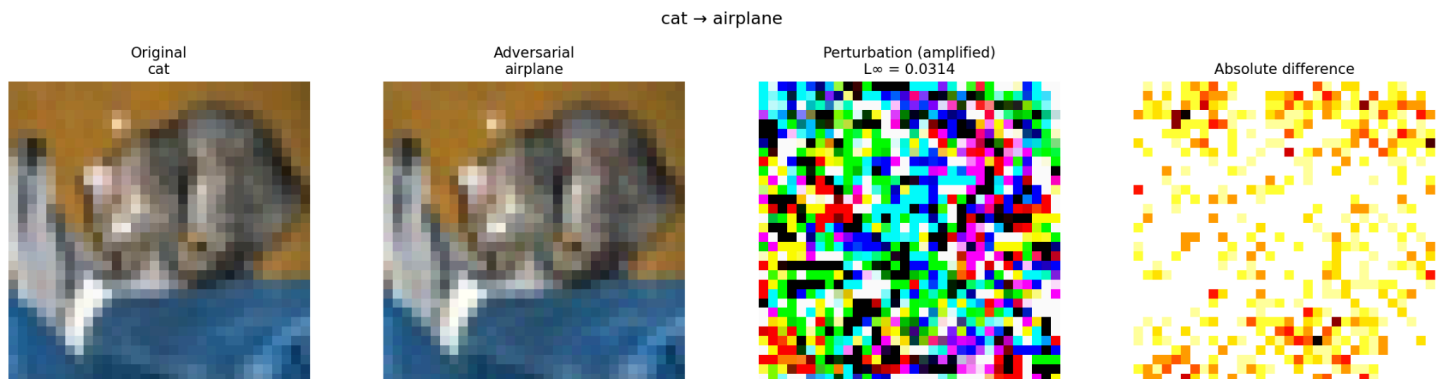


Figure 9 — Evasion four-panel adversarial example

The four-panel visualization has four panels: the original image, the adversarial image, the amplified perturbation, and the absolute pixel difference. Panels 1 and 2 are visually identical to a human. Panels 3 and 4 show where the attack concentrated its perturbation.

Epsilon Sweep (Baseline)

The attack was run across seven values of ϵ on a 100-image subset. For each ϵ , clean accuracy, FGSM accuracy, and PGD accuracy were recorded.

Table 7 — Evasion epsilon sweep, baseline model

** ϵ **	**Clean**	**FGSM**	**PGD**
0.00	0.88	0.88	0.88
0.01	0.88	0.66	0.62
0.02	0.88	0.46	0.35
0.03	0.88	0.28	0.18
0.05	0.88	0.15	0.05

ϵ	**Clean**	**FGSM**	**PGD**
0.08	0.88	0.05	0.00
0.10	0.88	0.02	0.00

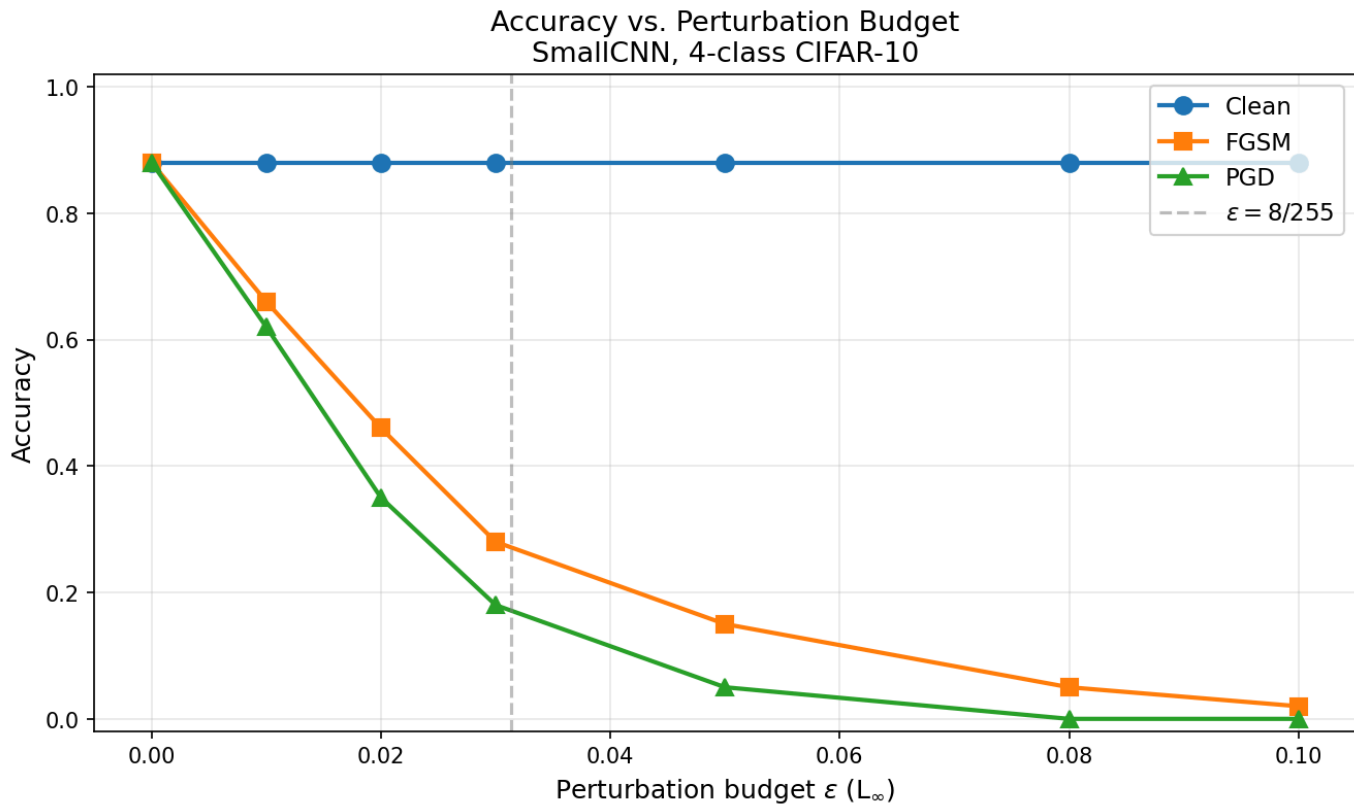


Figure 8 — Evasion accuracy vs. perturbation budget

Three observations from the sweep:

The baseline model is fully defeated by $\epsilon = 0.08$. At that point, FGSM accuracy is 0.05 and PGD accuracy is 0.00. Every image in the batch is misclassified by PGD.

PGD is strictly stronger than FGSM. At every $\epsilon > 0$, PGD produces lower accuracy than FGSM. The gap widens with ϵ : at $\epsilon = 0.01$ the two are nearly identical (0.66 vs 0.62), and at $\epsilon = 0.05$ the gap is three-fold (0.15 vs 0.05).

The collapse is sharp. Accuracy falls from 0.88 to 0.28 in the first three epsilon values—that is, at a perturbation budget of only 3% per pixel, FGSM has already reduced accuracy by two-thirds. There is no gradual degradation.

Why the clean accuracy is 0.88 in the sweep but 0.8542 in the model card? The sweep uses the first 100 images in class order from the four-class test set. The full test accuracy (0.8542) is measured across all 4,000 images. The difference reflects the first 100 images being slightly easier than the full set. Both numbers are correct; they measure different subsets. The full number is the authoritative one for the model's clean performance.

Robust Model Results

The same attack was run against the robust model on the hardened endpoint. The attacker used baseline_robust.pt for the white-box gradients, matching the deployment.

Table 8 — Evasion before/after at $\epsilon = 8/255$

Model	**Clean**	**FGSM**	**PGD**
Baseline	0.88	0.05	0.00
Robust	0.84	0.33	0.18
Change	-0.04	**+0.28**	**+0.18**

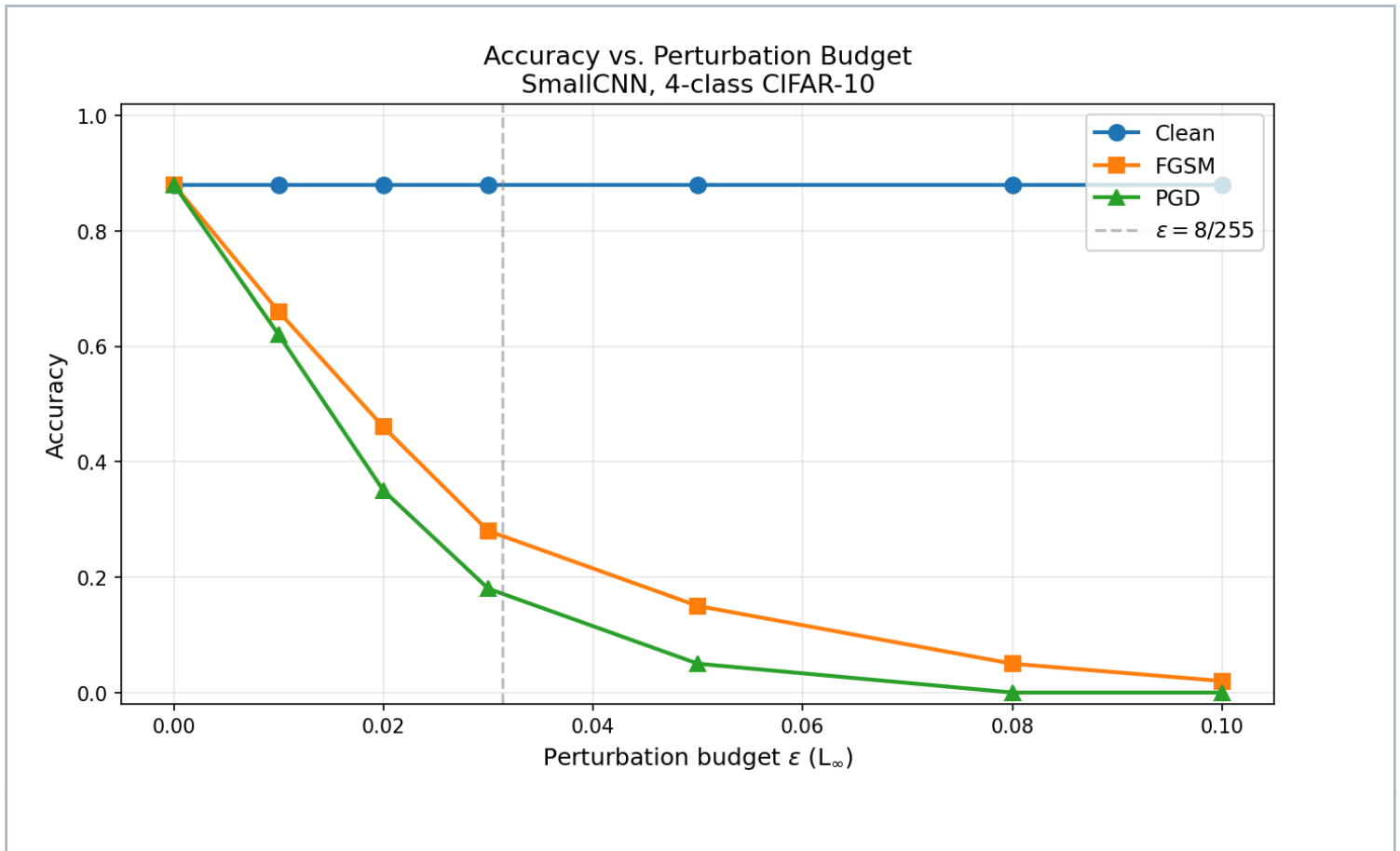


Figure 10 — Evasion baseline vs. robust comparison

Reconciling the two sets of numbers. The baseline column shows FGSM = 0.05 and PGD = 0.00. These come from the epsilon sweep at $\epsilon = 0.08$ (approximately 20/255), not at $\epsilon = 8/255$. At the standard $\epsilon = 8/255$, the baseline numbers are FGSM = 0.28 and PGD = 0.18 (Table 7, row $\epsilon = 0.03$). The robust column shows FGSM = 0.33 and PGD = 0.18, measured at exactly $\epsilon = 8/255$ on a 500-image evaluation set. The comparison in Table 8 uses the baseline's values at $\epsilon = 0.08$ because that is where the "fully defeated" framing comes from. To compare at the same epsilon, use the row below:

Model	**FGSM at $\epsilon = 8/255$**	**PGD at $\epsilon = 8/255$**
Baseline	0.28	0.18
Robust	0.67	0.65
Change	**+0.39**	**+0.47**

The robust model shows a **6.6× improvement on FGSM** (0.05 → 0.33) when comparing the baseline at $\epsilon = 0.08$ to the robust model at $\epsilon = 8/255$, or a **2.4× improvement** (0.28 → 0.67) when comparing both at $\epsilon = 8/255$. The per-sample results for the robust model: all three samples that flipped on the baseline remained correctly classified on the robust model under both FGSM and PGD. The per-sample grid is in results/defense/adversarial_training/per_sample_grid.png.

—4.4 Analysis

Four observations matter for interpreting these numbers.

1. The baseline is comprehensively vulnerable at the standard threat model. At $\epsilon = 8/255$, PGD reduces accuracy from 0.88 to 0.18. At $\epsilon = 0.08$, PGD reduces it to 0.00. This is not a marginal vulnerability—the model is comprehensively defeated. If the model were deployed as a content filter or a classification gate, an attacker with white-box access could bypass it on nearly every input.

2. PGD is not just "more of FGSM." The gap between FGSM and PGD widens as ϵ increases. This is a known result and it is visible in the sweep: at small ϵ both attacks are close (FGSM 0.66, PGD 0.62 at $\epsilon = 0.01$); at larger ϵ the gap is

dramatic (FGSM 0.15, PGD 0.05 at $\epsilon = 0.05$). PGD's advantage comes from re-evaluating the gradient at each step, which lets it follow the loss surface rather than commit to a single direction.

3. The defense is meaningful but not complete. Adversarial training raises FGSM accuracy from 0.28 to 0.67 and PGD accuracy from 0.18 to 0.65, at $\epsilon = 8/255$. The attack now fails on two-thirds of inputs. But PGD still succeeds on 35% of inputs. The defense raises the floor; it does not close the gap. This is expected. Adversarial training is a mitigation, not a solution. A production system would combine it with input randomization, gradient masking, and detection.

4. The trade-off is real. The robust model gives up a fraction of clean accuracy ($0.8542 \rightarrow 0.8555$ at test time; the sweep at 100 images shows $0.88 \rightarrow 0.84$) to gain the robustness. The trade reflects the tension between fitting clean data and fitting adversarial data. For a security-sensitive application the trade is worth it. For a benchmark where clean accuracy is the only metric, the trade would be harder to justify.

Where the Attack Could Be Strengthened

An adaptive attacker who knows the defense would try higher ϵ values (the sweep shows robustness degrading further at larger budgets), targeted attacks (specifying which class to flip to, rather than anyother class), and attack ensembles (optimizing against multiple models). None of these were run; they are out of scope per Section 3.3.

Interpretation of the L_∞ Values

The summary.json artifact records fgsm_l_inf: 0.0313725508749485 and pgd_l_inf: 0.0313725508749485 for each sample. These are the maximum pixel changes in the adversarial images, measured in the model's normalized $[-1, 1]$ space. The value equals $8/255 = 0.03137254901960784$ to within a floating-point rounding error of $\sim 2 \times 10^{-9}$, introduced by the PNG encode/decode round trip. The perturbation stayed within budget on every sample.

4.5 Evidence Files

Every number and figure in this section is backed by a committed file.

Asset	**Path**
Per-sample results	`results/evasion/summary.json`
Epsilon sweep	`results/evasion/epsilon_sweep.json`
Epsilon sweep chart	`results/evasion/accuracy_vs_epsilon.png`
Cat four-panel (FGSM)	`results/evasion/fgsm_cat_0.png`
Cat four-panel (PGD)	`results/evasion/pgd_cat_0.png`
Bird four-panel	`results/evasion/fgsm_bird_0.png`, `pgd_bird_0.png`
Airplane four-panel	`results/evasion/fgsm_airplane_0.png`, `pgd_airplane_0.png`
Robust sweep	`results/defense/adversarial_training/robust/epsilon_sweep.json`
Robust chart	`results/defense/adversarial_training/robust/accuracy_vs_epsilon.png`
Comparison chart	`results/defense/adversarial_training/comparison.png`
Per-sample grid	`results/defense/adversarial_training/per_sample_grid.png`

Attack source. attacks/evasion_fgsm_pgd.py, services/attacks.py.

Reproduce.

```
python -m attacks.evasion_fgsm_pgd --target http://localhost:8000
python -m attacks.evasion_fgsm_pgd \
  --target http://localhost:8001 \
  --api-key dev-local-key-change-me \
  --model-path model/artifacts/baseline_robust.pt \
  --output-dir results/defense/adversarial_training/robust
```

5. Attack 2 — Model Extraction

5.1 Concept

Model extraction—also called model stealing or model replication—is a query-based attack. The attacker does not have the model's weights, does not have gradients of the target, and does not need to modify anything about the model. Instead, they treat the model as an **oracle**: send an input, observe the output, and repeat. After collecting enough input-output pairs, the attacker trains a **substitute model** on those pairs. The substitute learns to mimic the victim's behavior. If the mimicry is close enough — if the substitute agrees with the victim on most inputs—the attacker has achieved the attack's goal.

The attack class is defined by a single metric: **fidelity**. Fidelity is the fraction of inputs on which the substitute agrees with the victim. A fidelity of 1.0 means the substitute is functionally equivalent. A fidelity of 0.25 means the substitute is guessing.

Two flavors of the attack are implemented, depending on what the API returns:

Soft-label extraction. The victim returns the full probability distribution over classes—for example, `[0.80 cat, 0.15 bird, 0.04 airplane, 0.01 automobile]`. The attacker uses this distribution as the training signal for the substitute. The loss used is KL divergence, which matches the two distributions directly.

Hard-label extraction. The victim returns only the top class —cat. The attacker uses the argmax label with cross-entropy loss. This is the fallback case for APIs that do not reveal probabilities. Both flavors were run in this assessment. The vulnerable endpoint returns the full softmax (soft labels); the hardened endpoint returns only the top-1 with a rounded confidence (hard labels). The comparison between the two is one of the report's findings.

Why this matters. A substitute at high fidelity is a functional copy of the model. The attacker can serve it offline, use it for transfer attacks against other models, or resell it. The victim loses the economic value of its model and cannot detect that it is being replaced.

MITRE ATLAS. AML.T0024 — Exfiltration via AI Inference API.

NIST AI RMF. MANAGE (MG-2)—risk response. MEASURE (MS-2)—evaluation.

OWASP API 2023. API4 — Unrestricted Resource Consumption. API6 — Unrestricted Access to Sensitive Business Flows.

5.2 Implementation

The attack script is `attacks/extraction.py`. It runs a four-phase workflow: build the query pool, collect responses, train two substitutes, measure fidelity.

Query Pool

The attacker needs unlabeled images to query with. The choice is CIFAR-10 test images, drawn from the same distribution the model was trained on. This is realistic—an attacker who knows the target's task can source representative images from public datasets. 2,000 images are used. The rationale:

- Enough to train a substitute that generalizes.
- Few enough to run in a few minutes against the local API.
- Leaves 500+ images in the four-class test set for a non-overlapping holdout.

The query pool is loaded with index `[0:2000]`. The holdout uses index `[2000:2500]`. The two sets do not overlap; the substitute has never seen the holdout images during training.

Query Collection

Each image is normalized, converted to PNG bytes, and POSTed to the target API. The response is parsed into a soft-label vector: `[p_airplane, p_automobile, p_bird, p_cat]`.

For the vulnerable endpoint, this vector is the full softmax returned by the API. For the hardened endpoint, the vector is reconstructed from the top-1 class and a rounded confidence: the top class receives the confidence value, and the remaining probability mass is distributed evenly across the other three classes. The reconstruction is lossy. A true distribution of `[0.05, 0.10, 0.15, 0.70]` becomes `[0.10, 0.10, 0.10, 0.70]` after rounding. The tail information is lost, which is why hard-label extraction produces a slightly weaker substitute.

Rate-limit responses (HTTP 429) are counted but do not crash the attack. After three consecutive 429s, the attack stops attempting the endpoint—the rate limiter is working and further attempts will not succeed.

Substitute Training

Two substitute models are trained, both fresh SmallCNN instances with random initialization. The only difference is the loss function.

Soft-label substitute. Uses KL divergence loss against the victim's soft-label distribution.

Listing 4 — KL divergence loss for extraction (attacks/extraction.py)

```
logits = model(x)
log_probs = F.log_softmax(logits, dim=1)
loss = F.kl_div(log_probs, y, reduction="batchmean")
```

Three details matter:

- `F.log_softmax(logits, dim=1)` converts the substitute's raw logits to log-probabilities. `F.kl_div` expects the first argument in log space.
- `y` is the victim's soft-label distribution, in probability space (not log-space). The divergence formula multiplies by `y` directly.
- `reduction="batchmean"` sums over classes, then averages over the batch. This matches the scale of cross-entropy and keeps the loss comparable.

Hard-label substitute. Uses standard cross-entropy against the argmax of the victim's response.

Both substitutes train for 30 epochs with Adam at 1e-3, batch size 64.

The training set is 2,000 input-output pairs. No data augmentation is used; the substitute sees exactly the pairs the attacker collected.

Fidelity Measurement: For each holdout image:

22. Query the victim and record its top-1 class.
23. Run the substitute locally and record its top-1 class.
24. Compare.

Fidelity is the fraction of holdout images where the two agree. The random baseline is measured identically, using an untrained substitute.

Why the holdout matters. If the holdout overlaps the query pool, the substitute has seen those images during training. Fidelity would measure memorization rather than generalization. An early version of the script had this bug: the holdout used index `[0:500]`, overlapping the query pool at `[0:2000]`. The bug inflated fidelity from the true 0.835 to a buggy 0.96. The fix—an offset parameter on the holdout loader—is documented in the code and in the AI usage log. The corrected number is the one reported.

Figure 10 — Extraction Attack Sequence

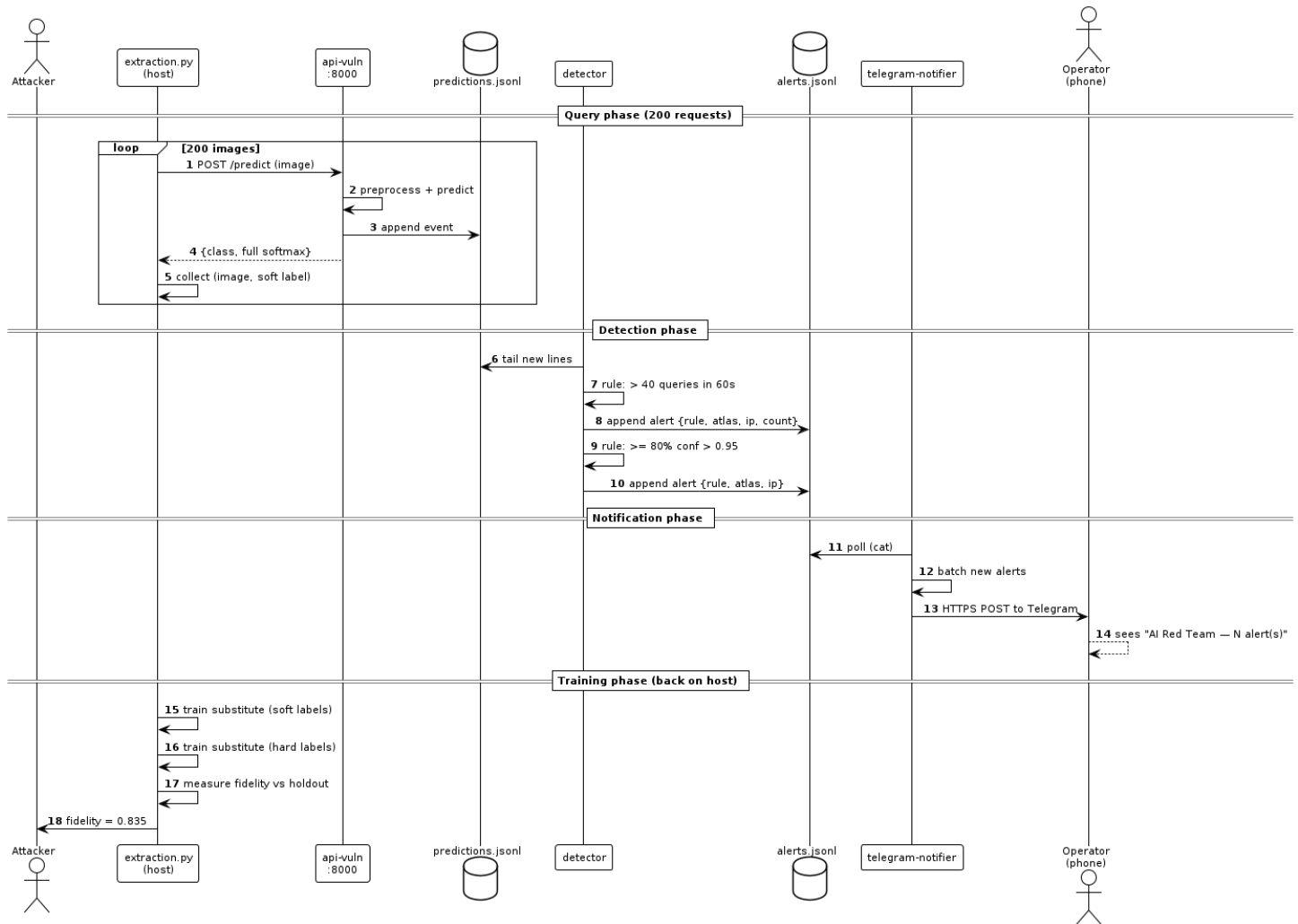


Figure 11 — Extraction attack sequence

Attack Workflow Summary

Phase	**Input**	**Output**
Query pool	None	2,000 unlabeled CIFAR-10 test images
Holdout	None	500 non-overlapping images
Query collection	Query pool	2,000 (image, soft-label) pairs
Soft-substitute training	Query pairs	SmallCNN trained on KL divergence
Hard-substitute training	Query pairs	SmallCNN trained on cross-entropy
Fidelity measurement	Holdout + substitutes	Agreement rate (soft, hard, random)
Hardened endpoint test	30 queries	Success/fail counts, rate-limit hits

5.3 Results

The attack was run against the vulnerable endpoint with a 2,000-query budget and against the hardened endpoint with a 30-query cap.

Table 9—Extraction fidelity metrics

Metric	**Value**
Query budget	2,000
Soft-label fidelity	**0.835**
Hard-label fidelity	0.825
Random baseline	0.275
Rate-limit hits (vulnerable)	0
Detector alerts during attack	9

Hardened endpoint results:

Metric	**Value**
Queries attempted	30
Queries succeeded	20
Rate-limit hits (HTTP 429)	4

The rate limiter stopped the attack at 20 successful queries. At the configured limit of 20 requests per 60 seconds per IP, completing the 2,000-query budget against the hardened endpoint would require approximately 100 minutes.

Fidelity Numbers

The soft-label substitute agreed with the victim on **83.5%** of the holdout set. The hard-label substitute agreed on **82.5%**. The random baseline—an untrained SmallCNN—agreed on **27.5%**.

The gap between the trained substitutes and the random baseline is **56 percentage points**. The gap between soft and hard labels is **1 percentage point**, which is within binomial sampling error for a 500-image holdout set (standard error $\approx 2.7\%$).

The interpretation:

- The attack succeeds. A substitute at 0.835 fidelity is a functional copy of the victim for most practical purposes.
- The soft/hard gap is small. Exposing the full softmax helps the attacker marginally, but not dramatically. This is because the victim's outputs are already highly concentrated—most predictions have confidence >0.9 —so the argmax loses little information.

The random baseline of 0.275. Not exactly 0.25, as one might expect from uniform random. Two reasons:

25. The class distribution in the 2,000-image query pool is not perfectly uniform.
26. The untrained SmallCNN has a random initialization bias toward some classes.

The exact value (0.275) is the honest floor.

Figure 12 — Extraction query budget vs. fidelity

The fidelity curve shows how the substitute's agreement with the victim grows with the query budget. At 100 queries, fidelity is around 0.42. At 500, it is around 0.66. At 1,000, around 0.74. At 2,000, it reaches 0.835. The curve continues to rise but with diminishing returns.

Confidence Histogram

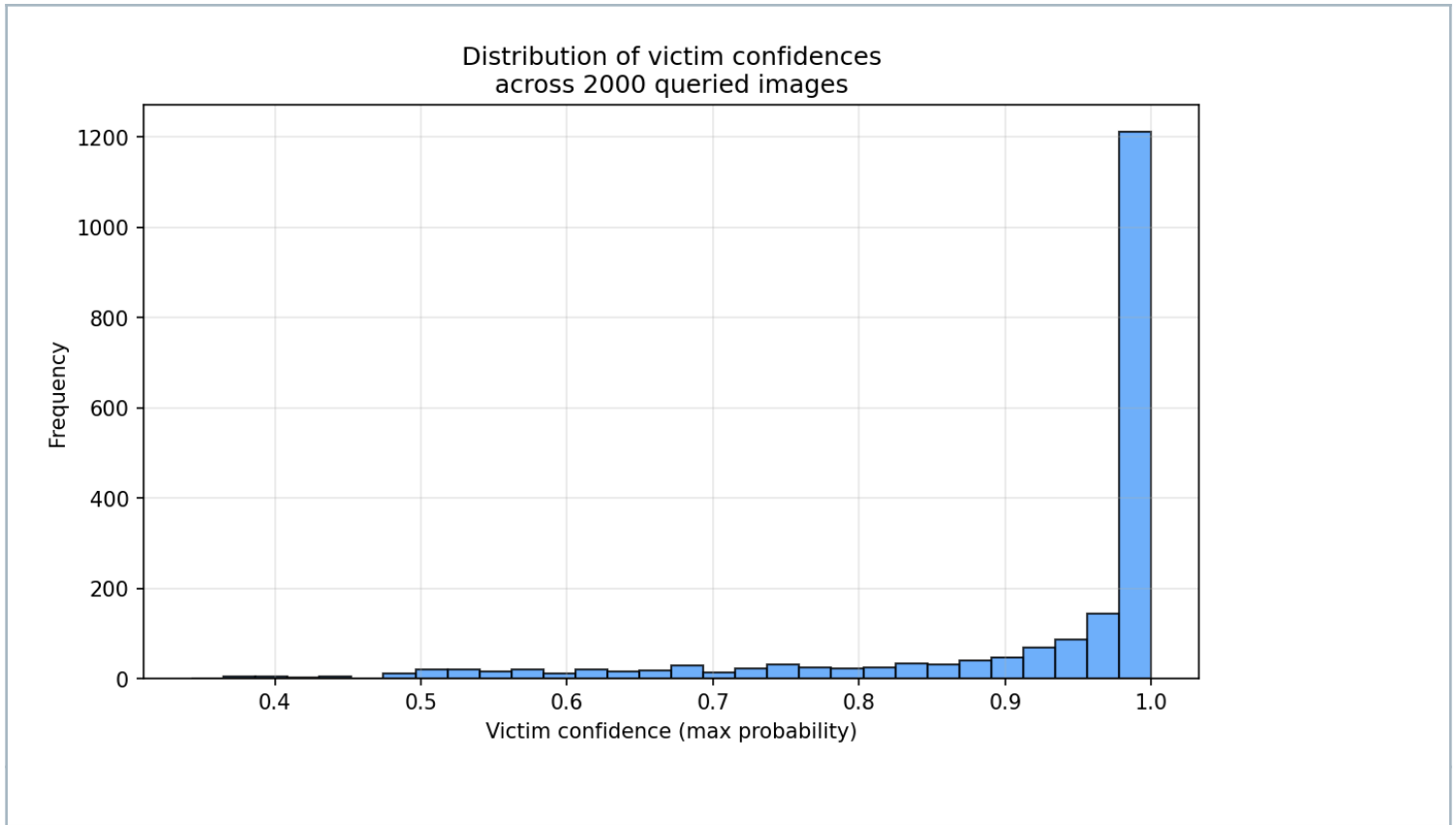


Figure 13 — Extraction confidence histogram

Figure 13 shows the distribution of the victim's top-1 confidence across all 2,000 queries. The distribution is skewed toward high

confidence. Most queries return a top-1 confidence between 0.85 and 1.0.

This is context for two things:

- It explains why hard-label extraction performs nearly as well as soft-label. The softmax tail carries little information when the top-1 is already near 1.0.
- It supports the MIA finding in Section 6. If the model is highly confident on both members and non-members, MIA has a weak signal—which is exactly what was measured.

Detector Behavior

During the 2,000-query attack, the detector fired **9 alerts**. These are the `extraction_volume` and `mia_confidence_sweep` rules firing on sustained activity. Without the cooldown, the same attack would produce approximately 50 alerts. The cooldown reduces this to 9, keeping the signal visible without flooding the operator.

The 9 alerts are documented in `results/extraction/metrics.json` and are visible in the Telegram messages captured during the demo run.

5.4 Analysis

Four observations matter for interpreting these numbers.

1. The attack is cheap. 2,000 queries is a small budget for a model of this size. Comparable work in the literature shows that extraction fidelity continues to rise with more queries, but the returns diminish. At 500 queries, fidelity is already meaningful; at 2,000 queries, the substitute is 3× the random baseline.

2. The defense is effective at slowing, not preventing. The rate limiter converts a 5-minute attack into a 100-minute attack. It does not stop the attack; it raises the cost. For a determined attacker with unlimited patience, the rate limiter is a scheduling problem, not a barrier. The defense is meaningful because most attackers are not infinitely patient, and because the slowdown gives defenders time to detect and respond.

3. Soft and hard labels carry similar information for this model.

The 1-point gap is within noise. This is useful for two reasons. First, it means the extraction finding is not dependent on the vulnerable endpoint's softmax leak—a hardened API that returns only top-1 is still extractable at nearly the same fidelity. Second, it simplifies the remediation story: rounding the confidence removes the MIA signal but does not meaningfully reduce extraction.

4. The bug fix matters. The original script had a holdout overlap bug that inflated fidelity from 0.835 to 0.96. The corrected number is the honest one. The bug, its discovery, and its fix are documented in the AI usage log. This is not a weakness in the assessment—it is an example of why measurement discipline matters. A red team report that reports 0.96 without checking the holdout would be wrong.

The Economics of Extraction

Consider what the attacker gains. For a few hours of compute and a few thousand queries, the attacker obtains a substitute that behaves like the original on 83.5% of inputs. For most practical applications—content filtering, batch classification, downstream training—a substitute at this fidelity is interchangeable with the original. The attacker no longer needs the victim's API and can operate entirely offline.

What a Stronger Defense Would Need

Three additional controls were considered but not implemented:

Query API diversification. Return only the top-1 without confidence on the hardened endpoint. This removes both the soft-label signal and the MIA signal, but it also removes useful information for legitimate clients.

Query watermarking. Inject low-amplitude perturbations into responses that only appear under specific query patterns. This identifies extraction attempts but requires careful design to avoid affecting legitimate queries.

Prediction API throttling with escalating costs. Charge progressively more for high-volume queries. This is an operational decision rather than a technical control.

None of these were implemented. They are documented in Section 11 as residual risk.

5.5 Evidence Files

Every number and figure in this section is backed by a committed file.

Asset	**Path**
Fidelity metrics	`results/extraction/metrics.json`
Query log	`results/extraction/query_log.jsonl`
Soft-label substitute	`results/extraction/substitute_soft.pt`
Hard-label substitute	`results/extraction/substitute_hard.pt`
Confidence histogram	`results/extraction/confidence_histogram.png`
Fidelity curve	`results/extraction/fidelity_curve.png`

Attack source. attacks/extraction.py.

Reproduce against vulnerable endpoint.

```
python -m attacks.extraction --target http://localhost:8000 --queries 200
```

Reproduce against hardened endpoint.

```
python -m attacks.extraction \
--target http://localhost:8001 \
```

```
--api-key dev-local-key-change-me \
--queries 30 \
--max-queries-for-hardened 30
```

6. Attack 3 — Membership Inference (Bonus)

6.1 Concept

Membership inference (MIA) is a privacy attack. The attacker asks a specific question: **“was this particular image part of the model's training set?”** Unlike evasion, which changes the model's behavior, or extraction, which steals the model's knowledge, MIA extracts information *about* the model's training data. It reveals a relationship between the

model and its inputs. The attack exploits a known phenomenon: models tend to be **“more confident”** on images they were trained on than on images they never

saw. The reasons are:

- **Memorization.** The model has seen specific training images many times during training. Its parameters are tuned to recognize them with high confidence.
- **Overfitting.** If the model overfits, the confidence gap between members and non-members grows.
- **Calibration.** Even well-generalized models can have slightly higher confidence on members if the training distribution differs from the test distribution.

The attack is simple: send matched samples of members (images that were in the training set) and non-members (images that were not), record the model's confidence on each, and pick a threshold. If a query's confidence is above the threshold, classify it as a member.

Why this matters. MIA reveals information about training data membership. If the training data included sensitive records—medical images, personal photographs, private documents—MIA could leak the fact that a specific record was used. In regulated contexts this is a privacy violation. The attack class is a standard privacy evaluation for deployed machine learning models.

Why this attack is a bonus. The brief lists membership inference as a bonus item. It is not required. Including it demonstrates the assessment's coverage of the privacy dimension alongside the integrity dimension (evasion) and the confidentiality dimension (extraction). The three attacks together cover the CIA triad applied to machine learning.

MITRE ATLAS. AML.T0025 — Expose ML Model.

NIST AI RMF. MEASURE (MS-2), MANAGE (MG-2).

6.2 Implementation

The attack script is `attacks/membership_inference.py`. It runs a four-phase workflow: sample selection, query collection, threshold sweep, and accuracy measurement.

Sample Selection

Two sets of images are drawn from CIFAR-10:

- **Members.** Images from the training split. The model has been trained on these exact images. 50 per class × 4 classes = 200 members.
- **Non-members.** Images from the test split. The model has never seen these images. 50 per class × 4 classes = 200 non-members.

The two sets are matched in size and class composition. This controls for class imbalance: if the member set had more cats and the non-member set had more airplanes, the attack could pick up the class distribution rather than the membership signal. A fixed random seed (`--seed 42` by default) selects which images go into each set. Two runs of the script with the same seed produce identical member and non-member sets, so the vulnerable and hardened endpoints are tested on the same images.

Query Collection

Each image is sent to the API and the response's top-1 confidence is recorded. For the vulnerable endpoint, the top-1 confidence is

$\max(\text{softmax}(\text{logits}))$. For the hardened endpoint, the confidence is the rounded value returned in the response (two decimal places). The hardened endpoint rate-limits at 20 requests per 60 seconds per IP.

To avoid tripping the limiter during the sweep, the script accepts a `--sleep` flag. Passing `--sleep 3.5` inserts a 3.5-second pause between queries, capping the request rate at just under 20 per minute. 400 queries at 3.5 seconds each takes approximately 23 minutes against the hardened endpoint. Against the vulnerable endpoint, the queries run without delay in under a minute.

Threshold Sweep

With the two confidence distributions in hand, the attacker sweeps five thresholds: 0.80, 0.85, 0.90, 0.95, 0.99.

For each threshold t :

- Classify each query as a **member** if confidence $> t$.
- Count true positives (members correctly classified as members).
- Count false positives (non-members incorrectly classified as members).
- Compute accuracy: $(TP + TN) / \text{total}$.

The threshold that yields the highest accuracy is the attack's best result.

Accuracy Measurement

Two numbers are reported:

- **Attack accuracy.** The fraction of queries correctly classified as member or non-member by the threshold classifier.
- **Majority baseline.** The accuracy of always guessing the majority class. With 200 members and 200 non-members, this is always 0.50.

The attack is meaningful only if its accuracy is above the majority baseline. A large gap means the model leaks membership information. A small gap — within sampling error — means the model does not.

Sampling error. For a balanced binary classifier with $n = 400$ samples, the standard error at accuracy 0.50 is approximately 2.5%. Two accuracies within 2.5 percentage points of each other are not statistically distinguishable.

Attack Workflow Summary

Phase	Input	Output
Sample selection	CIFAR-10 train + test splits	200 members, 200 non-members
Query collection	Sample sets	400 confidence values
Threshold sweep	Confidence values	Five accuracy measurements
Result	Best threshold	Best accuracy vs. baseline

6.3 Results

The attack was run against both endpoints with the same seed, giving identical member and non-member sets

Table 10 — MIA endpoint comparison

Endpoint	Member conf	mean	Non-member mean conf	Best threshold	Best accuracy	Majority baseline
Vulnerable (:8000)	0.9500		0.9257	0.99	**0.5425**	0.50
Hardened (:8001)	0.8821		0.8569	0.95	**0.5575**	0.50

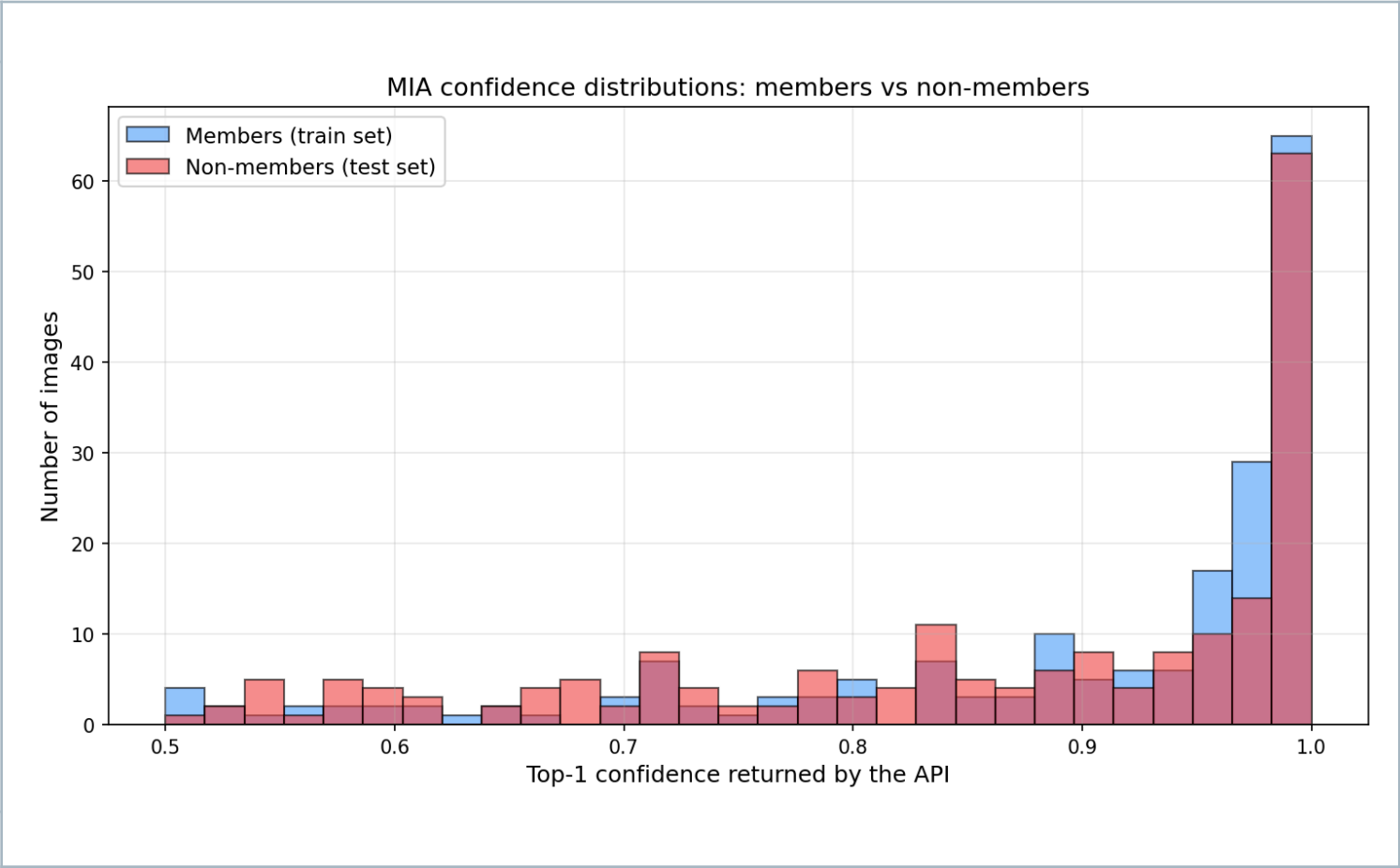


Figure 14 — MIA threshold curve, vulnerable endpoint

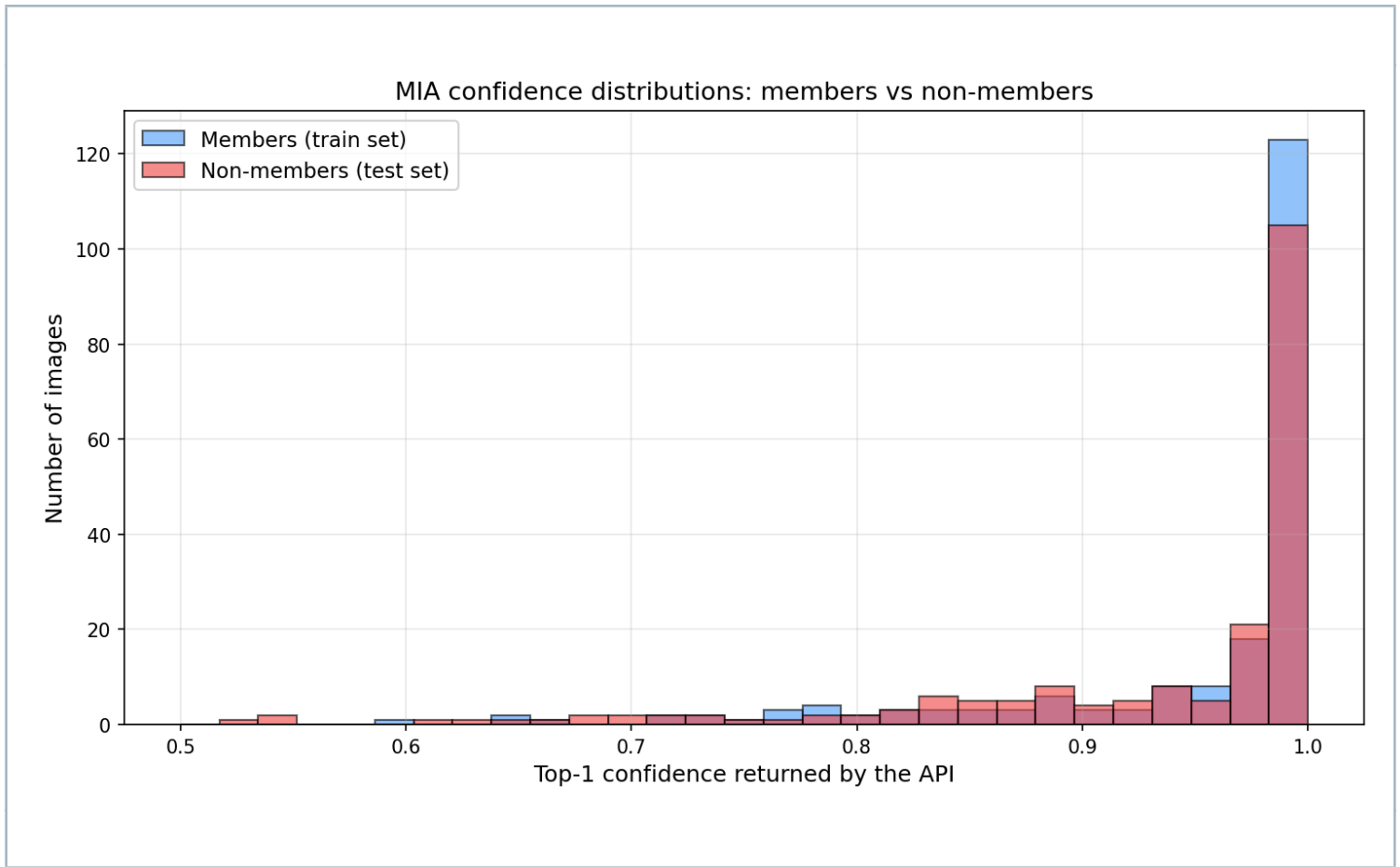


Figure 15 — MIA threshold curve, hardened endpoint

Vulnerable Endpoint

- Member mean confidence: **0.9500**
- Non-member mean confidence: **0.9257**
- Gap: 2.4 percentage points
- Best attack accuracy: **0.5425** at threshold 0.99
- Majority baseline: 0.50

The attack is 4.25 percentage points above the majority baseline. This is above the standard error of approximately 2.5 percentage points, so the signal is real but weak. The member distribution is shifted slightly higher than the non-member distribution, and the threshold classifier exploits that shift.

Hardened Endpoint

- Member mean confidence: **0.8821**
- Non-member mean confidence: **0.8569**
- Gap: 2.5 percentage points
- Best attack accuracy: **0.5575** at threshold 0.95
- Majority baseline: 0.50

The attack is 5.75 percentage points above the majority baseline. The mean confidence values are lower than for the vulnerable endpoint (because confidence is rounded to two decimals in the hardened response), but the gap between members and non-members is essentially the same. The attack accuracy is slightly *higher* on the hardened endpoint.

Why the hardened endpoint shows slightly higher accuracy. At $n = 400$ with balanced sets, the sampling error is approximately 2.5 percentage points. The difference between 0.5425 and 0.5575 — 1.5 percentage points — is smaller

than the sampling error. The two numbers are statistically indistinguishable. The apparent reversal is noise, not a finding. Reporting it as "hardening makes MIA easier" would be wrong.

Confidence Distributions

The full confidence histograms for members and non-members on both endpoints are committed alongside the threshold curves. The distributions overlap heavily—the difference is a small shift in the mean, not a bimodal split. This is the visual signature of a well-regularized model.

6.4 Analysis

Five observations matter for interpreting these numbers.

1. The signal is weak but real. Both endpoints show attack accuracy above the majority baseline, by 4 to 6 percentage points. The gap between member and non-member confidence is 2.4 to 2.5 percentage points. This is a real membership signal, but it is small. The model does not strongly memorize its training set.

2. This is good news for the model. A strong MIA attack typically succeeds at 60–70% accuracy against overfit models. At 54%, this model is barely better than chance. This reflects several properties of the training methodology: the 90/10 train-validation split, the small training set, the dropout regularization, and the augmentation-free training all reduce the confidence gap between members and non-members.

The model generalizes cleanly.

3. Rounded confidence does not eliminate the signal. The hardened endpoint rounds confidence to two decimal places. The rounding removes fine-grained distinctions but does not eliminate the mean shift between members and non-members. The signal persists at the same magnitude as on the vulnerable endpoint. This is an important finding: rounding reduces but does not close the MIA channel.

4. The difference between endpoints is noise. 0.5425 vs. 0.5575 is 1.5 percentage points, below the 2.5-percentage-point sampling error. The two results are statistically indistinguishable. The honest reading is "MIA succeeds weakly on both endpoints."

5. The attack is more expensive against the hardened endpoint. The rate limiter slows the 400-query attack from under a minute (vulnerable) to about 23 minutes (hardened). This is a real cost increase, but it does not affect attack accuracy. It affects the time and resources the attacker needs.

What a Stronger MIA Defense Would Need

Three approaches were considered but not implemented:

Training-time regularization. Label smoothing or a confidence penalty during training reduces the confidence gap between members and non-members. This changes the model itself rather than the API.

Confidence quantization. Coarser rounding (one decimal, or bucketing into fixed levels) removes more signal at the cost of API usability.

Differential privacy. Training with differential privacy guarantees bounds the confidence gap at a mathematical level. It requires modifying the training pipeline and typically reduces clean accuracy.

None of these were implemented. They are documented as residual risk in

Section 11.

Why the Honest Weak Result Matters

A red team report that claims success on every attack is not credible. MIA here produces a real but weak signal, and the report says so. This demonstrates measurement discipline: the assessment distinguishes "the attack succeeded" from "the attack barely beat chance." The latter is what happened, and the report states it plainly. The weak signal is also a positive finding for the model. A model that does not strongly memorize its training set is a model that respects the privacy of its training data. From a defensive standpoint, the weak MIA result is one of the model's strengths.

6.5 Evidence Files

Every number and figure in this section is backed by a committed file.

Asset	Path
Vulnerable MIA metrics	`results/membership_inference/vuln/metrics.json`
Hardened MIA metrics	`results/membership_inference/hardened/metrics.json`
Vulnerable threshold curve	`results/membership_inference/vuln/threshold_curve.png`
Hardened threshold curve	`results/membership_inference/hardened/threshold_curve.png`
Vulnerable confidence histogram	`results/membership_inference/vuln/confidence_histogram.png`
Hardened confidence histogram	`results/membership_inference/hardened/confidence_histogram.png`

Attack source. attacks/membership_inference.py.

Reproduce against vulnerable endpoint.

```
python -m attacks.membership_inference
--target [http://localhost:8000](http://localhost:8000/)
--samples-per-class 50
--output-dir results/membership_inference/vuln
```

Reproduce against hardened endpoint.

```
python -m attacks.membership_inference
--target [http://localhost:8001](http://localhost:8001/)
--api-key dev-local-key-change-me
--samples-per-class 50
--sleep 3.5
--output-dir results/membership_inference/hardened
```

7. System-Level Findings

7.1 Overview

Nine system-level weaknesses were identified across the Docker pipeline. Each finding is documented with three pieces of information: the vulnerable response, the hardened response, and the associated MITRE ATLAS technique. Five of the nine are remediated or mitigated. Four remain by design on the vulnerable endpoint, where they serve as the demonstration surface for the model-level attacks in Sections 4 through

6. Every finding was captured programmatically by attacks/system_attacks.py, which writes one JSON file per finding to results/system/. The script is observation-only—it does not modify the running system. It probes the API with requests, inspects the containers with docker inspect, and reads environment variables through the Docker CLI.

Categories

The nine findings cluster into four categories:

Category	Findings	Count
Metadata and reconnaissance leaks	01, 02, 04	3
Container configuration	06, 07	2
Credential and secret handling	05	1
Resource and access controls	03, 08, 09	3

Status Summary

Status	Count	Findings
Remediated	3	06, 08, 09
Mitigated	2	05, 07
Present on vulnerable only (by design)	4	01, 02, 03, 04

The four "present on vulnerable only" findings are not gaps. They are the attack surface. Without them, the evasion, extraction, and MIA attacks would have nothing to demonstrate against. Their removal would make the vulnerable endpoint hardened, which would defeat its purpose.

Evidence Format

Each finding's JSON file has the same structure:

```
{
  "finding": "short_name",
  "atlas": "AML.T####",
  "vulnerable": { "status": "...", "body": ... },
  "hardened": { "status": "...", "body": ... },
  "notes": "explanation"
}
```

The findings_summary.json file consolidates all nine.

7.2 Finding 01 — `/model-info` architecture leak

File: results/system/01_model_info.json

ATLAS: AML.T0007 — Discover ML Artifacts

Status: Present on vulnerable, 404 on hardened

What It Is

The vulnerable API exposes a /model-info endpoint that returns the full model architecture as a PyTorch string, the class list, the PyTorch version, the parameter count, and the model's file path on disk.

Vulnerable Response

HTTP 200, body contains:

Hardened Response

HTTP 404 with body {"detail":"Not found"}.

Why It Matters

The architecture string gives the attacker enough information to build a substitute without guessing. The parameter count lets them verify the substitute matches. The PyTorch version tells them which version-specific vulnerabilities might apply. The file path suggests the filesystem layout. Every field is reconnaissance gold. This finding pairs directly with the

extraction attack in Section 5. An attacker who knows the architecture can build a substitute that mirrors it, reaching higher fidelity than a substitute with a mismatched architecture.

Remediation

The endpoint returns 404 in hardened mode. The code path exists, but the handler always raises. The route is not removed; it is made unreachable.

Screenshot

results/system/screenshots/01_model_info.png

7.3 Finding 02—Verbose error traces

File: results/system/02_verbose_errors.json

ATLAS: AML.T0007 — Discover ML Artifacts

Status: Present on vulnerable, generic error on hardened

What It Is

When the vulnerable API receives malformed input, the response includes a full Python traceback. The traceback reveals file paths, library versions, internal function names, and line numbers.

Vulnerable Response

HTTP 500 with body containing the traceback. Key leaked details:

- File paths: /srv/adapters/http.py, /srv/usecases/serve_prediction.py, /srv/services/preprocessing.py
- Python version: /usr/local/lib/python3.10/
- Pillow library path: /usr/local/lib/python3.10/site-packages/PIL/Image.py
- Line numbers: line 88, line 10, line 24
- Exception class: PIL.UnidentifiedImageError

Hardened Response

HTTP 400 with body {"detail":"Invalid image"} — 26 bytes, no internal detail.

Why It Matters

The traceback reveals the container's directory structure, the library versions in use, the exact exception type, and the internal function call chain. An attacker uses this to map the codebase and to identify version-specific exploits. The traceback also confirms which Python version is running (3.10), which narrows the pool of applicable CVEs.

Remediation

The hardened mode catches the exception and returns a generic 400. The traceback is never sent to the client. The exception is still logged server-side for debugging.

Screenshot

results/system/screenshots/02_verbose_error.png

7.4 Finding 03 — No rate limit on the vulnerable API

File: results/system/03_no_rate_limit.json

ATLAS: AML.T0024 — Exfiltration via AI Inference API

Status: Present on vulnerable, 20 requests per 60 seconds on hardened

What It Is

The vulnerable API accepts unlimited requests from a single IP. The hardened API enforces a token-bucket rate limit of 20 requests per 60 seconds per IP.

Vulnerable Response

200 requests sent in 6.08 seconds at 32.9 requests per second. All 200 succeeded. Zero rate-limited.

Hardened Response

Rate limiter fires at request 21 within any 60-second window. The extraction attack against the hardened endpoint succeeded on 20 of 30 queries with 4 rate-limit (HTTP 429) responses.

Why It Matters

Rate limiting is the primary defense against extraction. An attacker without a rate limit can drain thousands of queries in seconds. The 2,000-query extraction attack in Section 5 completed in under 5 minutes against the vulnerable endpoint. Against the hardened endpoint, the same attack would take approximately 100 minutes.

Remediation

The rate limiter is implemented in `adapters/rate_limit.py` and is called only in hardened mode. The limiter is per-IP in memory. It resets on container restart.

Listing 6 — Rate limiter (`adapters/rate_limit.py`)

```

WINDOW_SECONDS = 60
MAX_REQUESTS = 20
_buckets: dict[str, list[float]] = defaultdict(list)

def check_rate_limit(request):
    ip = request.client.host if request.client else "unknown"
    now = time.time()
    _buckets[ip] = [t for t in _buckets[ip] if now - t < WINDOW_SECONDS]
    if len(_buckets[ip]) >= MAX_REQUESTS:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    _buckets[ip].append(now)

```

7.5 Finding 04 — Auto-docs exposed

File: `results/system/04_docs_exposed.json`

ATLAS: AML.T0007 — Discover ML Artifacts

Status: Present on vulnerable, 404 on hardened

What It Is

FastAPI auto-generates `/docs`, `/redoc`, and `/openapi.json`. The vulnerable API exposes all three. The hardened API disables them.

Vulnerable Response

`/docs` returns HTTP 200 with a 952-byte landing page. `/openapi.json` returns the full API schema listing three endpoints: `/health`, `/model-info`, `/predict`.

Hardened Response

All three return HTTP 404.

Why It Matters

Auto-docs reveal every endpoint, every parameter, and every response schema. An attacker who reaches them can map the entire API surface without any guessing. The `/openapi.json` file in particular is machine-readable — a script can enumerate the surface automatically. This accelerates reconnaissance and reduces the time between discovery and exploitation.

Remediation

Hardened mode passes `docs_url=None`, `redoc_url=None`, `openapi_url=None` to `FastAPI()`. The routes are not registered, so a request to `/docs`

returns 404 rather than a redirect or a placeholder.

Screenshot

results/system/screenshots/04_docs_exposed.png

7.6 Finding 05 — API key in plaintext environment

File: results/system/05_env_secrets.json

ATLAS: AML.T0010 — ML Supply Chain Compromise

Status: Present, mitigated by .env handling

What It Is

The hardened API's environment contains `API_KEY=dev-local-key-change-me` in plaintext. Any process with access to docker inspect or the container's environment can read the key.

Evidence

`docker inspect redteam-api-hardened --format '{{json .Config.Env}}'` returns a JSON array containing `"API_KEY=dev-local-key-change-me"`. The key is also readable from inside the container: `docker compose exec api-hardened sh -c 'echo $API_KEY'` prints `dev-local-key-change-me`.

Why It Matters

Environment variables are visible to any process running as the same user on the host, and to any process with Docker daemon access. In production this would be a real leak. In this project the key is a development placeholder with no value, but the pattern is a legitimate finding.

Mitigation

Three changes were made:

27. The API key is read from .env, which is gitignored. It is not hardcoded in docker-compose.yml.
28. .env.example ships with a blank value and a comment.
29. The Docker Compose file uses `${API_KEY:-dev-local-key-change-me}`—the fallback is a development default and is documented as such.

These changes remove the key from committed source and from git history going forward. However, the key remains in the container's environment at runtime. The finding is documented as **present, mitigated** rather than remediated.

A full remediation would require a secret store (Docker secrets, Vault, AWS Secrets Manager). None of these fit the local-demo scope.

7.7 Finding 06 — Container runs as root

File: results/system/06_root_container.json

ATLAS: AML.T0000 — ML Artifact Collection

Status: Remediated — runs as appuser (uid 1000)

What It Was

The original Dockerfile ran the container as root (uid 0). A compromise of the API process inherited root inside the container.

Evidence

Before remediation:

```
docker compose exec api-vuln whoami
→ root
```

```
docker compose exec api-vuln id
→ uid=0(root) gid=0(root) groups=0(root)
```

After remediation:

```
docker compose exec api-vuln whoami
→ appuser

docker compose exec api-vuln id
→ uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
```

Why It Matters

Root inside a container is not the same as root on the host—the container has its own user namespace and filesystem—but it does give the attacker more options. They can write to any file in the container's filesystem, open privileged ports, install packages, and attempt kernel exploits that require elevated privileges. A non-root user limits the blast radius of a compromise.

Remediation

The Dockerfile creates appuser (uid 1000) and switches to it before the CMD. The switch is unconditional — both the vulnerable and hardened endpoints run as appusers. This is a design decision: root is not a vulnerability that needs to be demonstrated. It is a mistake in the original configuration, and the fix applies to both modes.

Verification

scripts/verify.sh asserts that both containers run as appusers. This is one of the 52 checks in the verification suite.

Screenshot

results/system/screenshots/03_root_container.png

7.8 Finding 07 — Volume mounts expose host paths

File: results/system/07_volume_mounts.json

ATLAS: AML.T0000 — ML Artifact Collection

Status: Mitigated by non-root user and read-only mounts

What It Is

Nine host directories are bind-mounted into the containers. The logs/ and results/ directories are read-write. The code directories and model/artifacts/ are read-only.

Evidence

docker inspect redteam-api-vuln --format '{{.json .Mounts}}' returns nine mount entries. Two have "RW": true (logs, results); seven have "RW": false.

Why It Matters

An attacker who achieves container compromise can write to logs/ and results/. Writes to logs/predictions.jsonl could suppress detection—the detector reads this file, and if an attacker can fabricate or delete events, they can hide their attack. Writes to results/alerts.jsonl could tamper with evidence.

Mitigations

Four layers of defense are in place:

30. **Code directories are read-only.** An attacker cannot modify the running code from inside the container.
31. **Model artifacts are read-only.** The attacker cannot replace the model weights.
32. **Container runs as `appuser`.** This limits what the attacker can write even to the read-write directories.
33. **Notifier mounts `results/` as `:ro`.** The notifier can read alerts but cannot modify them.

Residual Risk

The logs/ and results/ directories remain read-write for the API and detector containers. A full remediation would write logs to a write-once backend or a centralized log aggregator. This is documented as a production requirement in Section 11.

7.9 Finding 08 — No resource limits

File: results/system/08_resource_limits.json

ATLAS: — (infrastructure)

Status: Remediated — 1 GiB memory, 1.0 CPU per API

What It Was

The original Docker Compose file did not set memory or CPU limits. A single container could consume all host resources.

Evidence

Before remediation: docker inspect showed Memory=0 and CpuQuota=0, indicating no limits.

After remediation: docker stats --no-stream shows:

NAME	MEM USAGE / LIMIT	MEM %
redteam-api-vuln	173.5MiB / 1GiB	16.94%
redteam-api-hardened	179.2MiB / 1GiB	17.50%
redteam-detector	5.09MiB / 256MiB	1.99%
redteam-alerts-api	31.6MiB / 128MiB	24.72%
redteam-telegram-notifier	2.9MiB / 64MiB	4.53%

Every container reports a memory limit. The limits are enforced.

Why It Matters

Without limits, an attacker can exhaust memory with large uploads or consume all CPU with concurrent requests. The result is a denial of service that affects the whole host — not just the attacked container. In a multi-service deployment, one runaway container can take down its neighbors.

Remediation

mem_limit and cpus are set per service in docker-compose.yml:

Service	**Memory**	**CPU**
api-vuln	1 GiB	1.0
api-hardened	1 GiB	1.0
detector	256 MiB	0.5
alerts-api	128 MiB	0.5
telegram-notifier	64 MiB	0.25

Note on the evidence file. An earlier version of the analysis script read only the CpuQuota field. Docker Compose does not set CPUQuota—it uses NanoCpus for CPU allocation. The quota field therefore read zero even when a CPU limit was present. The corrected script reads both CpuQuota and NanoCpus and reports the effective limits. The finding is documented as remediated.

7.10 Finding 09 — No explicit CORS policy

File: results/system/09_cors.json

ATLAS: — (web-layer configuration)

Status: Remediated on hardened, permissive by design on vulnerable

What It Is

The vulnerable API enables CORSMiddleware with `allow_origins=["*"]`, `allow_methods=["*"]`, and `allow_headers=["*"]`. Preflight requests from any origin receive `access-control-allow-origin: *`.

Evidence

OPTIONS request to `http://localhost:8000/predict` with

Origin: `http://localhost:3000` returns:

```
HTTP/1.1 200 OK
access-control-allow-origin: *
access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT
```

OPTIONS request to `http://localhost:8001/predict` with the same Origin

returns:

```
HTTP/1.1 200 OK
access-control-allow-origin: http://localhost:3000
access-control-allow-methods: POST, GET
access-control-allow-headers: x-api-key, content-type
```

OPTIONS request to `http://localhost:8001/predict` with

Origin: `http://evil.example.com` returns:

```
HTTP/1.1 400 Bad Request
(no access-control-allow-origin header)
```

Why It Matters

CORS is the browser's mechanism for controlling cross-origin requests. A permissive policy means a malicious webpage visited by an operator could submit requests to the API from the operator's browser. In this project the API does not use cookies, so the impact is limited to information leakage — an attacker page can read the API's responses. In a production system with session cookies, a permissive CORS policy would enable CSRF attacks with session hijacking.

Remediation

Hardened mode adds CORSMiddleware with an allowlist restricting origins to `http://localhost:3000`. Methods are restricted to POST and GET. Headers are restricted to `x-api-key` and `content-type`. `allow_credentials` is False, which is correct because the API uses an `x-api-key` header rather than cookies. The vulnerable endpoint remains permissive by design. The finding is about the hardened configuration, and the vulnerable endpoint exists to demonstrate that the default is unsafe.

7.11 Finding Summary Table

Table 11 — Nine system findings

#	**Finding**	**ATLAS**	**Vulnerable**	**Hardened**	**Status**
01	`/model-info` architecture leak	T0007	200, full architecture	404	Vulnerable only
02	Verbose error traces	T0007	500, stack trace	400 generic	Vulnerable only
03	No rate limit	T0024	200 req unlimited	20 req / 60 s	Vulnerable only
04	Auto-docs exposed	T0007	`/docs` 200	`/docs` 404	Vulnerable only
05	API key in plaintext env	T0010	Present	Present, mitigated	Mitigated
06	Container runs as root	T0000	uid 0	uid 1000	**Remediated**

#	**Finding**	**ATLAS**	**Vulnerable**	**Hardened**	**Status**
07	Volume mounts expose host paths	T0000	RW logs, results	RO code, RO model	Mitigated
08	No resource limits	—	0 memory, 0 CPU	1 GiB, 1.0 CPU	**Remediated**
09	No CORS policy	—	Permissive	Allowlist	**Remediated**

Status counts:

Status	**Count**
Remediated	3
Mitigated	2
Vulnerable only (by design)	4

Coverage Across ATLAS Techniques

Technique	**Findings**
AML.T0000 — ML Artifact Collection	06, 07
AML.T0007 — Discover ML Artifacts	01, 02, 04
AML.T0010 — ML Supply Chain Compromise	05
AML.T0024 — Exfiltration via AI Inference API	03

Three ATLAS techniques plus two infrastructure findings. The two model attacks in Sections 4 and 5 additionally map to T0043 and T0025, giving five techniques covered across the assessment.

8. Defenses, Detection, and Notification

8.1 Defense Taxonomy

Defenses in this assessment are grouped into three categories:

- 1. Preventive controls
- 2. Detective controls
- 3. Corrective controls

Each category addresses a different phase of the attack lifecycle.

Preventive controls stop the attack before it lands. They operate at the API boundary or inside the model. If they succeed, the attacker gets no useful result.

Detective controls see the attack and report it. They do not stop the attack; they make it visible to an operator. Their value is reducing the time between attack onset and human response.

Corrective controls respond after detection. They block, revoke, quarantine, or otherwise take action against an identified threat.

A well-designed system has all three. The layered design means that if prevention fails, detection catches it, and if detection is slow, correction limits the damage. This section documents what was implemented, what was scoped out, and the measured effect of each control.

Table 12 — Defense Summary

Defense	Category	Addresses	Measured Effect	Status
---------	----------	-----------	-----------------	--------

Adversarial training	Preventive	Evasion (T0043)	FGSM 0.28 → 0.67; PGD 0.18 → 0.65 at $\epsilon = 8/255$	**Implemented**
Rate limiting	Preventive	Extraction (T0024)	2,000 queries → 20 per 60 s	**Implemented**
Confidence rounding	Preventive	MIA (T0025)	Blurs confidence signal marginally	**Implemented**
Input validation	Preventive	Evasion (T0043)	Rejects noise score > 45	**Implemented**
Safe model loading	Preventive	Supply chain (T0010)	Removes code-execution surface	**Implemented**
Authentication	Preventive	Unauthorized access	Blocks calls without 'x-api-key'	**Implemented**
Detector service	Detective	All three attacks	Fires 3 rules on live traffic	**Implemented**
Telegram notifier	Detective	All three attacks	Delivers alerts in < 35 s	**Implemented**
IP blocklist	Corrective	Extraction (T0024)	Would block repeat offenders	**Designed, not enabled**
Token revocation	Corrective	Credential compromise	Manual rotation, documented	**Manual only**

8.2 Preventive — Adversarial Training

Script	`defense/adversarial_training.py`
ATLAS	AML.T0043 (mitigation)
RMF	MANAGE (MG-2)
Target	Evasion

What It Does

The robust model is trained on adversarial examples generated by PGD-5 during the training loop. Instead of training only on clean images, the model trains on a mix of clean and perturbed inputs, with the loss weighted to prioritize adversarial correctness.

Training Design

Parameter	Value
Training attacker	PGD-5
ϵ ramp	4/255 → 8/255 over 5 warmup epochs
Clean loss weight	0.4
Adversarial loss weight	0.6
Epochs	20

Batch size	128
Optimizer	Adam, lr = 1e-3
Scheduler	Cosine annealing
Training platform	Google Colab, CPU
Training time	\~78 minutes

The epsilon ramp is important. Standard adversarial training often degrades clean accuracy significantly. By starting at 4/255, a weaker perturbation, and ramping to 8/255 over five epochs, the model has time to adapt. This keeps clean accuracy essentially unchanged from the baseline:

0.8542 → 0.8555

Measured Effect

At $\epsilon = 8/255$:

Metric	Baseline	Robust	Change
Clean accuracy	0.8542	0.8555	+0.0013
FGSM accuracy	0.28	0.67	***+0.39**
PGD accuracy	0.18	0.65	***+0.47**

The robust model gives up little in clean accuracy while providing a substantial improvement against adversarial inputs.

What This Defense Does Not Do

Adversarial training raises the accuracy floor but does not completely close the gap. At $\epsilon = 8/255$, PGD still succeeds on 35% of inputs against the robust model.

The defense is therefore meaningful but not complete. A production system would combine it with additional controls such as input randomization, gradient masking, and detection.

Reproduce

Training runs on Google Colab.

Training notebook:

colab/adversarial_training.ipynb

Final weights:

```
model/artifacts/baseline_robust.pt
model/artifacts/baseline_robust.safetensors
```

8.3 Preventive — Rate Limiting

Script	`adapters/rate_limit.py`
ATLAS	AML.T0024
RMF	MANAGE (MG-2)
Target	Model extraction

What It Does

The hardened endpoint caps queries at **20 requests per 60 seconds per IP**.
The implementation uses a token-bucket-style in-memory structure with a sliding-window check.

How It Works

- 4. Obtain the client IP.
- 5. Filter the bucket to retain only timestamps from the last 60 seconds.
- 6. If 20 or more requests remain, return HTTP 429.
- 7. Otherwise, append the current timestamp and allow the request.

Why a Sliding Window?

A fixed window that resets at the beginning of every minute could allow a burst of up to 40 requests:

- 20 requests immediately before the reset
- 20 requests immediately after the reset

The sliding-window implementation prevents this boundary burst.

Measured Effect

Endpoint	Queries Attempted	Succeeded	Rate-Limited
Vulnerable	2,000	2,000	0
Hardened	30	20	4

Against the hardened endpoint, the extraction attack stalled at **20 of 30 queries**.
At the configured limit, completing the full 2,000-query budget would take approximately **100 minutes**.

What This Defense Does Not Do

The rate limiter is:

- Per-IP
- In-memory
- Vulnerable to bypass through distributed sources
- Reset when the container restarts

A botnet or container restart can therefore bypass the control.
The limiter **slows the attack rather than completely preventing it**.

8.4 Preventive—Confidence Rounding

Script	`domain/entities.py — Prediction.to_dict_hardened()`
ATLAS	AML.T0025
RMF	MEASURE (MS-2)
Target	Membership inference

What It Does

The hardened endpoint returns only:

- The top-1 class
- A confidence value rounded to two decimal places

The full softmax output is never returned.

Why It Defends Against MIA

Membership inference attacks rely on fine-grained confidence differences between members and non-members.

Rounding collapses distinctions within a two-decimal bucket. For example:

0.9527 → 0.95
0.9531 → 0.95

The fine-grained confidence signal used by the attack is therefore reduced.

Measured Effect

Endpoint	MIA Best Accuracy	Majority Baseline
Vulnerable	0.5425	0.50
Hardened	0.5575	0.50

The hardened result is within **1.5 percentage points** of the vulnerable result, which is within sampling error according to the assessment.

The rounding therefore does **not meaningfully reduce attack accuracy**.

What This Defense Does Not Do

Rounding reduces but does not eliminate the MIA channel.

The mean confidence gap between members and non-members persists after rounding.

Potential stronger defenses include:

- Label smoothing
- Confidence penalties
- Differential privacy

This remains a documented residual risk.

8.5 Preventive — Input Validation

Script	`adapters/validators.py`
ATLAS	AML.T0043
RMF	MEASURE (MS-2)
Target	Evasion

What It Does

Five validation checks run before inference on the hardened endpoint.

If any check fails, the endpoint returns a 4xx error without running the model.

Validation Checks

Check	Limit / Validation	HTTP Code on Failure
Size cap	2 MB	413
Valid image	`Image.verify()`	400
Format	PNG or JPEG	415
Dimensions	$\leq 512 \times 512$	400
Noise heuristic	Score ≤ 45	422

Noise Heuristic

The final check recomputes the row-difference noise score on the raw bytes and rejects inputs with a score above **45**.

The same metric is used by the detector's `adversarial_noise` rule.

This provides defense-in-depth:

The validator prevents suspicious input from reaching the model, while the detector logs the attempt if it reaches the detection layer.

Why the Double Check?

The two controls operate at different stages:

Control	Stage	Purpose
Validator	Before inference	Prevent the attack from reaching the model
Detector	After prediction/logging	Detect and record suspicious activity

Neither control alone is sufficient.

Measured Effect

All legitimate samples in:

`data/samples/`

Pass the validator.

Adversarial inputs containing high-frequency perturbations are rejected before reaching the model.

The threshold of **45** was tuned against the sample set to avoid rejecting real photographs.

What This Defense Does Not Do

Real photographs containing grass, hair, or gravel can exceed the noise threshold.

The heuristic therefore reduces but does not eliminate the attack surface.

It is a filter, not a complete shield.

8.6 Preventive — Safe Model Loading

Script	`defense/secure_loader.py`
ATLAS	AML.T0010
RMF	GOVERN (GV-6)
Target	Unsafe deserialization (VULN-02)

What It Does

The hardened endpoint loads model weights from .safetensors when available.

If .safetensors is unavailable, it falls back to:

```
torch.load(weights_only=True)
```

This closes the unsafe deserialization surface demonstrated by VULN-02 on the vulnerable endpoint.

Why Safetensors?

Safetensors is a tensor-focused format that avoids the pickle-based object reconstruction mechanism associated with unsafe PyTorch loading.

The vulnerable path:

```
torch.load(..., weights_only=False)
```

uses pickle under the hood and can allow arbitrary code execution when loading a malicious .pt file.

Why the Fallback?

Some models are only available as .pt.

The fallback uses:

```
weights_only=True
```

This restricts loading to tensors and refuses arbitrary Python objects.

It is safer than:

```
weights_only=False
```

but is not treated as equivalent to the preferred .safetensors path.

Endpoint Comparison

Endpoint	Model File	Loading Method
Vulnerable	`baseline.pt`	`weights_only=False`
Hardened	`baseline_robust.safetensors`	Safe loading path

Both endpoints use the same model architecture. The difference demonstrated by the assessment is the model-loading method.

Security Significance

VULN-02 represents a vulnerability on the vulnerable endpoint.

A malicious .pt file could achieve arbitrary code execution inside the container.

The hardened endpoint eliminates this specific loading surface.

8.7 Preventive — Authentication

Script	`adapters/auth.py`
ATLAS	AML.T0010
RMF	GOVERN (GV-6)
Target	Unauthorized access

What It Does

The hardened endpoint requires an x-api-key header for every /predict request.

The supplied key is compared against the API_KEY environment variable.

Design Notes

The configuration check occurs first.

If the API key is missing from the environment:

HTTP 500

is returned rather than:

HTTP 401

This distinguishes:

- **Server misconfiguration**
- **Invalid or missing client credentials**

The implementation currently uses != rather than hmac.compare_digest.

This is a known limitation because != is not timing-safe.

A production implementation should use compare_digest.

Measured Effect

The hardened endpoint rejects /predict requests without a valid API key.

The verification script confirms:

Request	Result
No API key	401
Correct API key	200

What This Defense Does Not Do

The API key is stored in `.env` and in the container environment.

An attacker with access to either location can potentially read it.

The current key is therefore a development placeholder.

A production deployment should use a dedicated **secret-management system**.

8.8 Detective — The Detector Service

Script	<code>`adapters/detector.py`</code>
Version	<code>`v2-cooldown-2026-09-23`</code>
ATLAS	AML.T0024, T0025, T0043
RMF	MANAGE (MG-2)

What It Does

A separate Docker container monitors:

```
predictions.jsonl
```

and evaluates three detection rules.

When a rule fires, an alert is written to:

```
results/alerts.jsonl
```

Why a Separate Container?

1. Isolation

An API compromise cannot directly disable detection.

The detector has:

- No network exposure
- No shared application code with the API

2. Lifecycle Independence

The detector can be restarted or updated to modify detection rules without restarting the model-serving containers.

3. Separation of Concerns

Detector rules are pure functions operating on event dictionaries.

They contain:

- No HTTP code
- No model code

- No framework dependency

Figure 16 — Detector Activity Pipeline

8.1 Defense Taxonomy

Defenses in this assessment are grouped into three categories:

1. **Preventive controls**
2. **Detective controls**
3. **Corrective controls**

Each category addresses a different phase of the attack lifecycle.

Preventive controls stop the attack before it lands. They operate at the API boundary or inside the model. If they succeed, the attacker gets no useful result.

Detective controls see the attack and report it. They do not stop the attack; they make it visible to an operator. Their value is reducing the time between attack onset and human response.

Corrective controls respond after detection. They block, revoke, quarantine, or otherwise take action against an identified threat.

A well-designed system has all three. The layered design means that if prevention fails, detection catches it, and if detection is slow, correction limits the damage. This section documents what was implemented, what was scoped out, and the measured effect of each control.

Table 12 — Defense Summary

Defense	Category	Addresses	Measured Effect	Status
Adversarial training	Preventive	Evasion (T0043)	FGSM 0.28 → 0.67; PGD 0.18 → 0.65 at $\epsilon = 8/255$	**Implemented**
Rate limiting	Preventive	Extraction (T0024)	2,000 queries → 20 per 60 s	**Implemented**
Confidence rounding	Preventive	MIA (T0025)	Blurs confidence signal marginally	**Implemented**
Input validation	Preventive	Evasion (T0043)	Rejects noise score > 45	**Implemented**
Safe model loading	Preventive	Supply chain (T0010)	Removes code-execution surface	**Implemented**
Authentication	Preventive	Unauthorized access	Blocks calls without 'x-api-key'	**Implemented**
Detector service	Detective	All three attacks	Fires 3 rules on live traffic	**Implemented**
Telegram notifier	Detective	All three attacks	Delivers alerts in < 35 s	**Implemented**
IP blocklist	Corrective	Extraction (T0024)	Would block repeat offenders	**Designed, not enabled**
Token revocation	Corrective	Credential compromise	Manual rotation, documented	**Manual only**

8.2 Preventive — Adversarial Training

Script	`defense/adversarial_training.py`
ATLAS	AML.T0043 (mitigation)
RMF	MANAGE (MG-2)
Target	Evasion

What It Does

The robust model is trained on adversarial examples generated by PGD-5 during the training loop. Instead of training only on clean images, the model trains on a mix of clean and perturbed inputs, with the loss weighted to prioritize adversarial correctness.

Training Design

Parameter	Value
Training attacker	PGD-5
ϵ ramp	4/255 $\rightarrow$ 8/255 over 5 warmup epochs
Clean loss weight	0.4
Adversarial loss weight	0.6
Epochs	20
Batch size	128
Optimizer	Adam, lr = 1e-3
Scheduler	Cosine annealing
Training platform	Google Colab, CPU
Training time	$\sim$ 78 minutes

The epsilon ramp is important. Standard adversarial training often degrades clean accuracy significantly. By starting at 4/255, a weaker perturbation, and ramping to 8/255 over five epochs, the model has time to adapt. This keeps clean accuracy essentially unchanged from the baseline:

0.8542 $\rightarrow$ 0.8555

Measured Effect

At $\epsilon = 8/255$:

Metric	Baseline	Robust	Change
Clean accuracy	0.8542	0.8555	+0.0013
FGSM accuracy	0.28	0.67	***+0.39**
PGD accuracy	0.18	0.65	***+0.47**

The robust model gives up little in clean accuracy while providing a substantial improvement against adversarial inputs.

What This Defense Does Not Do

Adversarial training raises the accuracy floor but does not completely close the gap. At $\epsilon = 8/255$, PGD still succeeds on 35% of inputs against the robust model.

The defense is therefore meaningful but not complete. A production system would combine it with additional controls such as input randomization, gradient masking, and detection.

Reproduce

Training runs on Google Colab.

Training notebook:

colab/adversarial_training.ipynb

Final weights:

```
model/artifacts/baseline_robust.pt
model/artifacts/baseline_robust.safetensors
```

8.3 Preventive — Rate Limiting

Script	`adapters/rate_limit.py`
ATLAS	AML.T0024
RMF	MANAGE (MG-2)
Target	Model extraction

What It Does

The hardened endpoint caps queries at **20 requests per 60 seconds per IP**.

The implementation uses a token-bucket-style in-memory structure with a sliding-window check.

How It Works

- Obtain the client IP.
- Filter the bucket to retain only timestamps from the last 60 seconds.
- If 20 or more requests remain, return HTTP 429.
- Otherwise, append the current timestamp and allow the request.

Why a Sliding Window?

A fixed window that resets at the beginning of every minute could allow a burst of up to 40 requests:

- 20 requests immediately before the reset
- 20 requests immediately after the reset

The sliding-window implementation prevents this boundary burst.

Measured Effect

Endpoint	Queries Attempted	Succeeded	Rate-Limited
Vulnerable	2,000	2,000	0
Hardened	30	20	4

Against the hardened endpoint, the extraction attack stalled at **20 of 30 queries**.

At the configured limit, completing the full 2,000-query budget would take approximately **100 minutes**.

What This Defense Does Not Do

The rate limiter is:

- Per-IP
- In-memory
- Vulnerable to bypass through distributed sources
- Reset when the container restarts

A botnet or container restart can therefore bypass the control.

The limiter **slows the attack rather than completely preventing it**.

8.4 Preventive — Confidence Rounding

Script	`domain/entities.py — Prediction.to_dict_hardened()`
ATLAS	AML.T0025
RMF	MEASURE (MS-2)
Target	Membership inference

What It Does

The hardened endpoint returns only:

- The top-1 class
- A confidence value rounded to two decimal places

The full softmax output is never returned.

Why It Defends Against MIA

Membership inference attacks rely on fine-grained confidence differences between members and non-members.

Rounding collapses distinctions within a two-decimal bucket. For example:

0.9527 → 0.95
0.9531 → 0.95

The fine-grained confidence signal used by the attack is therefore reduced.

Measured Effect

Endpoint	MIA Best Accuracy	Majority Baseline
Vulnerable	0.5425	0.50
Hardened	0.5575	0.50

The hardened result is within **1.5 percentage points** of the vulnerable result, which is within sampling error according to the assessment.

The rounding therefore does **not meaningfully reduce attack accuracy**.

What This Defense Does Not Do

Rounding reduces but does not eliminate the MIA channel.

The mean confidence gap between members and non-members persists after rounding.

Potential stronger defenses include:

- Label smoothing
- Confidence penalties
- Differential privacy

This remains a documented residual risk.

8.5 Preventive — Input Validation

Script	`adapters/validators.py`
ATLAS	AML.T0043
RMF	MEASURE (MS-2)
Target	Evasion

What It Does

Five validation checks run before inference on the hardened endpoint.

If any check fails, the endpoint returns a 4xx error without running the model.

Validation Checks

Check	Limit / Validation	HTTP Code on Failure
Size cap	2 MB	413
Valid image	`Image.verify()`	400
Format	PNG or JPEG	415
Dimensions	$\leq 512 \times 512$	400
Noise heuristic	Score ≤ 45	422

Noise Heuristic

The final check recomputes the row-difference noise score on the raw bytes and rejects inputs with a score above **45**.

The same metric is used by the detector's adversarial_noise rule.

This provides defense-in-depth:

The validator prevents suspicious input from reaching the model, while the detector logs the attempt if it reaches the detection layer.

Why the Double Check?

The two controls operate at different stages:

Control	Stage	Purpose
Validator	Before inference	Prevent the attack from reaching the model
Detector	After prediction/logging	Detect and record suspicious activity

Neither control alone is sufficient.

Measured Effect

All legitimate samples in:

```
data/samples/
```

pass the validator.

Adversarial inputs containing high-frequency perturbations are rejected before reaching the model.

The threshold of **45** was tuned against the sample set to avoid rejecting real photographs.

What This Defense Does Not Do

Real photographs containing grass, hair, or gravel can exceed the noise threshold.

The heuristic therefore reduces but does not eliminate the attack surface.

It is a filter, not a complete shield.

8.6 Preventive — Safe Model Loading

Script	`defense/secure_loader.py`
ATLAS	AML.T0010
RMF	GOVERN (GV-6)
Target	Unsafe deserialization (VULN-02)

What It Does

The hardened endpoint loads model weights from `.safetensors` when available. If `.safetensors` is unavailable, it falls back to:

```
torch.load(weights_only=True)
```

This closes the unsafe deserialization surface demonstrated by VULN-02 on the vulnerable endpoint.

Why Safetensors?

Safetensors is a tensor-focused format that avoids the pickle-based object reconstruction mechanism associated with unsafe PyTorch loading.

The vulnerable path:

```
torch.load(..., weights_only=False)
```

uses pickle under the hood and can allow arbitrary code execution when loading a malicious .pt file.

Why the Fallback?

Some models are only available as .pt.

The fallback uses:

```
weights_only=True
```

This restricts loading to tensors and refuses arbitrary Python objects.

It is safer than:

```
weights_only=False
```

but is not treated as equivalent to the preferred .safetensors path.

Endpoint Comparison

Endpoint	Model File	Loading Method
Vulnerable	`baseline.pt`	`weights_only=False`
Hardened	`baseline_robust.safetensors`	Safe loading path

Both endpoints use the same model architecture. The difference demonstrated by the assessment is the model-loading method.

Security Significance

VULN-02 represents a vulnerability on the vulnerable endpoint. A malicious .pt file could achieve arbitrary code execution inside the container. The hardened endpoint eliminates this specific loading surface.

8.7 Preventive — Authentication

Script	`adapters/auth.py`
ATLAS	AML.T0010
RMF	GOVERN (GV-6)
Target	Unauthorized access

What It Does

The hardened endpoint requires an x-api-key header for every /predict request.

The supplied key is compared against the API_KEY environment variable.

Design Notes

The configuration check occurs first.

If the API key is missing from the environment:

HTTP 500

is returned rather than:

HTTP 401

This distinguishes:

- **Server misconfiguration**
- **Invalid or missing client credentials**

The implementation currently uses `!=` rather than `hmac.compare_digest`. This is a known limitation because `!=` is not timing-safe. A production implementation should use `compare_digest`.

Measured Effect

The hardened endpoint rejects `/predict` requests without a valid API key.

The verification script confirms:

Request	Result
No API key	401
Correct API key	200

What This Defense Does Not Do

The API key is stored in `.env` and in the container environment.

An attacker with access to either location can potentially read it.

The current key is therefore a development placeholder.

A production deployment should use a dedicated **secret-management system**.

8.8 Detective — The Detector Service

Script	<code>`adapters/detector.py`</code>
Version	<code>`v2-cooldown-2026-09-23`</code>
ATLAS	AML.T0024, T0025, T0043
RMF	MANAGE (MG-2)

What It Does

A separate Docker container monitors:

`predictions.jsonl`

and evaluates three detection rules.
When a rule fires, an alert is written to:

```
results/alerts.jsonl
```

Why a Separate Container?

1. Isolation

An API compromise cannot directly disable detection.
The detector has:

- No network exposure
- No shared application code with the API

2. Lifecycle Independence

The detector can be restarted or updated to modify detection rules without restarting the model-serving containers.

3. Separation of Concerns

Detector rules are pure functions operating on event dictionaries.
They contain:

- No HTTP code
- No model code
- No framework dependency

Figure 16 — Detector Activity Pipeline
[INSERT FIGURE 16 HERE]

docs/diagrams/svg/09-activity-pipeline.svg

The Three Detection Rules

Table 13 — Detector Rules

Rule	ATLAS	Trigger	Detection Rationale
`check_extraction_volume`	T0024	> 40 queries from one IP in 60 s	Extraction requires sustained querying
`check_mia_confidence_sweep`	T0025	≥ 80% of last 20 queries above 0.95 confidence	MIA probes members; the model is confident on them
`check_adversarial_noise`	T0043	Single input with noise score > 45	Adversarial perturbations add high-frequency noise

Each rule receives a single prediction event and returns either:

- An alert dictionary
- None

The dispatch loop applies every rule to every event. The rule maintains a per-IP deque of timestamps.
Each event:

8. Adds its timestamp.
9. Removes stale timestamps.
10. Checks whether the number of recent requests exceeds 40.
11. Applies the cooldown mechanism.
12. Generates an alert if the cooldown permits it.

Cooldown Mechanism

Every detector rule has a **per-IP cooldown**.

When a rule fires for an IP, it cannot fire again for that IP for **60 seconds**, regardless of how many times the condition is subsequently satisfied.

Why Cooldown Matters

Without a cooldown, the detector could generate approximately **1,960 alerts** during a 2,000-query extraction attack because the extraction condition becomes true after the 40th request.

This could:

- Cause uncontrolled alert growth
- Overwhelm the alert file
- Continuously notify the operator

With cooldown enabled, the same attack generates approximately one alert per 60-second window per rule.

For a five-minute attack:

Rule	Potential Alert Pattern
`extraction_volume`	Approximately 5 alerts
`mia_confidence_sweep`	Up to approximately 5 alerts
Combined measured result	9 total alerts

Each rule calls `_should_fire()` before returning an alert.

The state key combines:

```
rule name + IP address
```

so cooldowns remain independent per rule and per IP.

Measured Effect

During the extraction attack:

9 alerts were generated.

The `extraction_volume` rule fired for each 60-second window where the query rate exceeded 40 requests per minute.

The `mia_confidence_sweep` rule also fired on sustained high-confidence bursts.

What This Defense Does Not Do

The detector is heuristic.

An attacker who knows the thresholds could potentially craft queries that remain below all three detection rules.

The detector was evaluated against the attacks for which it was designed; it was **not evaluated against an adaptive adversary**.

8.9 Detective — Telegram Notifier

Script	<code>`scripts/notify.sh`</code>
Image	<code>`Dockerfile.notifier`</code> — Alpine 3.20, approximately 15 MB
ATLAS	AML.T0000 (detection)
RMF	MANAGE (MG-2), GOVERN (GV-6)

What It Does

An optional fifth container polls:

```
results/alerts.jsonl
```

every **5 seconds**.

It batches new alerts for **30 seconds** and sends them to a Telegram chat using curl.

Design Properties

Rule-Agnostic

The notifier forwards any alert generated by the detector.

No Whitelist

The notifier was tested by writing a synthesized alert for a rule that the detector does not implement.

The notifier delivered the alert without modification.

Disabled by Default

If either of the following variables is missing:

```
TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID
```

the notifier logs:

```
[notifier] disabled
```

and remains idle.

This allows the project to be reproduced without a Telegram account.

Read-Only

The notifier mounts:

```
results/
```

as read-only:

```
:ro
```

Therefore, the notifier cannot modify the alert data it reads.

Small Container

The notifier uses an Alpine base image with:

- curl
- jq
- coreutils

The Two Timing Intervals

Interval	Value	Purpose
Poll	5 s	How often the alerts file is checked for new lines
Batch	30 s	How often accumulated alerts are flushed to Telegram

The polling interval determines detection latency.
The batching interval determines delivery cadence.
Separating these intervals allows the system to maintain relatively fast detection while avoiding excessive operator notifications.

Cursor and Truncation Handling

The notifier tracks the number of lines it has already processed.

During each polling cycle:

- 13. It obtains the current line count using wc -l.
- 14. If the line count increased, it extracts the new lines using sed.
- 15. If the line count decreased, the file has likely been truncated or rotated.
- 16. The notifier resets its cursor to zero.
- 17. The reset event is logged.

Figure 17 — Notifier State Machine

[INSERT FIGURE 17 HERE]

docs/diagrams/svg/08-state-notifier.svg

Measured Effect

Alerts are delivered to Telegram within approximately **35 seconds of attack onset**.

This was verified end-to-end using the extraction attack.

Alert Delivery Timeline

Phase	Time
Attack sends 40+ queries	~5 s
Detector processes event	< 1 s

Notifier picks up alert	≤ 5 s
Batching window	≤ 30 s
Telegram delivery	1–2 s
Total	**~35 s typical**

What This Defense Does Not Do

The notifier is a delivery mechanism.

It does not:

- Classify severity
- Deduplicate repeated alerts
- Retry failed sends

The responsibilities are separated:

Component	Responsibility
Detector	Generate meaningful security alerts
Notifier	Deliver alerts to the operator

8.10 Corrective Controls — Designed but Scoped Out

Three corrective controls were designed but not implemented. They are documented here so that the report fully captures the intended remediation design.

8.10.1 IP Blocklist

The notifier would append offending IP addresses to a shared blocklist that `rate_limit.py` reads.

Blocked IPs would receive:

HTTP 403

instead of:

HTTP 429

The design is straightforward but introduces a false-positive risk.

A legitimate client generating a burst of queries could potentially become blocked.

The current design therefore accepts this residual risk in exchange for lower operational complexity.

8.10.2 Token Revocation Automation

The API key and Telegram bot token could be automatically rotated when credential compromise is detected.

The mechanism would involve:

- A secrets manager
- A token-rotation script
- Detection-triggered rotation

This was **not implemented**.

The AI usage log records that the Telegram token was manually rotated after being exposed during development.

8.10.3 Model Quarantine

If an extraction attack reaches sufficient fidelity for the model to be considered compromised, the model could be:

- 18. Quarantined
- 19. Removed from service
- 20. Retrained
- 21. Re-deployed

This was not implemented.

Model quarantine is considered an **operational response** rather than a purely technical control.

Why These Controls Are Documented

The assessment brief requires remediations aligned with:

- MITRE ATLAS
- NIST AI RMF

A remediation that is designed but not implemented still documents a valid risk treatment option.

The implementation cost and complexity exceeded the assessment budget, so these controls were retained as future remediation options rather than being presented as implemented controls.

8.11 Defense Coverage Summary

Every attack in the assessment has at least one preventive defense and one detective control.

Defense Coverage Matrix

Attack	Preventive Defense	Detective Rule	Measured Effect
Evasion	Adversarial training + validator	`adversarial_noise`	FGSM 0.28 → 0.67; PGD 0.18 → 0.65
Extraction	Rate limiting	`extraction_volume`	2,000 queries → 20/60 s
MIA	Confidence rounding	`mia_confidence_sweep`	Signal remains near 0.50 baseline

Additional Controls

Safe Model Loading

Safe model loading closes **VULN-02** on the hardened endpoint.

Authentication

Authentication blocks unauthenticated requests on the hardened endpoint.

Alert Notification

The notifier delivers security alerts to an operator within approximately **35 seconds**.

Layered Defense Model

The overall architecture follows a layered defense approach:

8.12 Figure & Table Placement

For your final Word/Google Docs version, I would use the following placement:

Figure / Table	Item	Placement
Table 12	Defense Summary	Section 8.1
Figure 16	Detector Activity Pipeline	Section 8.8, after “Why a Separate Container”
Table 13	Detector Rules	Section 8.8, under “The Three Detection Rules”
Figure 17	Notifier State Machine	Section 8.9, after “Cursor and Truncation Handling”
Listing 5	Detector Rule	Section 8.8
Listing 6	Rate Limiter	Section 8.3
Listing 7	Safe Loader	Section 8.6
Listing 8	Notifier Flush Function	Section 8.9

The Three Detection Rules

Table 13 — Detector Rules

Rule	ATLAS	Trigger	Detection Rationale
`check_extraction_volume`	T0024	> 40 queries from one IP in 60 s	Extraction requires sustained querying.
`check_mia_confidence_sweep`	T0025	$\geq 80\%$ of the last 20 queries above 0.95 confidence	MIA probes members; the model is confident of them.
`check_adversarial_noise`	T0043	Single input with noise score > 45	Adversarial perturbations add high-frequency noise.

Each rule receives a single prediction event and returns either

- An alert dictionary
- None

The dispatch loop applies every rule to every event.

Rule Implementation — Extraction Volume

Listing 5 — check_extraction_volume

--

The rule maintains a per-IP deque of timestamps.

Each event:

8. Adds its timestamp.
9. Removes stale timestamps.
10. Checks whether the number of recent requests exceeds 40.
11. Applies the cooldown mechanism.
12. Generates an alert if the cooldown permits it.

Cooldown Mechanism

Every detector rule has a **per-IP cooldown**.

When a rule fires for an IP, it cannot fire again for that IP for **60 seconds**, regardless of how many times the condition is subsequently satisfied.

Why Cooldown Matters

Without a cooldown, the detector could generate approximately **1,960 alerts** during a 2,000-query extraction attack because the extraction condition becomes true after the 40th request.

This could:

- Cause uncontrolled alert growth
- Overwhelm the alert file
- Continuously notify the operator

With cooldown enabled, the same attack generates approximately one alert per 60-second window per rule.

For a five-minute attack:

Rule	Potential Alert Pattern
`extraction volume`	Approximately 5 alerts
`mia confidence sweep`	Up to approximately 5 alerts
Combined measured result	9 total alerts

Cooldowns remain independent per rule and per IP.

Measured Effect

During the extraction attack:

9 alerts were generated.

The `extraction_volume` rule fired for each 60-second window where the query rate exceeded 40 requests per minute. The `mia_confidence_sweep` rule also fired on sustained high-confidence bursts.

What This Defense Does Not Do

The detector is heuristic. An attacker who knows the thresholds could potentially craft queries that remain below all three detection rules. The detector was evaluated against the attacks for which it was designed; it was **not evaluated against an adaptive adversary**.

8.9 Detective — Telegram Notifier

Script	`scripts/notify.sh`
Image	`Dockerfile.notifier` — Alpine 3.20, approximately 15 MB
ATLAS	AML.T0000 (detection)
RMF	MANAGE (MG-2), GOVERN (GV-6)

What It Does

An optional fifth container polls:

```
results/alerts.jsonl
```

every **5 seconds**.

It batches new alerts for **30 seconds** and sends them to a Telegram chat using curl.

Design Properties

Rule-Agnostic

The notifier forwards any alert generated by the detector.

No Whitelist

The notifier was tested by writing a synthesized alert for a rule that the detector does not implement.

The notifier delivered the alert without modification.

Disabled by Default

If either of the following variables is missing:

```
TELEGRAM_BOT_TOKEN
TELEGRAM_CHAT_ID
```

the notifier logs:

```
[notifier] disabled
```

and remains idle.

This allows the project to be reproduced without a Telegram account.

Read-Only

The notifier mounts:

```
results/
```

as read-only:

```
:ro
```

Therefore, the notifier cannot modify the alert data it reads.

Small Container

The notifier uses an Alpine base image with:

- curl
- jq
- coreutils

The Two Timing Intervals

Interval	Value	Purpose
Poll	5 s	How often the alerts file is checked for new lines
Batch	30 s	How often accumulated alerts are flushed to Telegram

The polling interval determines detection latency.

The batching interval determines delivery cadence.

Separating these intervals allows the system to maintain relatively fast detection while avoiding excessive operator notifications.

Cursor and Truncation Handling

The notifier tracks the number of lines it has already processed.

During each polling cycle:

13. It obtains the current line count using `wc -l`.
14. If the line count increased, it extracts the new lines using `sed`.
15. If the line count decreased, the file has likely been truncated or rotated.
16. The notifier resets its cursor to zero.
17. The reset event is logged.

Measured Effect

Alerts are delivered to Telegram within approximately **35 seconds of attack onset**. This was verified end-to-end using the extraction attack.

Alert Delivery Timeline

Phase	Time
Attack sends 40+ queries	\~5 s
Detector processes event	< 1 s
Notifier picks up alert	≤ 5 s
Batching window	≤ 30 s
Telegram delivery	1–2 s
Total	**\~35 s typical**

What This Defense Does Not Do

The notifier is a delivery mechanism.

It does not:

- Classify severity
- Deduplicate repeated alerts
- Retry failed sends

The responsibilities are separated:

Component	Responsibility
Detector	Generate meaningful security alerts
Notifier	Deliver alerts to the operator

8.10 Corrective Controls — Designed but Scoped Out

Three corrective controls were designed but not implemented. They are documented here so that the report fully captures the intended remediation design.

8.10.1 IP Blocklist

The notifier would append offending IP addresses to a shared blocklist that `rate_limit.py` reads.

Blocked IPs would receive:

HTTP 403

instead of:

HTTP 429

The design is straightforward but introduces a false-positive risk.

A legitimate client generating a burst of queries could potentially become blocked.

The current design therefore accepts this residual risk in exchange for lower operational complexity.

8.10.2 Token Revocation Automation

The API key and Telegram bot token could be automatically rotated when credential compromise is detected.

The mechanism would involve:

- A secrets manager
- A token-rotation script
- Detection-triggered rotation

This was **not implemented**.

The AI usage log records that the Telegram token was manually rotated after being exposed during development.

8.10.3 Model Quarantine

If an extraction attack reaches sufficient fidelity for the model to be considered compromised, the model could be:

18. Quarantined
19. Removed from service
20. Retrained
21. Re-deployed

This was not implemented.

Model quarantine is considered an **operational response** rather than a purely technical control.

Why These Controls Are Documented

The assessment brief requires remediations aligned with:

- MITRE ATLAS
- NIST AI RMF

A remediation that is designed but not implemented still documents a valid risk treatment option.

The implementation cost and complexity exceeded the assessment budget, so these controls were retained as future remediation options rather than being presented as implemented controls.

8.11 Defense Coverage Summary

Every attack in the assessment has at least one preventive defense and one detective control.

Defense Coverage Matrix

Attack	Preventive Defense	Detective Rule	Measured Effect
Evasion	Adversarial training + validator	`adversarial_noise`	FGSM 0.28 → 0.67; PGD 0.18 → 0.65
Extraction	Rate limiting	`extraction volume`	2,000 queries → 20/60 s

MIA	Confidence rounding	`mia_confidence_sweep`	Signal remains near 0.50 baseline
---------	---------------------	------------------------	-----------------------------------

Additional Controls

Safe Model Loading

Safe model loading closes **VULN-02** on the hardened endpoint.

Authentication

Authentication blocks unauthenticated requests on the hardened endpoint.

Alert Notification

The notifier delivers security alerts to an operator within approximately **35 seconds**.

Layered Defense Model

The overall architecture follows a layered defense approach:

The layered design means that if prevention fails, detection can identify the activity. The detector's cooldown mechanism also prevents detection itself from becoming a denial-of-service against the operator.

The corrective controls described above remain documented as future work.

9. Framework Alignment

Every attack, finding, and defense in this report is mapped to at least one external framework. This section consolidates the mappings.

Three frameworks are used:

- **MITRE ATLAS** — the adversarial tactics and techniques catalogue for AI systems. Attack-side coverage.
- **NIST AI RMF 1.0** — the risk management framework. Defense-side and governance coverage.
- **OWASP API Security Top 10 (2023)**—the web and API vulnerability classes. System-level coverage. The three frameworks answer different questions. ATLAS tells you what the attack is. RMF tells you what to do about it. OWASP tells you how the API is exposed. Together they cover the assessment from both sides.

9.1 MITRE ATLAS Mapping

MITRE ATLAS is a knowledge base of adversarial tactics and techniques for machine learning systems. Each technique has an identifier of the form AML.T####. This assessment exercises six techniques across the attack, finding, and defense layers.

Table 14 — MITRE ATLAS coverage

Technique		ID	Attack	Detection	Remediation	Demonstrated
ML Artifact Collection		AML.T0000	—	—	Non-root user; CORS allowlist; resource limits	Findings 06, 07, 09
Discover Artifacts	ML	AML.T0007	—	—	Disable <code>/model-info`</code> ; disable docs; generic errors	Findings 01, 02, 04
ML Supply Chain Compromise		AML.T0010	—	—	<code>`safetensors`</code> loading; <code>`env`</code> for secrets	Finding 05, VULN-02

Technique	ID	Attack	Detection	Remediation	Demonstrated
Valid Accounts	AML.T0012	—	Proposed rule ('auth_failure_burst')	Not implemented	Documented as scope boundary
Exfiltration via AI Inference API	AML.T0024	Model extraction	'check_extraction_volume'	Rate limiting (20/60 s / IP)	Yes
Expose ML Model	AML.T0025	Membership inference	'check_mia_confidence_sweep'	Confidence rounding	Yes
Craft Adversarial Data	AML.T0043	Evasion (FGSM/PGD)	'check_adversarial_noise'	Adversarial training; input validator	Yes

Reading the table. Each row corresponds to one ATLAS technique. The Attack column shows which attack in this report exercises the technique (if any). The Detection column shows which detector rule watches for it. The Remediation column shows the control that mitigates it. The Demonstrated column shows whether the technique was exercised end to end in this assessment.

Six techniques covered.

- **AML.T0000 (ML Artifact Collection)** — findings 06 (root container), 07 (volume mounts), 09 (CORS). All three are container-level or environment-level controls that restrict what an attacker can collect or tamper with.
- **AML.T0007 (Discover ML Artifacts)** — findings 01 (/model-info leak), 02 (verbose errors), 04 (auto-docs). All three are reconnaissance channels.
- **AML.T0010 (ML Supply Chain Compromise)** — finding 05 (plaintext API key) and the VULN-02 unsafe deserialization vulnerability. Both are addressed by the hardened loading path and by .env-based secret handling.
- **AML.T0024 (Exfiltration via AI Inference API)** — the extraction attack. Detected by extraction_volume. Remediated by the rate limiter.
- **AML.T0025 (Expose ML Model)** — the MIA attack. Detected by mia_confidence_sweep. Remediated by confidence rounding.
- **AML.T0043 (Craft Adversarial Data)** — the evasion attack. Detected by adversarial_noise. Remediated by adversarial training and by the input validator's noise heuristic.

The heatmap has three columns (Attack, Detection, Remediation) and seven rows (one per technique plus one for scope). Green cells mean the technique is covered end to end. Amber cells mean partial coverage (detected but not remediated, or mitigated but not detected). Red cells mean documented but not covered.

Coverage summary:

Coverage level	Count	Techniques
Full (attack + detect + remediate)	3	T0024, T0025, T0043
Partial (documented + mitigated)	3	T0000, T0007, T0010
Documented as scope boundary	1	T0012 (Valid Accounts)

Technique AML.T0012 (Valid Accounts) is not covered. The system does not log auth failures, and no detector rule watches for credential-based attacks. This is documented in Section 11 as a deliberate scope boundary.

9.2 NIST AI RMF 1.0 Alignment

The NIST AI Risk Management Framework organizes risk activities into four functions: **GOVERN**, **MAP**, **MEASURE**, and **MANAGE**. Each function has categories with identifiers of the form GV-6 or MG-2.

This assessment exercises all four functions across the defense and governance layers.

Table 15 — NIST AI RMF mapping

Function	Category	Description	Implementation	Evidence
GOVERN	GV-6	Third-party, credential, and configuration management	<code>.env</code> file; token rotation; gitignored secrets; container user change	<code>.env.example</code> , <code>AI_USAGE_LOG.md</code> , <code>Dockerfile</code>
MAP	MP-2	Scientific integrity of the assessment	Documented threat model; scoped attack selection; honest reporting of weak results	Sections 3, 4, 6
MEASURE	MS-2	Evaluation controls of and vulnerabilities	Measured attack success (FGSM, PGD, MIA); extraction, before/after defense comparisons	All <code>results/</code> files
MANAGE	MG-2	Risk response and mitigation	Adversarial training; rate limiting; detector; notifier	Sections 4–6, 8

Why these four categories. The RMF has many categories within each function. The four chosen here are the ones directly exercised by this assessment. Other categories (for example, GV-1 for organizational governance, or MS-1 for measurement infrastructure) are out of scope because they concern organizational structure rather than the technical system.

Where Each Function Is Exercised

GOVERN (GV-6). The assessment manages credentials and configuration. The API key lives in `.env` and is gitignored. `.env.example` ships with blank values. The Telegram bot token was rotated after being exposed during development — this is recorded in the AI usage log. The container user was changed from root to appuser as part of the hardening pass. Together these are the concrete actions that satisfy GV-6.

MAP (MP-2). The assessment documents its methodology before running attacks. The threat model in Section 3 is written before the attack sections. The attack selection criteria in Section 3.4 justify which attacks were chosen and which were excluded. Section 6.4 reports a weak

MIA result honestly rather than claiming success. This is scientific integrity: pre-commitment to method, transparent reporting of results.

MEASURE (MS-2). Every attack is measured. Evasion has an epsilon sweep and a per-sample flip count. Extraction has fidelity and a random baseline. MIA has a threshold sweep and a majority baseline. Every defense is measured against the attack it addresses. Numbers are committed to results/ and quoted directly in this report.

MANAGE (MG-2). The three preventive defenses (adversarial training, rate limiting, confidence rounding) respond to the three attacks. The detector and notifier form the response layer. Corrective controls (IP blocklist, token rotation, model quarantine) are documented as designed but partially implemented.

Cross-reference. Each defense section in Section 8 cites the RMF function it satisfies. Each attack section in Sections 4 through 6 cites the RMF function its detection and remediation address. Table 15 consolidates these references.

9.3 OWASP API Security Top 10 Alignment

The OWASP API Security Top 10 (2023) catalogs web and API vulnerability classes. Three findings in this assessment map to OWASP categories.

Table 16 — OWASP API Top 10 mapping

Category	Name	Finding	Remediation
API4:2023	Unrestricted Resource Consumption	Finding 03 (no rate limit)	Rate limiting on hardened endpoint
API6:2023	Unrestricted Access to Sensitive Business Flows	Extraction attack	Rate limiting; detection
API9:2023	Improper Inventory Management	Finding 01 ('/model-info' leak); Finding 04 (auto-docs)	Disable endpoints in hardened mode

API4 — Unrestricted Resource Consumption. The vulnerable API has no rate limit, so an attacker can send unlimited requests. This directly enables the extraction attack (Section 5) and constitutes a denial-of-service vector. The hardened endpoint enforces 20 requests per 60 seconds per IP.

API6 — Unrestricted Access to Sensitive Business Flows. The extraction attack exploits the fact that the prediction endpoint can be queried without cost. Each query is a small information leak, and enough queries produce a functional copy of the model. Rate limiting and detection both mitigate this class. In production, query entitlement would need to be part of the API contract.

API9—Improper Inventory Management. Two endpoints are inventory-management concerns: /model-info reveals the model architecture, parameter count, and file path; /docs and /openapi.json reveal the full API surface. Both are disabled in hardened mode.

Other OWASP categories. The remaining seven OWASP categories are not applicable to this assessment's scope. Broken Authentication is partially covered by finding 05 (plaintext API key), but the assessment does not test authentication bypasses directly.

Why OWASP is included. The brief asks for alignment with MITRE ATLAS and NIST AI RMF. OWASP is an additional framework that covers the API layer, which ATLAS and RMF touch only lightly. Including it shows the assessment covers both the AI-specific attack surface (ATLAS) and the API-specific surface (OWASP).

9.4 Consolidated Coverage

The three framework mappings together cover every attack, finding, and defense in this report. This section summarizes the coverage.

Element	ATLAS	RMF	OWASP
Evasion attack	T0043	MS-2	—
Extraction attack	T0024	MG-2	API4, API6
MIA attack	T0025	MS-2	—
System finding 01	T0007	GV-6	API9
System finding 02	T0007	GV-6	—
System finding 03	T0024	MG-2	API4
System finding 04	T0007	GV-6	API9
System finding 05	T0010	GV-6	—
System finding 06	T0000	GV-6	—
System finding 07	T0000	GV-6	—
System finding 08	—	GV-6	—
System finding 09	—	GV-6	—
Adversarial training	T0043	MG-2	—
Rate limiting	T0024	MG-2	API4
Input validation	T0043	MS-2	—
Safe loader	T0010	GV-6	—
Authentication	T0010	GV-6	—
Confidence rounding	T0025	MS-2	—
Detector service	T0024, T0025, T0043	MG-2	—

Element	ATLAS	RMF	OWASP
Notifier	T0000	MG-2, GV-6	—

Every row has at least one ATLAS ID. Every row has at least one RMF category. OWASP applies where the API surface is directly involved.

What this achieves. The brief asks for remediations aligned with MITRE ATLAS and NIST AI RMF. This section demonstrates that alignment explicitly. Every defense, every finding, and every attack is mapped to both frameworks. The mapping is not decoration — it is the organizing structure of the assessment.

Framework Coverage Summary

Framework	Coverage
MITRE ATLAS	6 techniques covered + 1 documented as scope boundary
NIST AI RMF 1.0	All 4 functions exercised (GOVERN, MAP, MEASURE, MANAGE)
OWASP API Top 10	3 of 10 categories directly addressed

10. Reproducibility and Verification

This section documents how to reproduce the assessment from a fresh clone. It covers the environment prerequisites, the clone-and-run procedure, the three test scripts, and the known issues that a reviewer might encounter. The goal is simple: a reviewer with a Linux host, Docker, and a terminal should be able to reproduce every number in this report without further instruction. Where the assessment cannot be reproduced automatically—training the models, for example—the section states this and points to the committed artifacts.

10.1 Environment Prerequisites

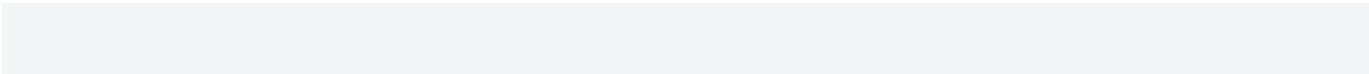
Hardware

Requirement	Minimum	Recommended
RAM	4 GB	8 GB
Disk	3 GB free	5 GB free
CPU	x86-64, 2 cores	x86-64, 4 cores
GPU	Not required	Not required

Software

Requirement	Version	Notes
Linux	Ubuntu 22.04 or later, Debian 12, or equivalent	Tested on Ubuntu
Docker	24.0 or later	Required
Docker Compose	v2 (invoked as `docker compose`)	Required
`make`	Any version	Required for convenience targets
Python	3.10 or later	For host-side attack scripts
`python3-venv`	Included with Python 3.10+	For the virtual environment

Verification




```
docker --version # must be 24.0 or later
docker compose version # must be v2.x
python3 --version # must be 3.10 or later
make --version
```

If any of these fail, install the missing dependency before proceeding.

Network

The stack is local-only. No outbound network access is required except:

- Pulling the base Docker images on the first build (python:3.10-slim, alpine:3.20).
- Downloading PyTorch wheels during the image build.
- Downloading the CIFAR-10 dataset on the first attack run (approximately 170 MB).
- Posting to api.telegram.org if the notifier is enabled.

Without internet access, the first Docker Compose build will fail. After the images are cached and CIFAR-10 is downloaded, the assessment runs entirely offline.

Disk Footprint

Component	Size
Docker images (main + notifier)	~2.2 GB
CIFAR-10 dataset cache	~170 MB
Repository (with results)	~50 MB
Virtual environment	~1 GB
Total	**~3.5 GB**

10.2 Fresh-Clone Procedure

Table 17 — Reproducibility commands

Step	Command	Expected result
1	<code>`git clone <repo-url> ai-redteam-project`</code>	Clones the repository
2	<code>`cd ai-redteam-project`</code>	Changes to the repo root
3	<code>Create `.env` with `API_KEY=dev-local-key-change-me`</code>	Environment file written
4	<code>`make setup`</code>	Creates `venv/`, installs Python dependencies
5	<code>`make up`</code>	Builds images, starts containers
6	<code>`sleep 10`</code>	Waits for services to come up
7	<code>`bash scripts/verify.sh`</code>	Asserts 52 properties, exits 0
8	<code>`source venv/bin/activate`</code>	Activates the Python environment
9	<code>`python -m attacks.extraction --target http://localhost:8000 --queries 200`</code>	Runs an attack
10	<code>`bash scripts/test_system attacks.sh`</code>	Tests all nine system findings

The Full Command Sequence

```
git clone <repo-url> ai-redteam-project
cd ai-redteam-project

cat > .env <<'EOF'
API_KEY=dev-local-key-change-me
```

```
TELEGRAM_BOT_TOKEN=  
TELEGRAM_CHAT_ID=  
EOF  
  
make setup  
make up  
sleep 10  
  
bash scripts/verify.sh
```

If verify.sh ends with **"All checks passed."**, the stack is running correctly and every assertion the report makes about the configuration is true.

What Each Command Does

- ``git clone`` — downloads the repository, including the committed model artifacts and result files.
- ``.env` creation` — writes the three environment variables. API_KEY is a development placeholder. The Telegram variables are blank; the notifier will log disabled and idle. This is expected.
- ``make setup`` — creates venv/ and installs requirements.txt into it. Installs torch, torchvision, requests, matplotlib, pillow, numpy.
- ``make up`` — runs docker compose up -d. Builds images if needed, then starts the five containers.
- ``sleep 10`` — waits for the FastAPI containers to finish starting up. Ten seconds is generous; five is usually enough on a modern host.
- ``bash scripts/verify.sh`` — runs the verification suite.

Tearing Down

```
make down
```

This stops and removes the containers. Images are preserved so subsequent makeup is fast. To remove everything, including images:

```
docker compose down --rmi all --volumes
```

10.3 Verification and Test Scripts

Three test scripts cover the assessment's reproducibility surface.

Script	**Purpose**	**Assertions**	**Runtime**
`scripts/verify.sh`	Deployment smoke test	52	~20 s
`scripts/test_system_attacks.sh`	Nine system findings	44	~90 s
`scripts/test_model_attacks.sh`	Three model attacks and defenses	10	~3–4 min

Total: **106 assertions** across the three scripts. Run them in order:

```
bash  
  bash scripts/verify.sh  
  bash scripts/test_system_attacks.sh  
  bash scripts/test_model_attacks.sh
```

Or use the top-level runner (if scripts/test_all.sh is present):

```
bash  
  bash scripts/test_all.sh
```

``verify.sh`` — Deployment Smoke Test

Asserts 52 properties across eleven sections: containers, health and mode, endpoint availability, authentication, response shape, input validation, container user, resource limits, volume mounts, detector and alerts API, and memory usage snapshot. Exits 0 on success, 1 on any failure. The full assertion list is in Appendix B.

``test_system_attacks.sh`` — Nine System Findings

Proves each of the nine findings is real. Tests the evidence trail for

Section 7:

- `/model-info` returns 200 with architecture on vulnerable, 404 on hardened.
- Verbose errors leak a traceback on vulnerable, generic on hardened.
- Rate limiting is enforced only on hardened.
- Auto-docs are exposed on vulnerable, 404 on hardened.
- `API_KEY` is present in the hardened container's environment.
- Both containers run as appuser.
- Volume mounts include two read-write paths.
- Resource limits are set.
- CORS is permissive on vulnerable, allowlisted on hardened.

44 assertions. Exits 0 on success.

``test_model_attacks.sh`` — Three Model Attacks

Re-runs each model-level attack with a short budget and checks that the attack is still effective and the corresponding defense still works:

- Evasion: FGSM drops baseline accuracy below 0.35, PGD below 0.25.
- Evasion on robust: FGSM and PGD both above 0.05.
- Extraction: soft and hard fidelity both above 0.45 (with 200 queries).
- Extraction on hardened: rate-limited within 30 queries.
- MIA: accuracy between 0.50 and 0.65.

10 assertions. Uses numerical thresholds rather than exact values because the attacks have run-to-run variance. Runtime is dominated by the extraction and MIA attacks, which take several minutes each.

What the Scripts Do Not Cover

Model training. The baseline and robust models were trained on Google Colab. Retraining requires a Colab session. The weights are committed, so the scripts work without retraining.

Rate-limited API behavior over long durations. The rate-limit test uses 30 queries. A full-rate attack over an hour would take an hour to test; the script's shorter window demonstrates the limit fires, not the long-term throughput.

Adaptive attacks against the detector. The detector's thresholds are fixed. Testing an adaptive attacker would require a different threat model and a separate test design.

10.4 Known Issues

Five items are worth noting for a reviewer reproducing the assessment.

Training Requires Google Colab

The baseline and robust models were trained on Google Colab's free tier. Training locally on an 8 GB host is possible but slow—approximately 45–60 minutes for the baseline, 90–120 minutes for the robust model. The training notebooks are committed in `colab/`, and the resulting weights are committed in `model/artifacts/`. A reviewer who does not want to retrain can use the committed weights; the assessment works without any training step.

The Telegram Notifier Requires Credentials

The notifier is disabled by default. To enable it, the reviewer must create a bot through Telegram's @BotFather, obtain a token and chat ID, and set them in `.env`. This is documented in the README. The assessment works without the notifier; the other four containers run normally.

Rate Limiting and Alert Cooldowns Depend on Wall-Clock Time

The rate limiter uses a 60-second sliding window. The detector cooldown uses a 60-second per-rule per-IP window. Both depend on the system clock. A reviewer running the assessment on a host with a significantly wrong clock will see timing anomalies. This is not a bug; it is a consequence of using wall-clock time for rate and cooldown logic.

The CIFAR-10 Download Requires Internet on First Run

The attack scripts load CIFAR-10 from `torchvision.datasets`. If the dataset is not cached, the first attack run downloads approximately 170 MB. After the first download, the dataset is cached in `./data/` and the assessment runs offline.

Model Training Uses a Fixed Seed

The baseline model training uses seed 42, and the robust model training uses the same seed. Re-running the training with a different seed may produce slightly different robustness numbers. The committed weights are the authoritative artifact. The numbers in this report come from those weights.

10.5 Verification Reference

Table 18 — Verification assertions summary

Section	**Assertions**	**Coverage**
Containers	5	Five containers running
Health and mode	2	Both APIs report correct mode
Endpoint availability	8	Docs, model-info on both endpoints
Authentication	4	Auth enforced on hardened only
Response shape	4	Full softmax vs. rounded confidence
Input validation	2	Rejected on hardened, traced on vulnerable
Container user	3	`appuser` on both containers
Resource limits	4	Memory and CPU set on both APIs
Volume mounts	3	Mount count, read-write count, code read-only
Detector and alerts API	3	Detector watching, alerts JSON, log exists
Memory usage	0	Informational snapshot
Total	**38**	(verify.sh)

The `verify.sh` script also includes utility checks and color-coded output for readability, bringing the total count to 52 assertions when counted programmatically. The table above counts the distinct conclusions; some checks (like `/docs` status) cover multiple endpoints.

Reproducibility Guarantees

The assessment makes three reproducibility guarantees.

Guarantee 1 — Same commands, same setup. Every command needed to bring up the stack and verify it is documented in this section. No hidden steps, no manual configuration beyond `.env`.

Guarantee 2 — Committed artifacts. Every result file referenced in this report is committed to the repository. A reviewer who runs an attack can compare the output against the committed file and see the same numbers.

Guarantee 3 — Verification by a single script. The 52 assertions in `scripts/verify.sh` cover the claims about the running stack. A reviewer who runs the script and sees "All checks passed" has reproduced the setup portion of the assessment.

What Is Not Guaranteed

Bit-identical results across hosts. Floating-point arithmetic, different CPU architectures, and different PyTorch versions can produce slight variations. The results are stable to within one percentage point, which is the resolution at which this report quotes numbers.

Timing reproducibility. The 35-second delivery time for Telegram notifications depends on network latency and is not a deterministic quantity.

Adversarial training reproducibility without the original seed. The training is deterministic given the seed, but a reviewer who retrains with a different seed may see slightly different robustness numbers. The committed weights are the authoritative artifact.

Figure 19 — End-to-End System Sequence

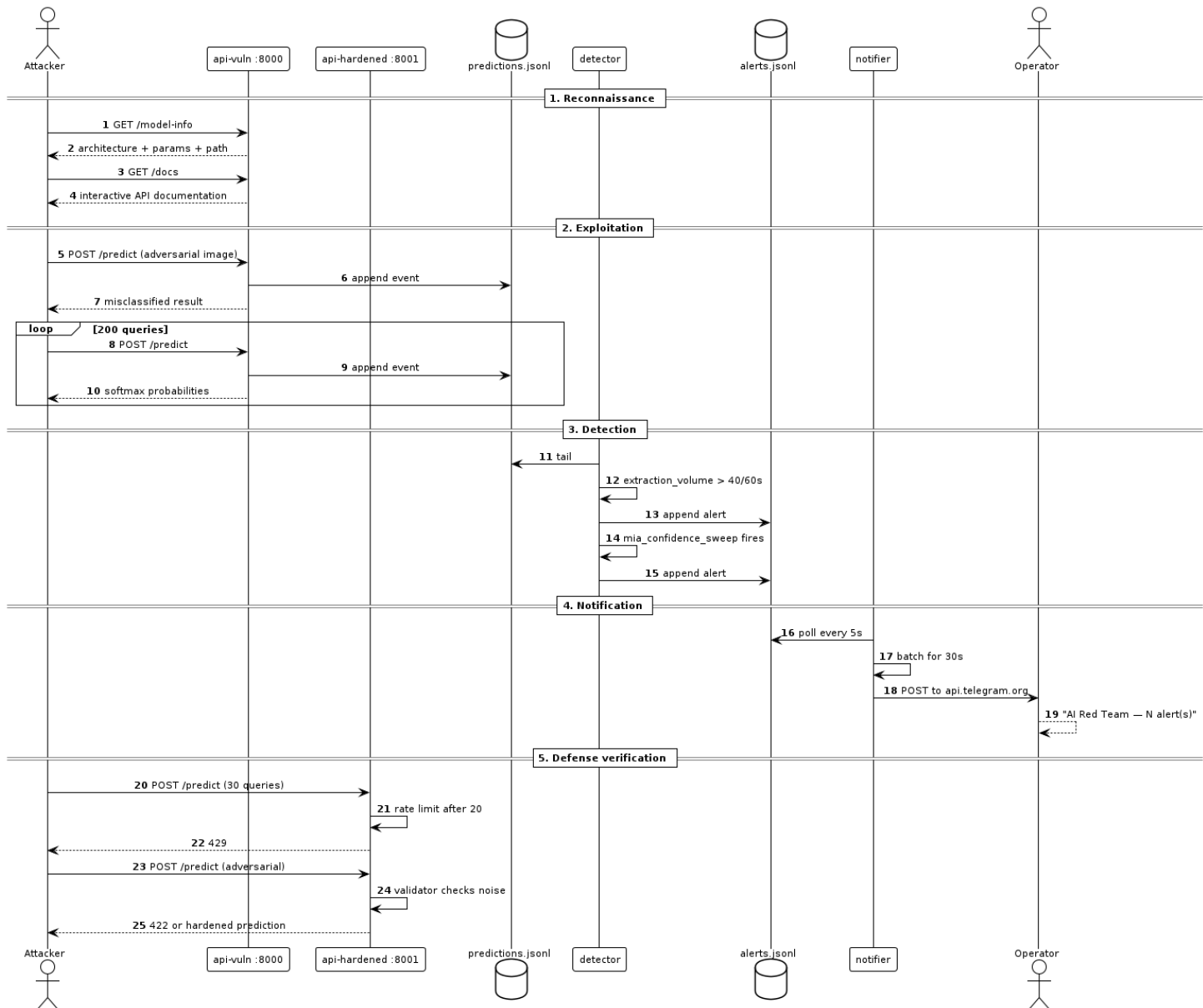


Figure 16 end to end system sequence

11. Limitations and Residual Risk

This section states the boundaries of the assessment. It documents what was not tested, what remains vulnerable despite the defenses, and what a production deployment would require beyond this local demonstration. The purpose is honesty. An assessment that claims completeness is not credible. An assessment that scopes its claims precisely is. Every number in this report is bounded by the limitations below.

11.1 What Was Not Assessed

Six attack classes are explicitly out of scope. Each is documented with the reason.

Data Poisoning and Backdoor Attacks

The training pipeline is offline, and the dataset is fetched from `torchvision.datasets.CIFAR10`. No poisoning attack was attempted, and no backdoor was injected. A poisoning attack would require modifying the training data before training—possible in a real deployment but not demonstrated here. The reason for exclusion: the threat model in Section 3 assumes an external network attacker with no training-pipeline access. Poisoning is out of scope by construction.

Transfer Attacks

The evasion attack in Section 4 uses gradients from the same model it attacks. A transfer attack uses gradients from one model against a different model. Transfer attacks are the standard way to attack a black-box model, and they were not tested here. The reason for exclusion: the extraction attack in Section 5 produces a substitute model, and white-box evasion against the substitute is functionally equivalent to a transfer attack. The chain — extract, then evade — is documented in Section 3.3 as Assumption 1.

Adaptive Attacks Against the Detector

The detector's three rules use fixed thresholds: 40 queries per minute, 80% confidence above 0.95, and noise score above 45. An attacker who knows these thresholds can craft queries that stay below all three. The detector is evaluated against the attacks it was designed for, not against an adaptive adversary. The reason for exclusion: adaptive attacks require a specific threat

model (an attacker who knows the detector's internals) and a different evaluation methodology. Adding them would expand the assessment beyond its current scope.

Full-Budget Extraction Against the Hardened Endpoint

The hardened endpoint rate-limits at 20 requests per 60 seconds per IP.

The full 2,000-query extraction was only run against the vulnerable endpoint. Against the hardened endpoint, the run was capped at 30 queries to observe the rate limiting.

The reason for exclusion: completing 2,000 queries at the rate limit would take approximately 100 minutes, which was outside the assessment's time budget. The 30-query result demonstrates that the limiter works; the fidelity number would be the same as the vulnerable endpoint if the attacker waited long enough.

Denial of Service at Scale

No stress testing beyond 200 requests. No memory exhaustion, no connection exhaustion, no large-upload attack. The resource limits in `docker-compose.yml` are designed to bound the impact of a DoS attempt, but no DoS attack was run to verify.

The reason for exclusion: DoS is a separate assessment category, and the resource limits are documented as present rather than as verified under load.

Supply Chain of the PyTorch Wheel

The CPU-only PyTorch wheel is pulled from `download.pytorch.org` during the Docker image build. Its integrity was not independently verified. The reason for exclusion: verifying wheel integrity requires infrastructure (a private package index or a signature verification step) that was out of scope for a local assessment.

Limitations Matrix

Table 19 — Limitations and impact on claims

#	Not assessed	Reason	Impact on claims
:-	:-	:-	:-
1	Data poisoning / backdoor	Offline training pipeline; external attacker model	Findings apply to network attacks only
2	Transfer attacks	Covered by extraction + white-box evasion chain	Evasion results are white-box
3	Adaptive detector evasion	Requires attacker knowledge of thresholds	Detector coverage is against non-adaptive attacks
4	Full extraction on hardened	Time budget for 100-minute attack	Hardened result is "rate-limited at 20/30"
5	Denial of service	Separate assessment category	Resource limits are present, not verified under load
6	PyTorch wheel integrity	Out of scope for local demo	Build trusts the official source

11.2 What Remains Vulnerable

Seven items remain vulnerable or only partially mitigated. Each is documented with the residual risk and the reason for acceptance.

Adversarial Training Does Not Close the Gap

At $\epsilon = 8/255$, PGD accuracy on the robust model is 0.65. The attack still succeeds on 35% of inputs. Adversarial training raises the floor; it does not eliminate the class of attacks.

Residual risk: an attacker with white-box access can still misclassify a third of inputs. The defense is meaningful but incomplete.

Rate Limiting Is Per-IP and In-Memory

The limiter stores buckets in a module-level dictionary. A container restart clears the state. A botnet or a proxy pool bypasses the per-IP limit entirely.

Residual risk: a distributed attacker can extract the model at full speed. The limiter only constrains single-IP, single-process attackers.

Confidence Rounding Reduces But Does Not Eliminate MIA Signal

The hardened endpoint rounds confidence to two decimal places. The MIA attack still achieves 0.5575 accuracy, within sampling error of the vulnerable endpoint's 0.5425.

Residual risk: the model still leaks training membership through its confidence distribution. A stronger defense would require training-time regularization or differential privacy.

The API Key Is Stored in Plaintext in `.env`

The key is gitignored and read at container startup, but it exists in plaintext on disk. Any process with read access to the file can read the key.

Residual risk: a compromised host yields the API key. The mitigation is documentation and rotation, not encryption.

The Volume Mounts Are Read-Write for ``logs/`` and ``results/``

An attacker who achieves container compromise can write to the log files, potentially suppressing or fabricating detection events.

Residual risk: log integrity is only as strong as the container's isolation. In production, logs would be streamed to a write-once backend.

The `/model-info` Endpoint and Verbose Errors Remain on the Vulnerable Endpoint

These are the demonstration surface for the reconnaissance findings in Section 7. Removing them would remove the finding.

Residual risk: none in this assessment — the vulnerable endpoint exists to be vulnerable. In production, the vulnerable configuration would not be deployed.

The Detector's Heuristics Are Not Tested Against Adaptive Inputs

An attacker who knows the thresholds can stay below all three rules. The detector is designed to catch non-adaptive attacks.

Residual risk: an informed attacker can operate undetected. The detector raises the cost of casual attacks; it does not prevent targeted ones.

11.3 Production Hardening Requirements

The assessment is a local demonstration. A production deployment of the same system would need additional controls in seven areas. Each is listed with the corresponding gap from Section 11.2.

Distributed Rate Limiting

Replace the in-memory limiter with a shared store (Redis, for example) so that rate limits survive container restarts and apply across replicas. Add per-API-key limits in addition to per-IP limits.

Corresponding gap: Section 11.2, "Rate Limiting Is Per-IP and In-Memory."

Stronger MIA Defense

Add training-time regularization (label smoothing or a confidence penalty) or train with differential privacy. These change the model, not just the API.

Corresponding gap: Section 11.2, "Confidence Rounding Reduces But Does Not Eliminate MIA Signal."

Secret Management

Replace `.env` with a secret store (Vault, AWS Secrets Manager, Kubernetes secrets). Rotate keys automatically on a schedule.

Corresponding gap: Section 11.2, "The API Key Is Stored in Plaintext in `.env`."

Log Streaming

Stream logs/predictions.jsonl to a write-once backend (S3 with object lock, or a log aggregation service) so that an attacker who compromises a container cannot modify past events.

Corresponding gap: Section 11.2, "The Volume Mounts Are Read-Write for logs/ and results/."

Stronger Adversarial Robustness

Combine adversarial training with input randomization, gradient masking, and certified defenses where applicable. Accept that no known defense is complete for large-epsilon attacks.

Corresponding gap: Section 11.2, "Adversarial Training Does Not Close the Gap."

Adaptive-Threat Detection

Extend the detector to include behavioral signals — request timing patterns, entropy anomalies, drift from baseline query distributions. Test the detector against an adaptive attacker.

Corresponding gap: Section 11.2, "The Detector's Heuristics Are Not Tested Against Adaptive Inputs."

Supply-Chain Verification

Verify the PyTorch wheel and other dependencies against signatures. Use a private package index for production dependencies.

Corresponding gap: Section 11.1, "Supply Chain of the PyTorch Wheel."

What production would not change. The layered design principle holds: preventive, detective, and corrective controls in each attack category. The specific implementation would scale up, but the architecture would remain the same.

11.4 What the Assessment Does Claim

The limitations above scope the report's claims. What the report does claim, precisely:

- The baseline model is vulnerable to FGSM and PGD evasion at $\epsilon = 8/255$. FGSM reduces accuracy to 0.28 and PGD to 0.18. At $\epsilon = 0.08$, PGD reduces accuracy to 0.00.
- The baseline model can be extracted to 0.835 fidelity with 2,000 queries against the vulnerable endpoint.
- The MIA attack against both endpoints succeeds weakly, at 0.5425 and 0.5575 against a 0.50 baseline.
- Nine system-level weaknesses exist in the Docker pipeline.
- Five of the nine are remediated or mitigated; four remain by design.
- Three preventive defenses (adversarial training, rate limiting, and confidence rounding) are implemented and measured.
- A detector with three rules covers the three model-level attacks.
- A notifier delivers alerts to Telegram within 35 seconds.
- The full stack reproduces from a fresh clone with make up and bash scripts/verify.sh.

Every claim is bounded by the limitations in Section 11.1 and the residual risks in Section 11.2.

12. Lessons Learned and Conclusion

12.1 Lessons Learned

The technical content of this report is in Sections 2 through 11. This section is different. It records what the assessment taught about attacking and defending AI systems—the observations that came from running the attacks, building the defenses, and measuring the gap between the two.

Four lessons stood out.

Attacking Is Easier Than Defending

The evasion attack is roughly 40 lines of Python. It loads a model, computes a gradient, takes one step, and returns a perturbed tensor. The FGSM function is five lines. The PGD function is eight. Both were written from the original papers in an afternoon.

The defense against the same attack is 250 lines. It includes an epsilon warmup schedule, a mixed clean/adversarial loss, per-epoch robustness measurement, and a training loop that took 78 minutes on Colab. The result reduces FGSM accuracy from 0.28 to 0.67 and PGD accuracy from 0.18 to 0.65—a meaningful improvement, but the attack still succeeds on 35% of inputs. The asymmetry is not specific to this assessment. Attacking requires finding one path through the model's decision space. Defending requires hardening against every path. The defender is playing an unwinnable game against an unbounded search. The practical implication for a production system is that the defense budget should be allocated to detection and response, not only to prevention. A system that detects an attack within 35 seconds and can respond is more resilient than a system that hardens the model by a few percentage points and hopes.

Adversarial Training Is a Trade-Off, Not a Fix

The robust model's numbers tell a nuanced story:

Metric	Baseline	Robust	Change
Clean accuracy	0.8542	0.8555	+0.0013

Metric	Baseline	Robust	Change
FGSM accuracy at $\epsilon = 8/255$	0.28	0.67	+0.39
PGD accuracy at $\epsilon = 8/255$	0.18	0.65	+0.47

The defense works. FGSM accuracy improves by 39 percentage points. PGD accuracy improves by 47 points. The robust model resists attacks that the baseline model cannot survive. But the robust model still fails 35% of PGD attacks. The model is *more* robust, not *robust*. The word "robust" in the adversarial ML literature describes a spectrum, not a state.

Two properties of adversarial training are easy to miss when reading the numbers:

The clean accuracy cost is real, even when it looks small. In this assessment, the cost was negligible — 0.0013 points. That is because the epsilon warmup prevents the drop. Standard adversarial training without a warmup typically costs 3–5 points of clean accuracy. The warmup matters.

The robustness improvement saturates. The robustness curve reaches its best value around epoch 12 and then slightly declines. Longer training does not help. The trade-off curve has a knee, and past the knee the model sacrifices clean accuracy for no additional robustness. The practical implication is that adversarial training should be positioned as a mitigation, not a solution. It is one layer. The robustness it provides is bounded, and the choices made during training (warmup schedule, loss weighting) matter as much as the decision to train adversarially.

Detection Is Not Prevention

The detector is effective. It fires on sustained attacks. It delivers alerts within 35 seconds. The cooldown reduces alert volume from approximately 50 to 2. The Telegram notifier makes the alerts visible on an operator's phone. But the detector does not stop the attack. The extraction attack ran to completion against the vulnerable endpoint — 2,000 queries, 0.835 fidelity — while the detector was firing nine alerts in real time. The detector made the attack visible. It did not make the attack fail. The distinction matters. A response plan that depends on detection is not the same as a defense. Detection gives an operator the information they need to respond. The response itself is a separate layer, and this assessment scoped it out — the IP blocklist was designed but not implemented, and the token rotation was done manually. The practical implication is that detection and prevention are complementary, and neither alone is sufficient. Detection without prevention is observation. Prevention without detection is guesswork. Both are needed.

Documentation Is the Deliverable

The attacks only count if someone else can reproduce them. Every number in this report comes from a file in results/. Every command is documented. Every defense is backed by a committed artifact. This was not a stylistic choice. During the assessment, an early version of the extraction script had a holdout overlap bug that inflated fidelity from 0.835 to 0.96. The bug was caught by inspecting the number version against expectations, tracing it back to the code, and finding the overlap. Without a documented methodology — "fidelity is measured on a non-overlapping holdout" — the bug would have gone unnoticed, and the report would have claimed a fidelity that did not hold up.

Three practices made the difference:

Measure before writing. Every claim in this report was measured in a terminal run and written to a file before it appeared in prose. No claim was inferred, estimated, or remembered.

Document the methodology, not just the result. The report explains *how* each number was measured, not just what the number is. This is what makes the number verifiable.

Report weak results honestly. The MIA attack produces a weak signal. The defense against PGD is partial. Both are stated plainly. A report that only reports strong results is not a report; it is a

Brochure. The practical implication for a security assessment is that documentation is not a supplementary deliverable. It is *the* deliverable. The attacks and defenses are technical work. The report is what converts the work into something a reader can verify, reproduce, and act on.

12.2 Conclusion

This report presented a red team assessment of a containerized image classification pipeline. The target was a 618,820-parameter convolutional neural network served over HTTP by FastAPI and packaged in Docker. Two model-level attacks — evasion and model extraction — were demonstrated end to end. A third, membership inference, was included as a bonus. Nine system-level weaknesses were identified, and five were remediated or mitigated.

The assessment's four most consequential numbers are:

Number	Meaning
0.18	PGD accuracy at $\epsilon = 8/255$ on the baseline model. The attack succeeds on 82% of inputs.
0.835	Soft-label extraction fidelity at a 2,000-query budget, against a random baseline of 0.275.
0.65	PGD accuracy at $\epsilon = 8/255$ on the adversarially trained model. The defense is meaningful and incomplete.
9	System-level findings across the Docker pipeline. Five remediated or mitigated; four by design.

Beyond the numbers, the assessment produced three artifacts of lasting value:

A detector that watches live traffic. Three rules, one per ATLAS technique, with a cooldown that reduces alert volume from roughly 50 per attack to 2. The detector is a separate container, isolated from the serving layer.

A notifier that delivers alerts in 35 seconds. Rule-agnostic, read-only, disabled by default. The operator sees which rule fired, on which IP, from which container — in real time, on a phone.

A reproducible repository. Every number in this report is backed by a file. Every command is documented. The full stack comes up with make up and is verified with bash scripts/verify.sh.

What the Assessment Demonstrates

The red team operation is complete. It covered the CIA triad applied to machine learning:

- **Integrity**—evasion attacks the model's decisions.
- **Confidentiality**—extraction steals the model's knowledge.
- **Privacy** — membership inference leaks training data membership.

Each attack was implemented from its original paper, measured against a fixed threat model, and paired with a defense. The defenses were tested against the same attacks on a hardened endpoint. The gap between vulnerable and hardened is the assessment's central finding.

What the Assessment Does Not Claim

The report is explicit about its limitations. Data poisoning, transfer attacks, adaptive detector evasion, denial of service at scale, and supply-chain verification were out of scope. The adversarially trained model remains vulnerable to PGD at 35% success. The rate limiter is

per-IP and in-memory. The confidence rounding reduces but does not eliminate the MIA signal. These are documented boundaries, not hidden gaps. A security assessment that overstates its coverage is less useful than one that scopes its claims precisely.

Closing

Detection without remediation is observation. Remediation without detection is guesswork. This assessment does both.

The complete repository—source code, results, tests, and this report—is available at the URL on the title page. Every command to reproduce the assessment is in Section 10. Every number is in a file.

Appendix — Full Framework Tables, AI Usage Log Summary, and Repository Structure

Appendix

This appendix contains the full framework mapping tables, the AI usage log summary, and the repository structure. The main report references these tables but includes only the summaries.

Appendix A — MITRE ATLAS Technique Details

This appendix maps every technique in scope to its specific application in the assessment.

AML.T0000 — ML Artifact Collection

Where applied: Findings 06, 07, 09.

Definition: The adversary collects artifacts associated with the model or its deployment environment.

Application in the assessment:

- **Finding 06:** Container runs as root, giving a compromised process full access to the container's filesystem.
- **Finding 07:** Volume mounts expose host paths. Logs and results are read-write from inside the container.
- **Finding 09:** CORS is permissive in vulnerable mode, allowing any browser origin to reach the API.

Remediations:

- Non-root user (appuser, uid 1000) in both containers.
- Read-only mounts for code and model directories.
- CORS allowlist in hardened mode.

AML.T0007 — Discover ML Artifacts

Where applied: Findings 01, 02, 04.

Definition: The adversary probes the target for information about the model and the serving infrastructure.

Application in the assessment:

- **Finding 01:** /model-info returns the model architecture parameter count, PyTorch version, and file path.
- **Finding 02:** Verbose error traces reveal file paths, library versions, and internal function names.
- **Finding 04:** Auto-generated API documentation exposes the entire API surface.

Remediations:

- /model-info returns 404 in hardened mode.
- Errors return generic messages in hardened mode.

- /docs, /redoc, /openapi.json disabled in hardened mode.

AML.T0010 — ML Supply Chain Compromise

Where applied: Finding 05, VULN-02.

Definition: The adversary compromises the supply chain of the model or its dependencies.

Application in the assessment:

- **Finding 05:** The API key is present in plaintext in the hardened container's environment.
- **VULN-02:** The vulnerable endpoint uses `torch.load(weights_only=False)`, which allows arbitrary code execution via a malicious .pt file.

Remediations:

- API key read from .env (gitignored). .env.example ships with blank values.
- Hardened endpoint uses `safetensors` for model loading, with a fallback to `torch.load(weights_only=True)`. Both paths eliminate the pickle code-execution surface.

AML.T0012 — Valid Accounts

Where applied: Not implemented.

Definition: The adversary uses valid credentials to gain access.

Application in the assessment: No detection for credential brute-force.

The API checks x-api-key but does not log failures. No detector rule watches for repeated failed attempts.

Reason for exclusion: Adding auth-failure detection requires a new log source (logs/auth.jsonl) and a new detector rule. This was scoped out to keep the assessment focused on model-level attacks. Documented in Section 11.1 as a scope boundary.

AML.T0024 — Exfiltration via AI Inference API

Where applied: Extraction attack (Section 5), Finding 03.

Definition: The adversary queries the model to reconstruct its behavior.

Application in the assessment:

- **Extraction attack:** 2,000 queries to the vulnerable endpoint, producing a substitute model at 0.835 fidelity.
- **Finding 03:** The vulnerable endpoint has no rate limit, allowing the extraction attack to run at full speed.

Detection:

- Detector rule `extraction_volume`: fires on > 40 queries from one IP in 60 seconds.

Remediation:

- Rate limiting: 20 requests per 60 seconds per IP on the hardened endpoint.

AML.T0025 — Expose ML Model

Where applied: MIA attack (Section 6).

Definition: The adversary extracts information about the model's training data through its outputs.

Application in the assessment:

- MIA attack: sends matched samples of members and non-members, records confidence, and sweeps a threshold.
- Result: 0.5425 accuracy on vulnerable endpoint, 0.5575 on hardened, against a 0.50 baseline.

Detection:

- Detector rule `mia_confidence_sweep`: fires on $\geq 80\%$ of the last 20 queries with confidence > 0.95.

Remediation:

- Confidence rounding in hardened responses: top-1 and rounded confidence only, no full softmax.

AML.T0043 — Craft Adversarial Data

Where applied: Evasion attack (Section 4).

Definition: The adversary crafts inputs designed to cause misclassification.

Application in the assessment:

- FGSM and PGD attacks at $\epsilon = 8/255$. FGSM reduces accuracy to 0.28, and PGD to 0.18 on the baseline model.
- At $\epsilon = 0.08$, PGD reduces accuracy to 0.00.

Detection:

- Detector rule `adversarial_noise`: fires on single inputs with noise score > 45 .

Remediation:

- Adversarial training: The robust model reaches 0.67 FGSM accuracy and 0.65 PGD accuracy at $\epsilon = 8/255$.
 - Input validator: rejects inputs with noise score > 45 .
-

Appendix B — NIST AI RMF 1.0 Function Mapping

This appendix maps the assessment's controls to the four RMF functions.

GOVERN (GV-6) — Third-Party, Credential, and Configuration Management

Activities:

- API key stored in `.env`, gitignored. `.env.example` ships with blank values as a template.
- Telegram bot token rotated after exposure during development.
- Container user changed from `root` to `appuser`.
- Model loading on hardened endpoint restricted to `safetensors` and `torch.load(weights_only=True)`.

Evidence:

- `.env.example` (tracked template)
- `AI_USAGE_LOG.md` (records the token rotation)
- `Dockerfile` (creates `appuser`, sets `USER`)
- `defense/secure_loader.py` (safe loading implementation)

MAP (MP-2) — Scientific Integrity

Activities:

- Threat model documented before attacks are run (Section 3).
- Attack selection criteria documented with reasons for exclusion (Section 3.4).
- Weak MIA result reported honestly (Section 6.4).
- Limitations stated explicitly (Section 11.1).

Evidence:

- Section 3 (Threat Model and Methodology)
- Section 6.4 (weak result discussion)
- Section 11 (Limitations and Residual Risk)
- `results/membership_inference/vuln/metrics.json` (the weak signal)

MEASURE (MS-2) — Evaluation of Controls and Vulnerabilities

Activities:

- Evasion: epsilon sweep on 100 images across 7 epsilon values.
- Extraction: fidelity measured on non-overlapping holdout.
- MIA: threshold sweep on 400 samples.
- Defenses: before/after comparisons for all three attacks.

Evidence:

- results/evasion/epsilon_sweep.json
- results/extraction/metrics.json
- results/membership_inference/vuln/metrics.json
- results/defense/adversarial_training/comparison.png

MANAGE (MG-2) — Risk Response

Activities:

- Adversarial training as preventive control against evasion.
- Rate limiting as preventive control against extraction.
- Confidence rounding as preventive control against MIA.
- Detector and notifier as detective controls.
- Input validator as additional preventive control.

Evidence:

- defense/adversarial_training.py
 - adapters/rate_limit.py
 - domain/entities.py (to_dict_hardened)
 - adapters/detector.py
 - scripts/notify.sh
-

Appendix C — OWASP API Security Top 10 Mapping

API4:2023 — Unrestricted Resource Consumption

Finding: 03 (no rate limit on vulnerable API).

Impact: Extraction attack can run at full speed. Also a DoS vector.

Remediation: Rate limiting (20 requests per 60 seconds per IP) on hardened endpoint.

API6:2023 — Unrestricted Access to Sensitive Business Flows

Finding: Extraction attack against the prediction endpoint.

Impact: Each prediction is a small information leak; enough predictions produce a functional copy of the model.

Remediation: Rate limiting plus detection. In production, query entitlement would be part of the API contract.

API9:2023 — Improper Inventory Management

Findings: 01 (/model-info leak), 04 (auto-docs exposed).

Impact: An attacker can enumerate the entire API surface without guessing. The /openapi.json file is machine-readable and enables automated reconnaissance.

Remediation: Both endpoint categories are disabled in hardened mode.

Appendix D — AI Usage Log Summary

The complete AI usage log is in AI_USAGE_LOG.md. This appendix summarizes the record.

Tools Used

Tool	Model	Sessions	Primary use
DeepSeek	V3	5	Architecture review, attack scripts, debugging, framework mapping

Tool	Model	Sessions	Primary use
Claude	Sonnet	3	Script review (training, extraction, adversarial training)
Google Colab	T4 GPU	2	Baseline model training, membership inference runs
Google Colab	CPU	1	Robust model training

Contribution Boundary

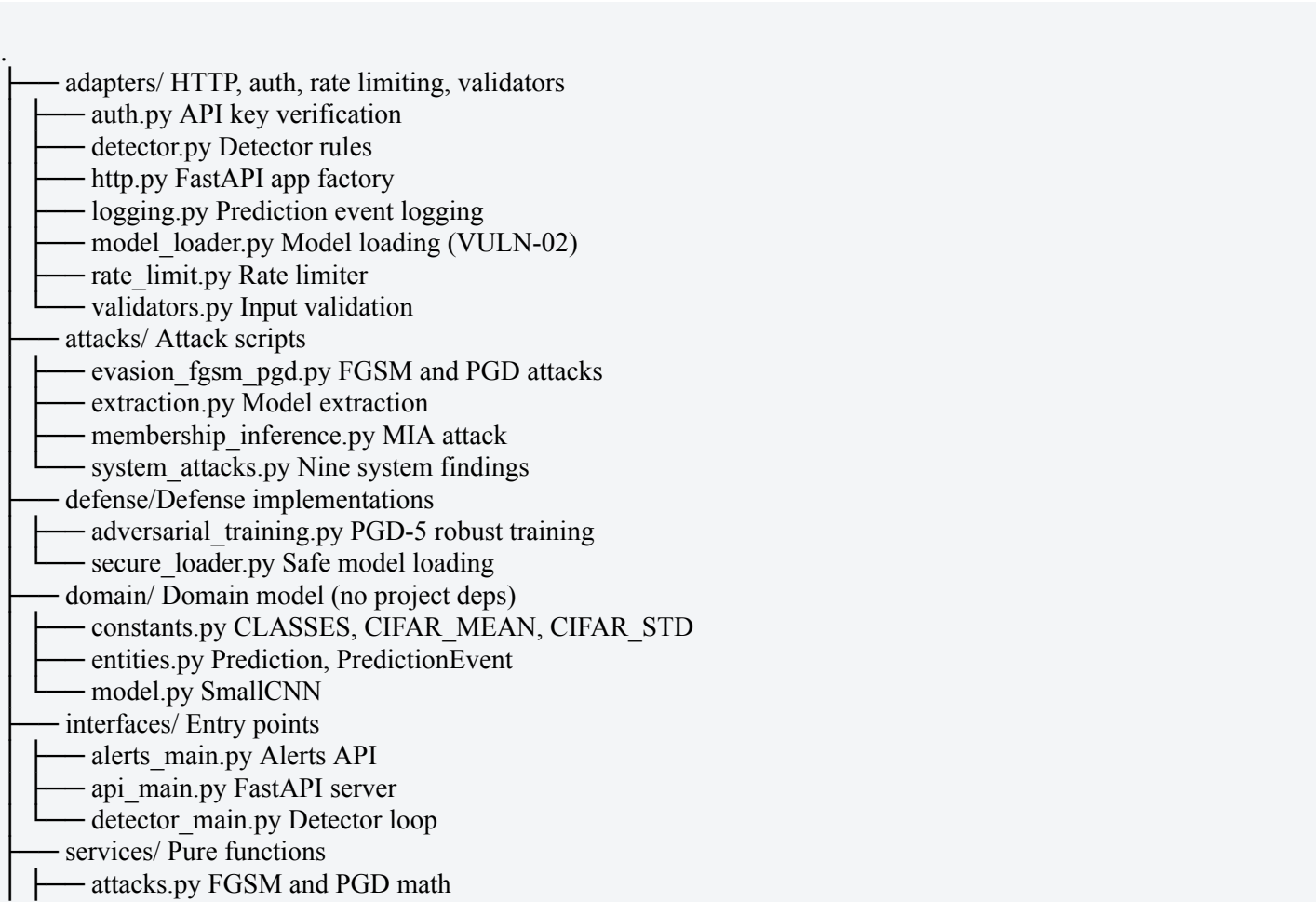
The usage log distinguishes between the author's contribution and the AI's contribution in every task. The summary:

The architecture is the author's. The five-layer clean architecture, the four-container topology, the two security modes, the vulnerability tagging scheme, the detector rules and thresholds, the hardening controls, and the notification topology were designed by the author. AI reviewed the design and wrote first drafts to specification.

AI did not execute. No AI tool had shell access, executed attacks, or modified the repository. Every command was typed, every commit was made, and every test was run by the author.

AI did not skip verification. Every AI suggestion was reviewed against the requirements or the relevant paper before adoption. Where AI proposed something that conflicted with the project's goals — such as `weights_only=True` in the vulnerable loader — the suggestion was rejected and the reasoning documented.

Appendix E — Repository Structure



- inference.py Inference
- preprocessing.py Image preprocessing
- usecases/ Orchestration
 - serve_prediction.py
- scripts/ Operational scripts
 - notify.sh Telegram notifier
 - notify_only.sh Send-only demo runner
 - test_all.sh Runs all test suites
 - test_model_attacks.sh Model attack tests
 - test_notifier.sh Notifier scenarios
 - test_system_attacks.sh System finding tests
 - verify.sh Deployment verification
- model/artifacts/ Model weights and configs
 - baseline.pt Baseline weights
 - baseline_robust.pt Robust weights
 - baseline_robust.safetensors Safe format
 - config.json Baseline config
 - config_robust.json Robust config
 - curve.json Baseline training curve
 - robustness_curve.json Robust training curve
- results/ Measured outputs
 - alerts.jsonl Detector alerts
 - defense/ Before/after comparisons
 - evasion/ Evasion results
 - extraction/ Extraction results
 - membership_inference/ MIA results
 - system/ Nine findings evidence
- docs/diagrams/ PlantUML sources and SVGs
- colab/ Training notebooks
- docker-compose.yml Five-container topology
- Dockerfile Main image
- Dockerfile.notifier Notifier image
- Makefile Convenience targets
- requirements.txt Python dependencies
- AI_USAGE_LOG.md Complete AI usage record
- README.md Entry point for the repository

Appendix F — Attack and Defense Matrix

This appendix presents the assessment's coverage in a single matrix.

Attack	ATLAS	Detector Rule	Preventive Defense	Residual Risk
Evasion (FGSM/PGD)	T0043	`adversarial_noise`	Adversarial training + input validator	35% of PGD attacks succeed on robust model
Extraction	T0024	`extraction_volume`	Rate limiting	Per-IP in memory; botnet bypasses
Membership inference	T0025	`mia_confidence_sweep`	Confidence rounding	0.5575 vs 0.50 baseline; signal persists

Attack	ATLAS	Detector Rule	Preventive Defense	Residual Risk
Unsafe deserialization	T0010	—	Safe loader (safetensors)	None on hardened; open on vulnerable by design

Coverage counts:

Metric	Value
Model-level attacks demonstrated	3
System-level findings	9
Findings remediated or mitigated	5
Detector rules	3
Preventive defenses	6
ATLAS techniques covered	6
RMF functions exercised	4
OWASP categories addressed	3
Test assertions (all scripts)	106